6 The Transport Layer

(Around 6 hours)

Outline

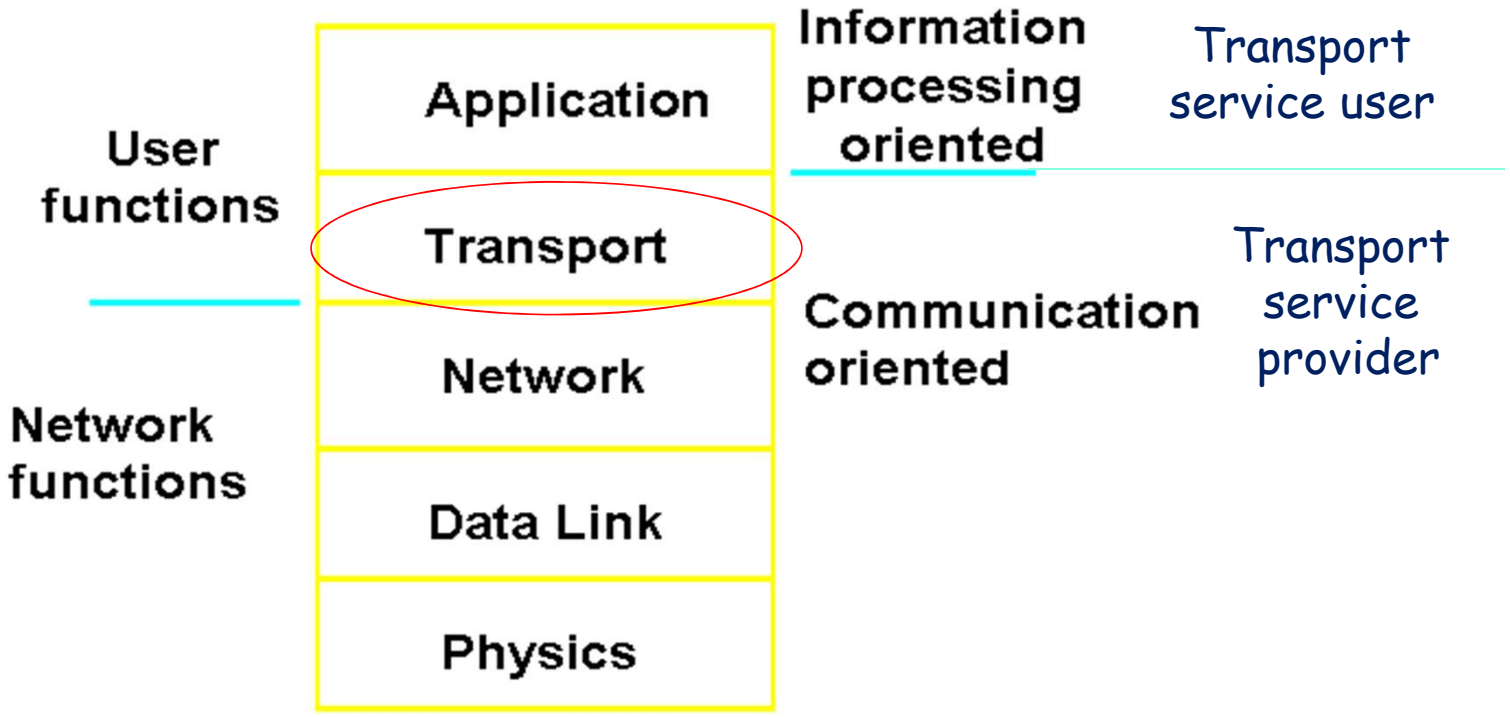
- *Function and Services to Transport Layer*
- Elements of Transport Protocols
  - Addressing
  - Connection Establishment and Release
  - Flow Control and Buffering
  - Congestion Control
- The Transport Layer in Internet
  - User Datagram Protocol (UDP)
  - Transmission Control Protocol (TCP)

Motivation of Transport Layer

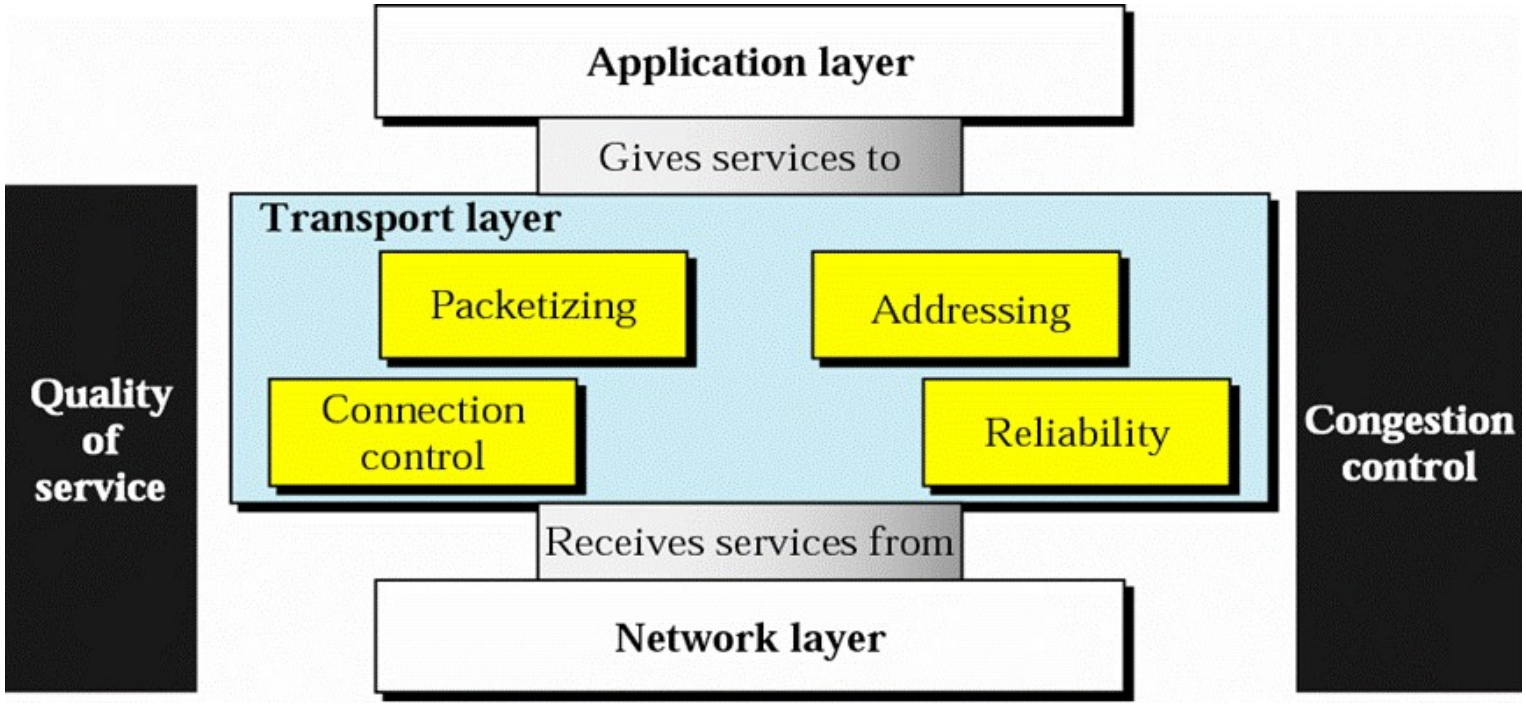
- IP provides a weak, but efficient service model (*best-effort*)
  - Packets may be delayed, dropped, reordered, duplicated
  - Packets have limited size
- IP packets are addressed to a host
  - How to decide which application gets which packets?
- How should hosts send packets into the network?
  - Too fast may overwhelm the network
  - Too slow is not efficient

Position of Transport Layer

- Heart of the whole protocol hierarchy.



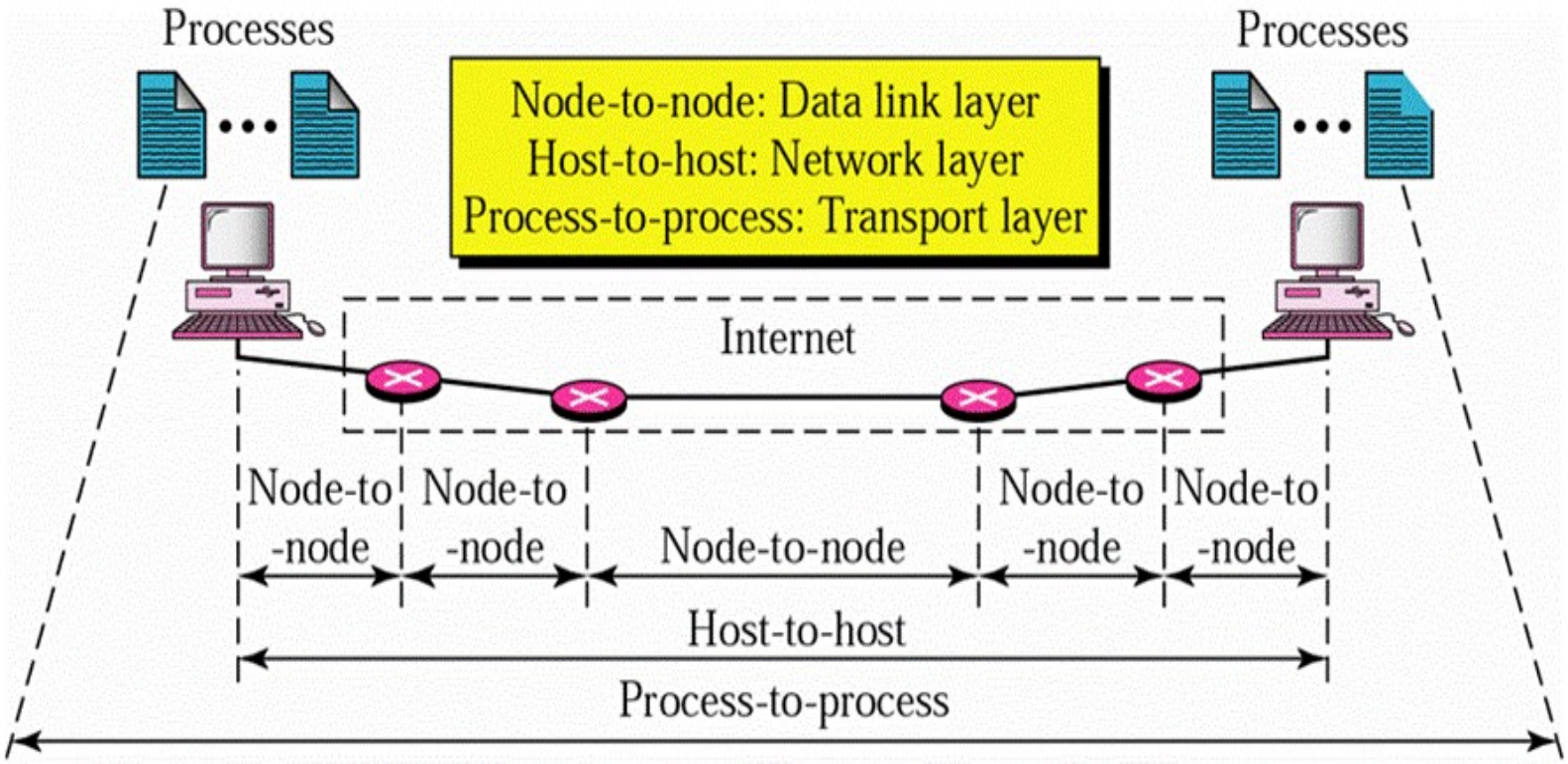
Functions of Transport Layer



- Providing efficient, reliable, cost-effective data transport from the source to the destination, independently of the physical network or networks currently in use.

[Forouzan]

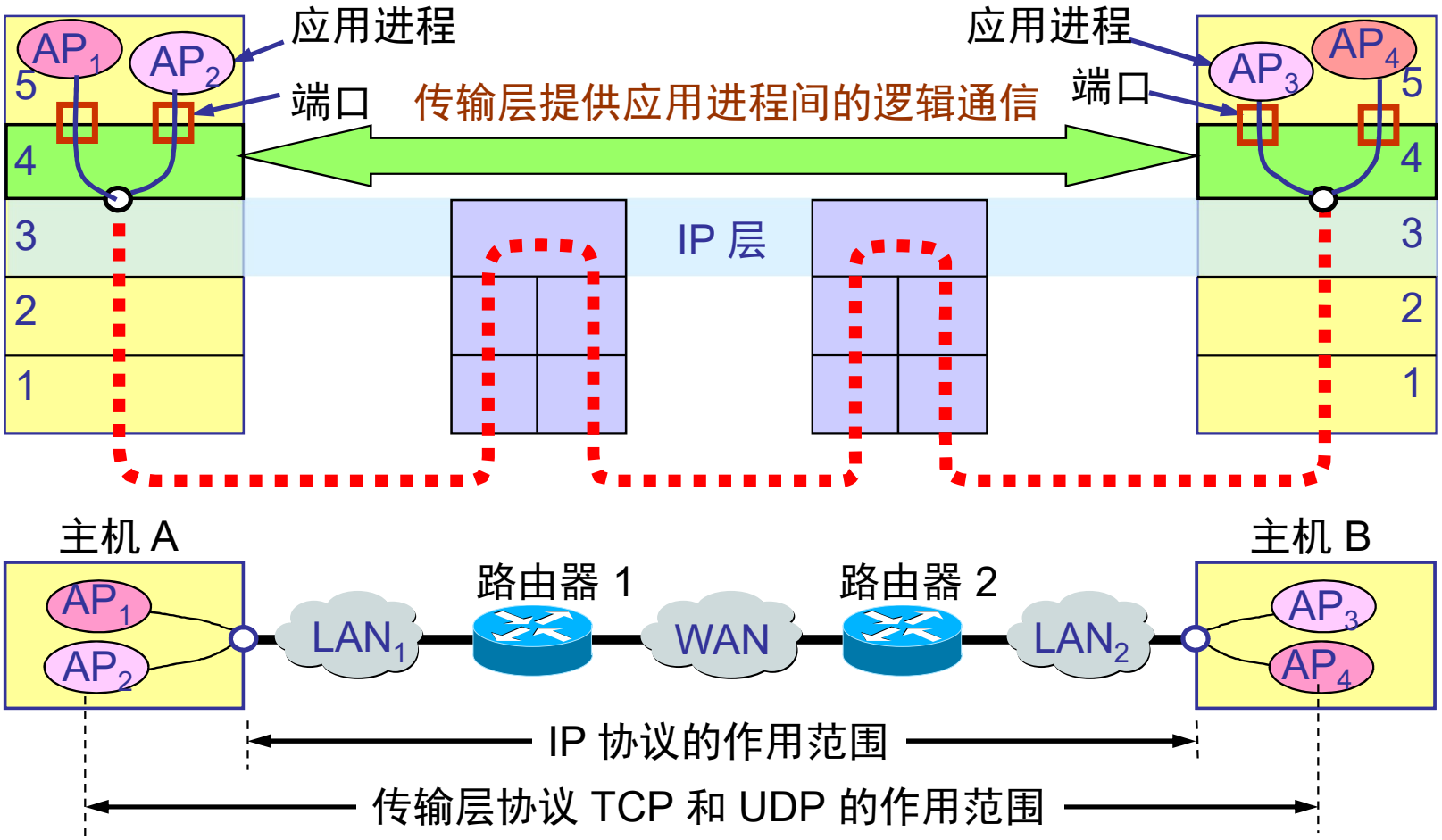
Process-to-Process Delivery



进程-进程

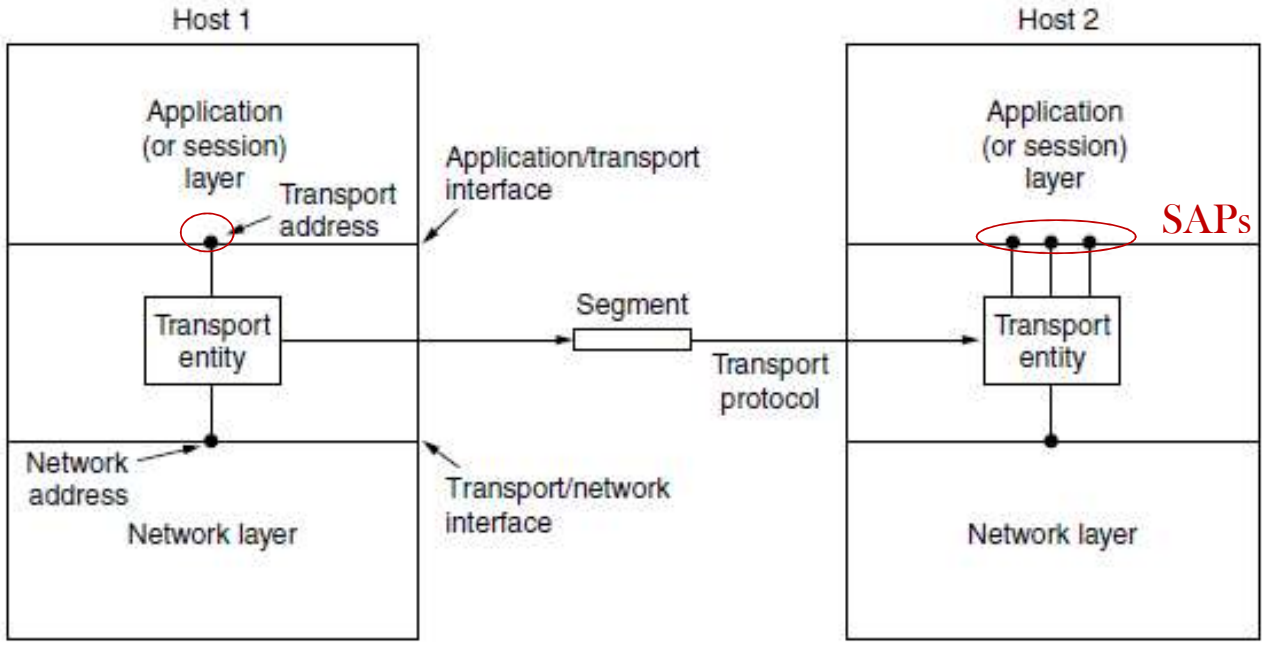
[Forouzan]

Scope of Transport Layer



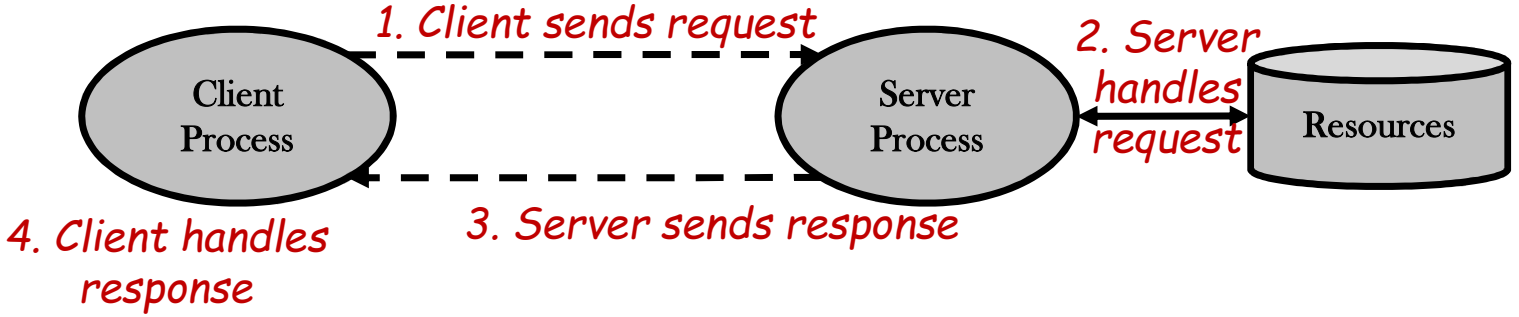
Services of Transport Layer

- Masking the diversity of communication subnets and providing a common interface to the upper layer
- Providing multiple SAPs(Service Access Points) sharing one network link
- Connection-oriented service vs. connectionless service



Client-Server Model

- Used by almost all network applications
- Server
  - Manages some resources and provides service by manipulating resources for clients
  - Begins execution first
  - Waits passively at prearranged location
- Client
  - Begins execution later
  - Actively initiates contact with a server



Transport Service Primitives(传输服务原语)

- 提供给应用进程的接口（API）
- Example of a simple connection-oriented service

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	Request a release of the connection

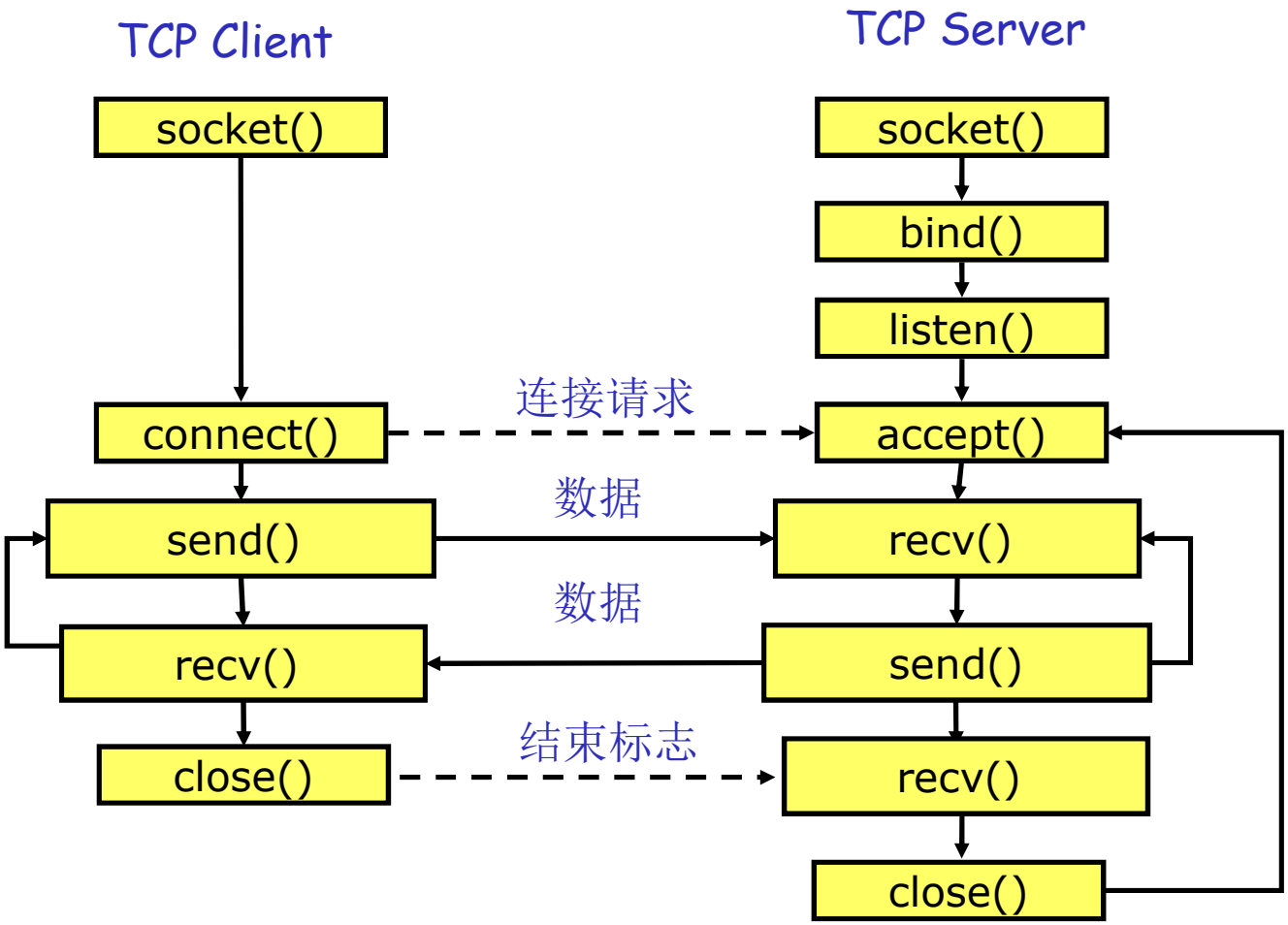
Berkeley Sockets（套接字）

- Widely used API in Internet

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Figure 6-5. The socket primitives for TCP.

基于TCP的Socket程序流程

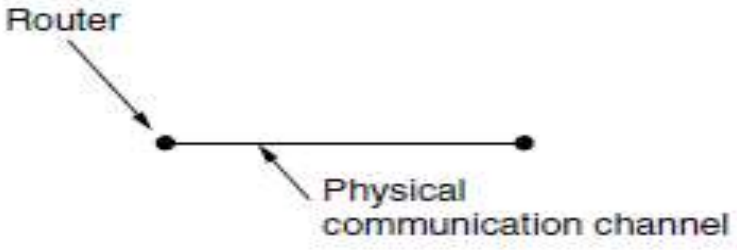


Outline

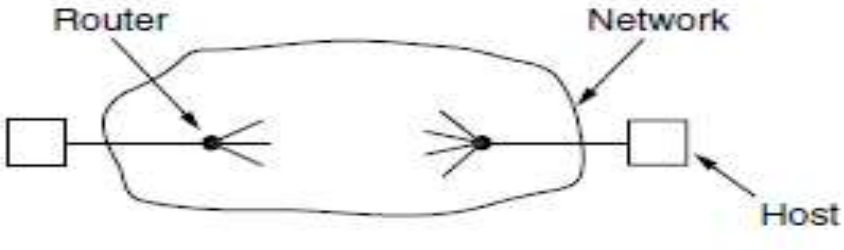
- Function and Services to Transport Layer
- *Elements of Transport Protocols*
  - Addressing
  - Connection Establishment and Release
  - Flow Control and Buffering
  - Congestion Control
- The Transport Layer in Internet
  - User Datagram Protocol (UDP)
  - Transmission Control Protocol (TCP)

Transport Protocol vs. Data Link Protocol

- Both have to deal with error control, sequencing, flow control (using ARQ)
- Node-to-node delivery vs. Process-to-process delivery
- Different Environments
  - Addressing of destinations
  - Initial connection establishment
  - Storage capacity in the subnet
  - Allocation of buffers



Data Link Layer: link in between



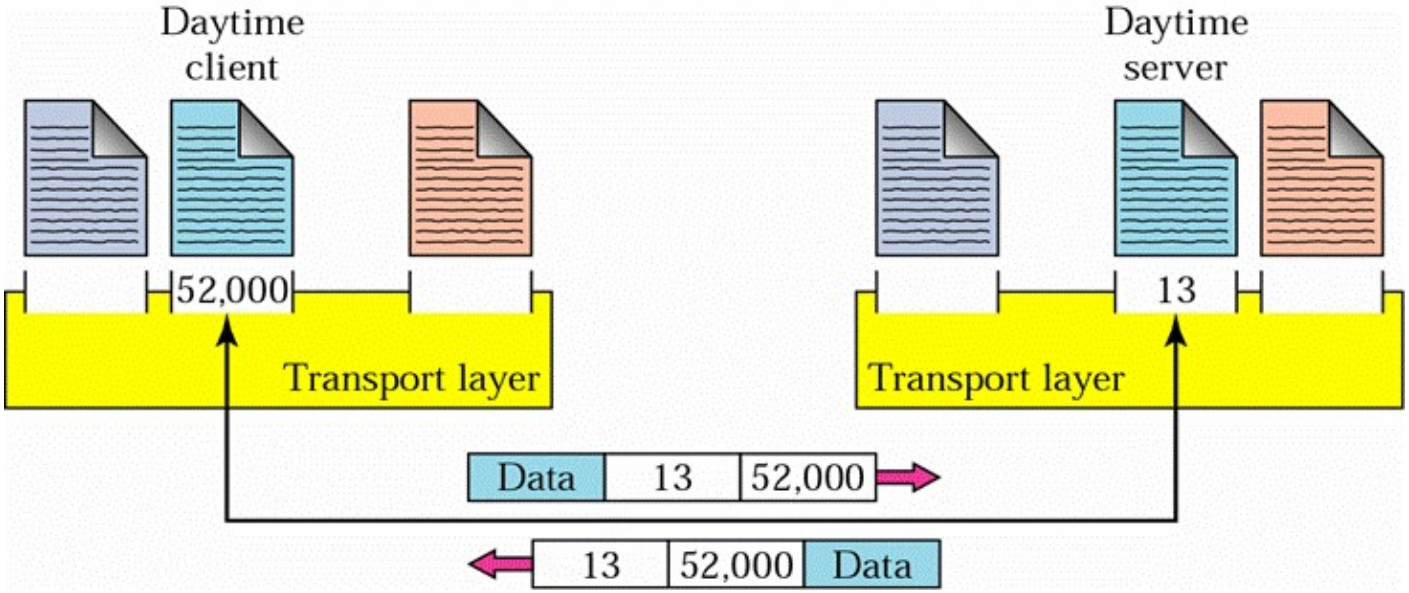
Transport Layer: network(s) in between

Process-to-Process Delivery involving

- Addressing
- Multiplexing and Demultiplexing
- Connectionless/Connection-Oriented
- Reliable/Unreliable
- Flow control and Buffering
- Congestion control

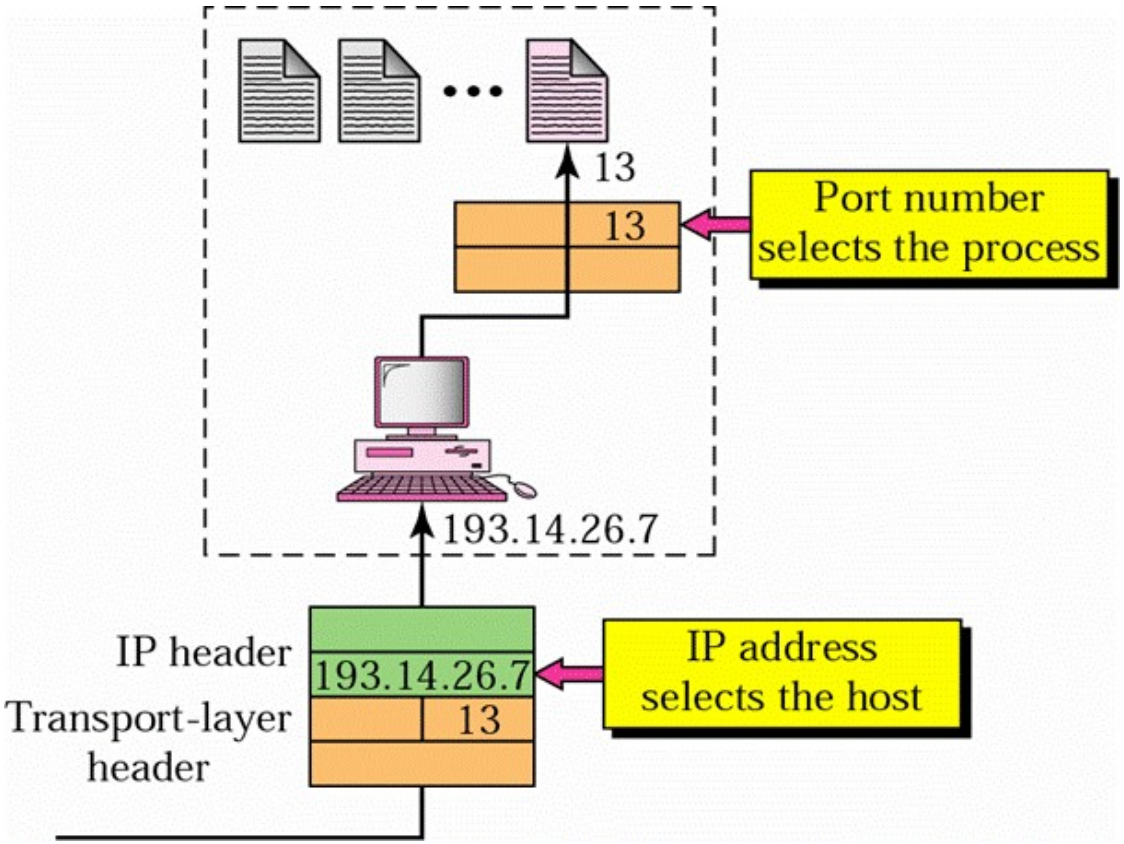
Addressing: Port Number

- SAP of the application process
- Deciding which application gets which messages



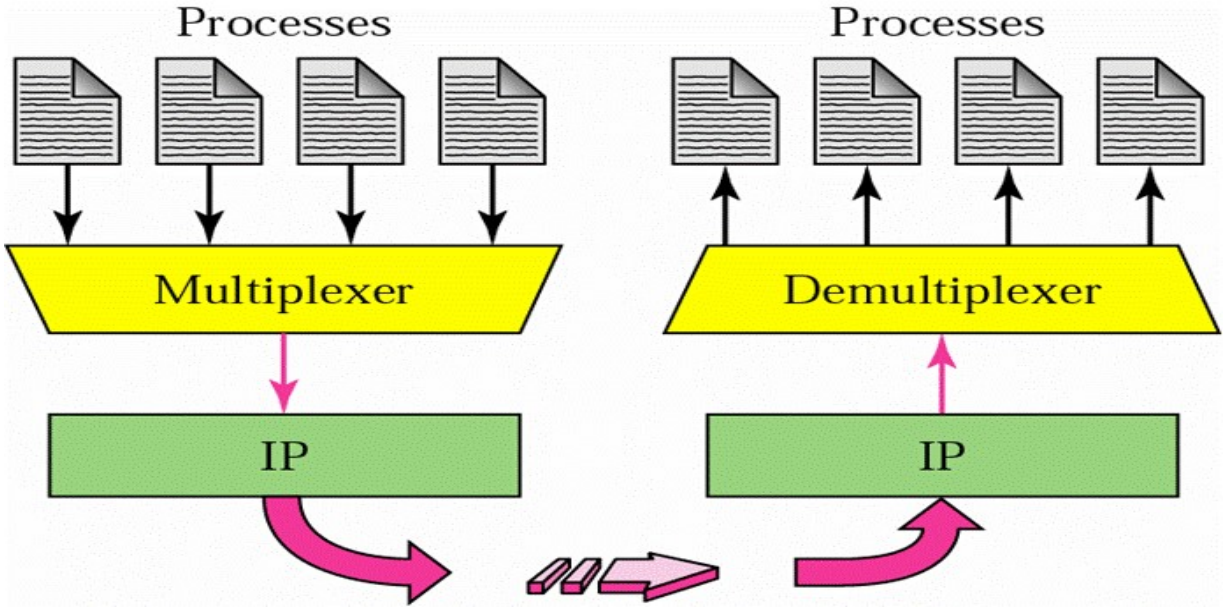
[Forouzan]

IP Address vs. Port Number



[Forouzan]

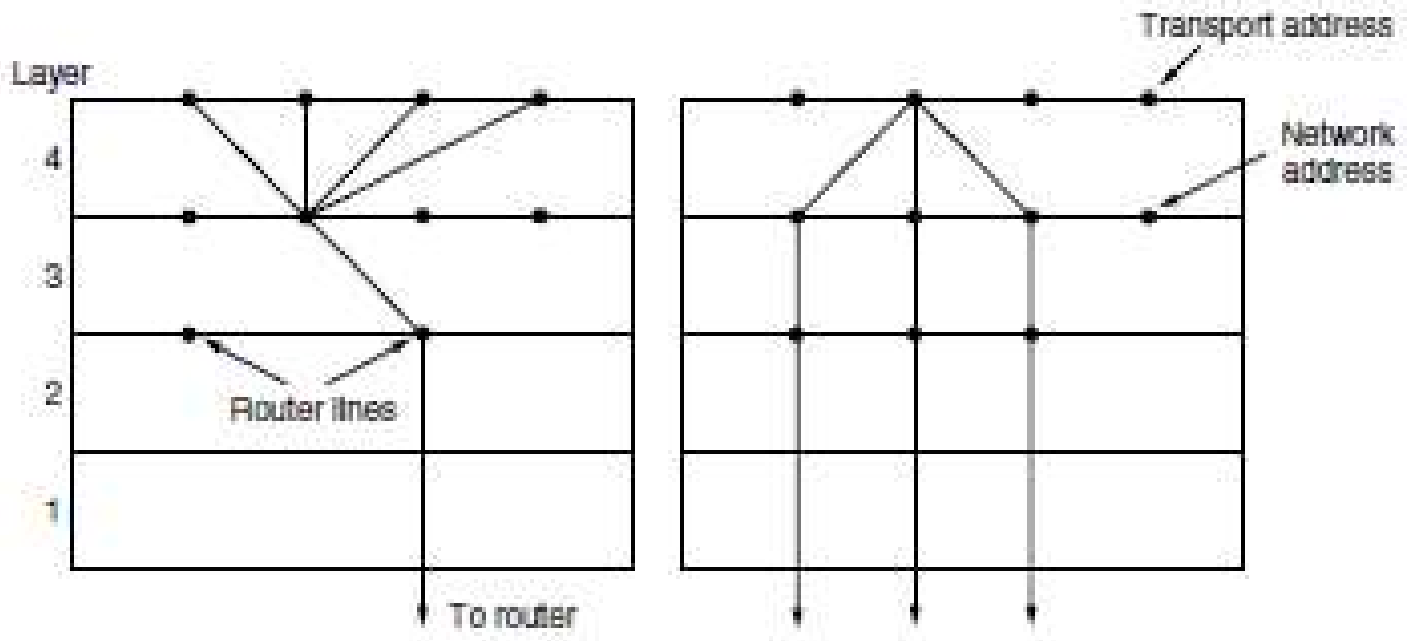
Multiplexing(复用)



- At sender site, multiplexing at transport layer accepts packets from different processes, differentiated them by port numbers.
- At the receiver site, demultiplexing process works. Transport layer delivers each message to appropriate process based on port number.

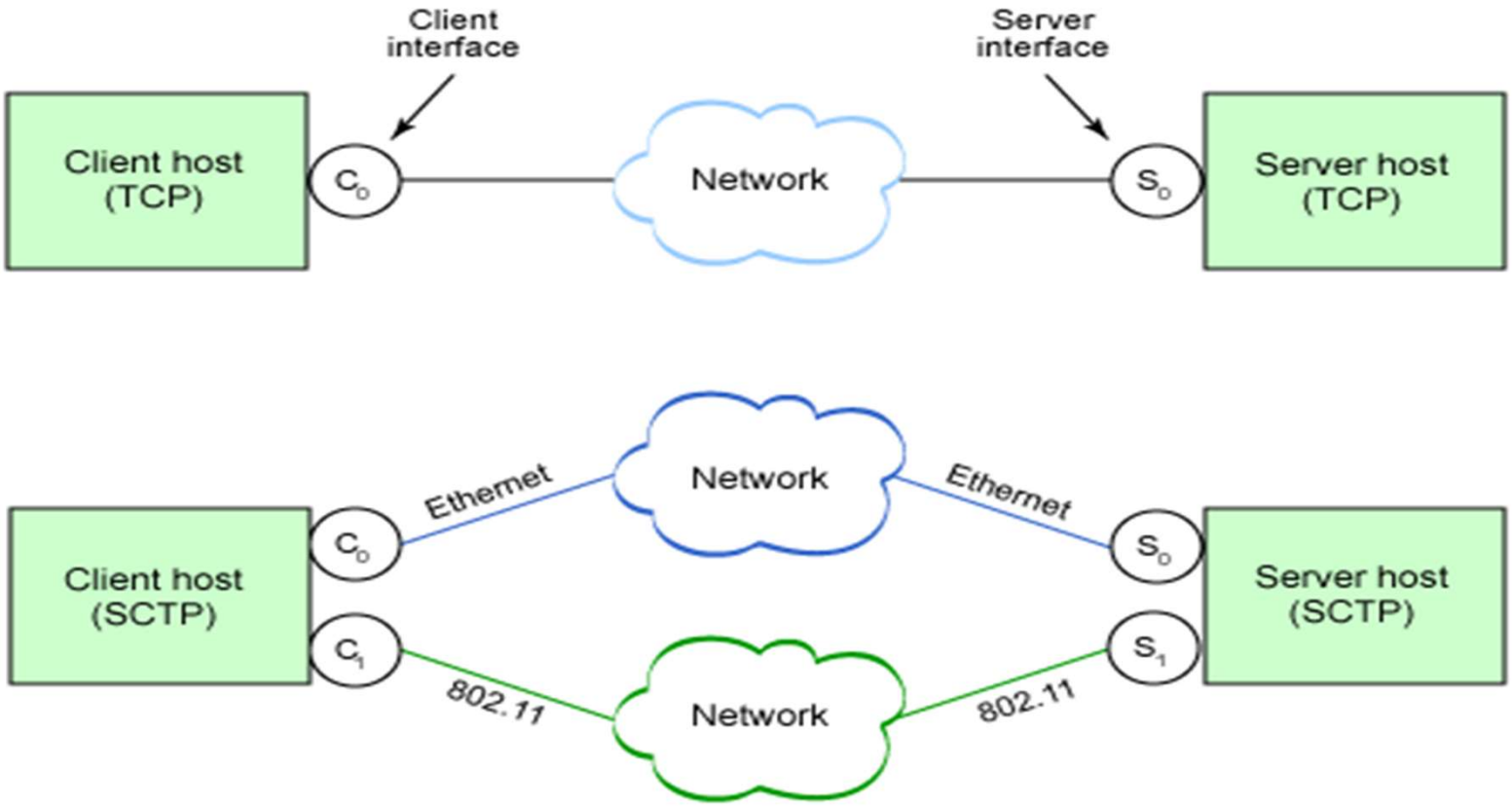
[Forouzan]

Inverse Multiplexing (Fig.6-17)



- Used by SCTP (Stream Control Transmission Protocol)
- RFC 4960
- A connection using multiple network interfaces

TCP connection vs.
SCTP association(SCTP联合)



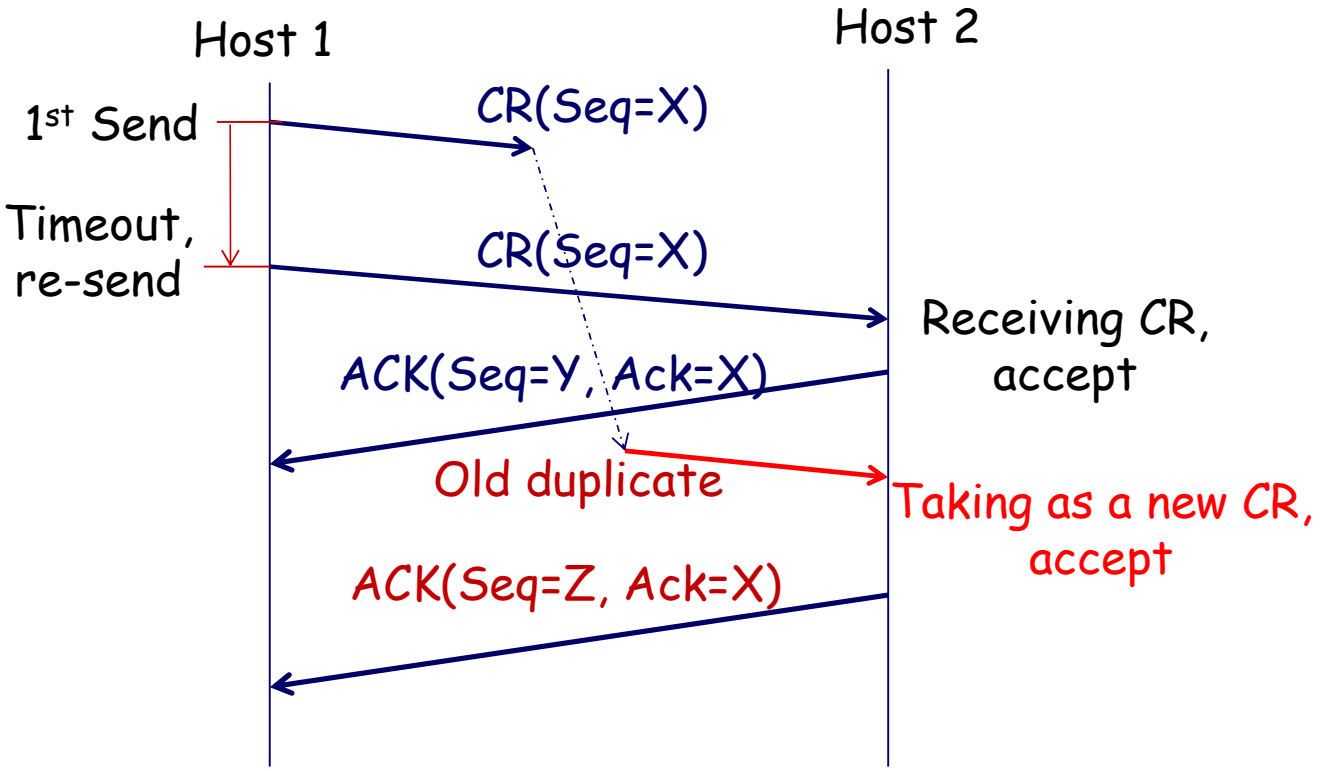
https://www.ibm.com/developerworks/library/l-sctp/

Connection Establishment: Problems

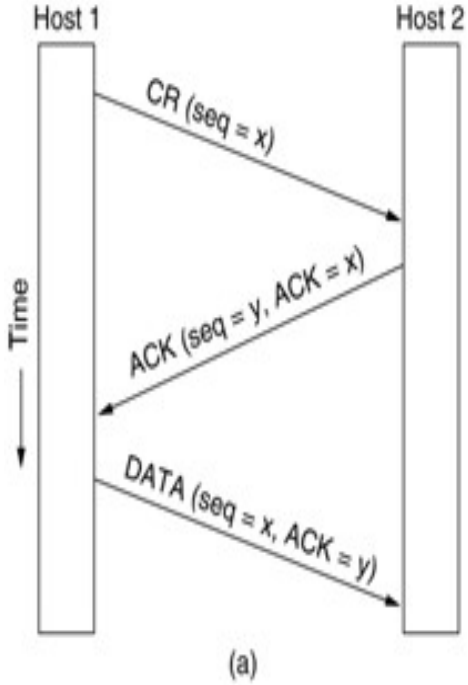
- Establishing a connection sounds easy
  - 2-way handshake （两次握手）
  - One sends a ConnectionRequest to the destination and wait for a ConnectionAccepted reply
- Problems
  - Network may delay packets, causing duplicate TPDU (Transport layer PDU)
  - Receiver can not distinguish the duplicate and regard as a new one

Problem of Delayed Duplicate CR

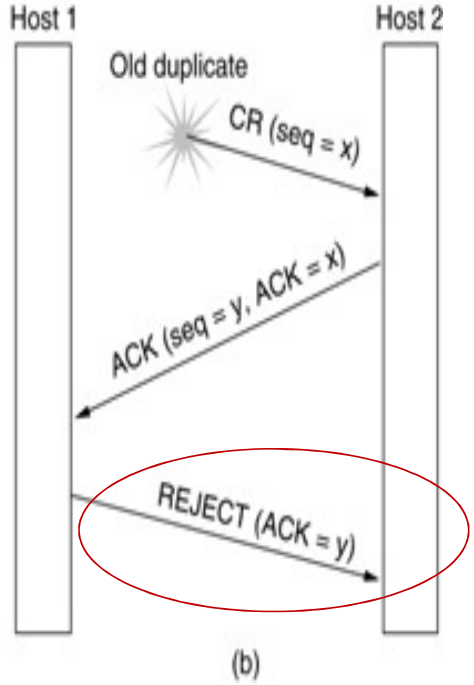
- Connection Request(CR) TPDU was delayed and retransmitted , causing Host2 to regard the duplicate CR as a new one



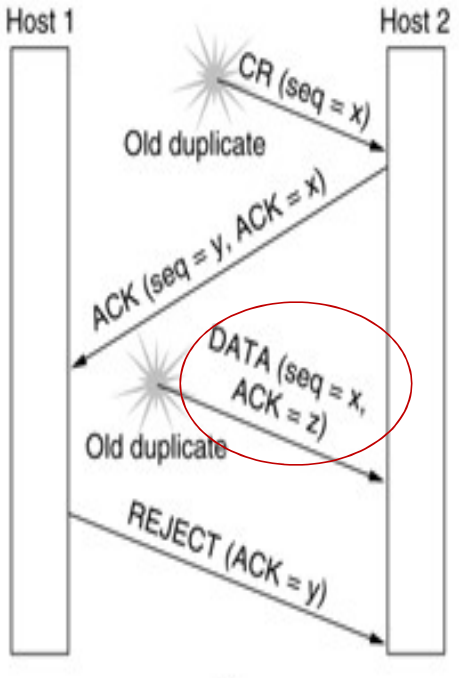
Solutions: 3 Way Handshake(三次握手)



Normal operation



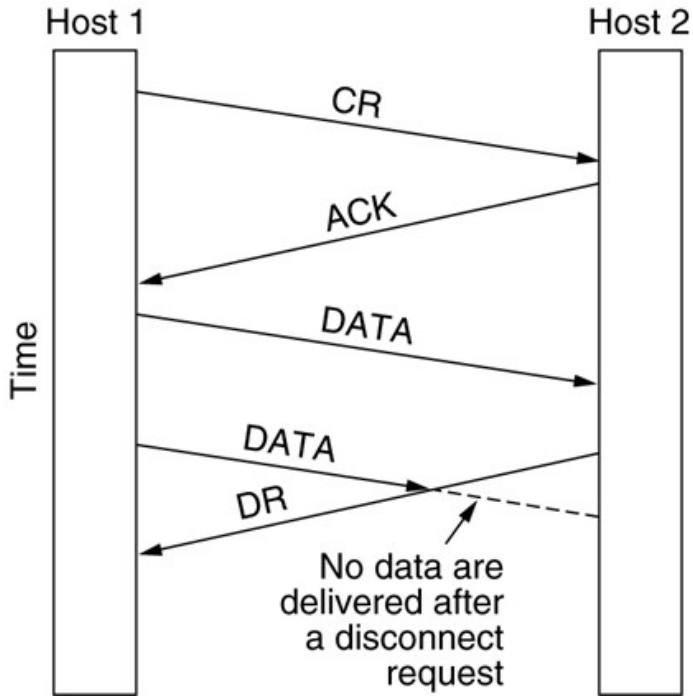
Duplicate
Connection Request



Duplicate
Connection Request
and
duplicate ACK

Connection Release: Problem

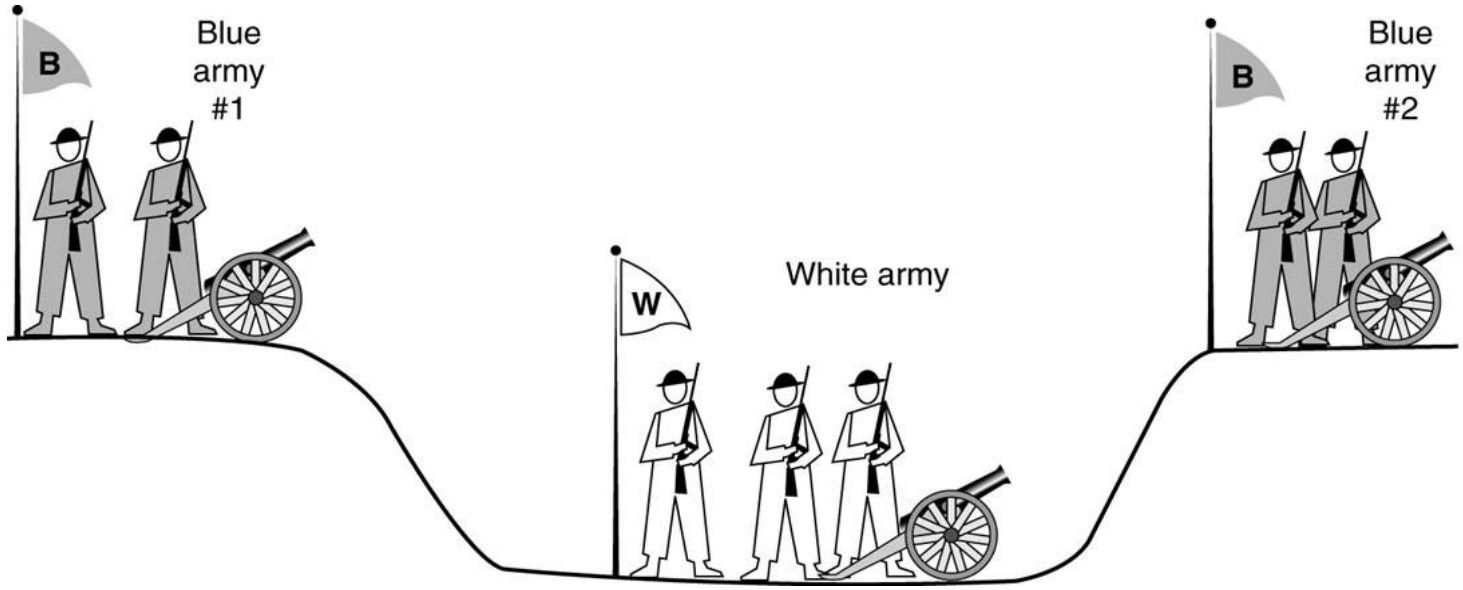
- Asymmetric release：abrupt disconnection(突然释放), may cause data loss



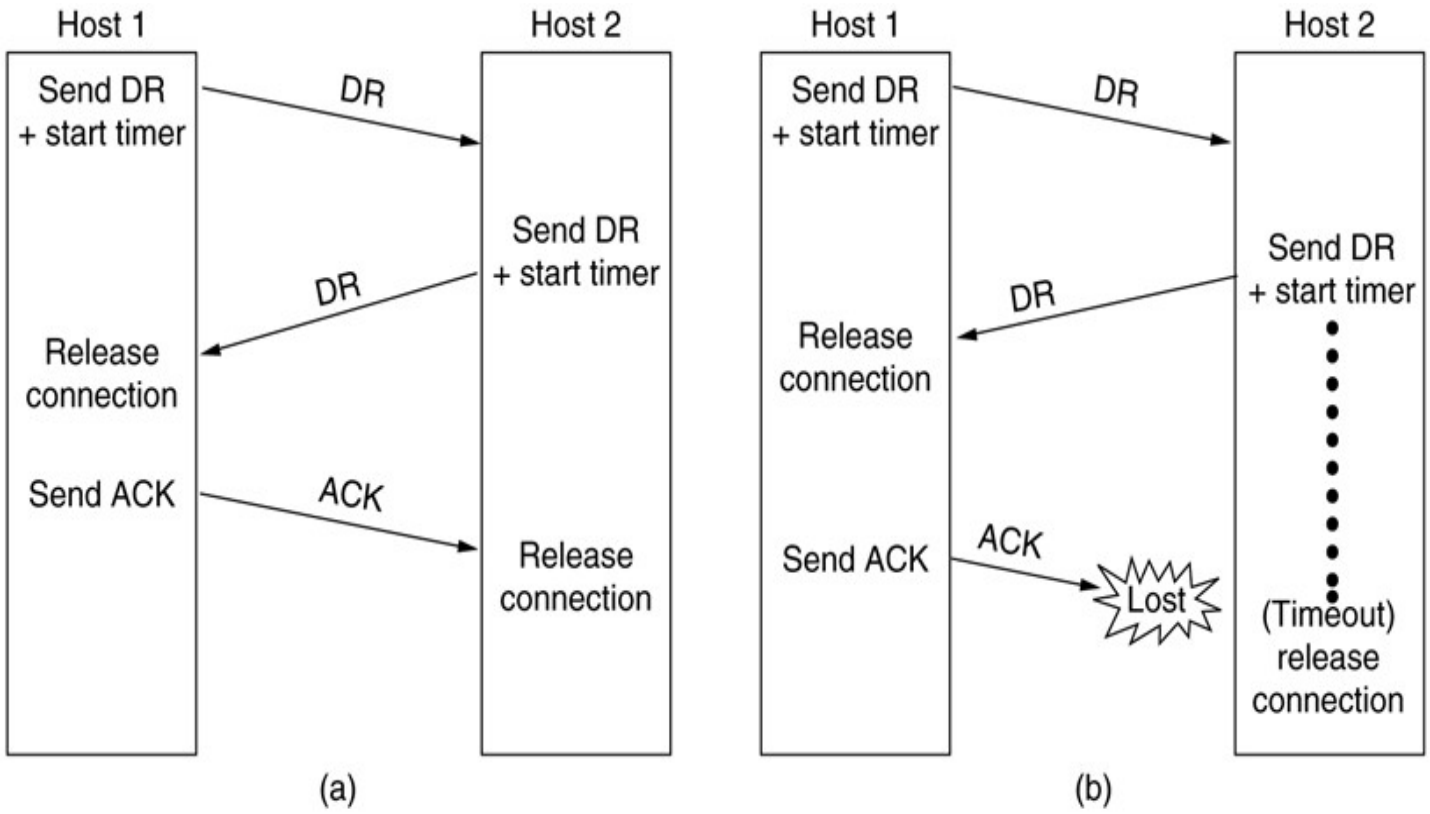
- Symmetric release, in which each direction is released independently of the other one.

Connection Release: 2 Army Problem

- "Two blue armies need to simultaneously attack the white army to win; otherwise they will be defeated.
- The blue army can communicate only across the area controlled by the white army which can intercept the messengers."



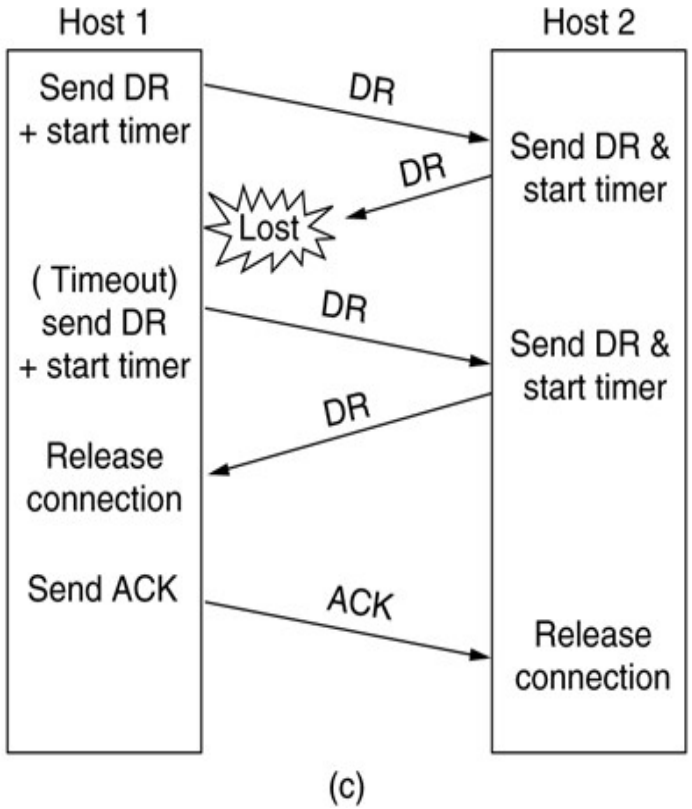
Solution: 3 Way Handshake(1)



Normal operation

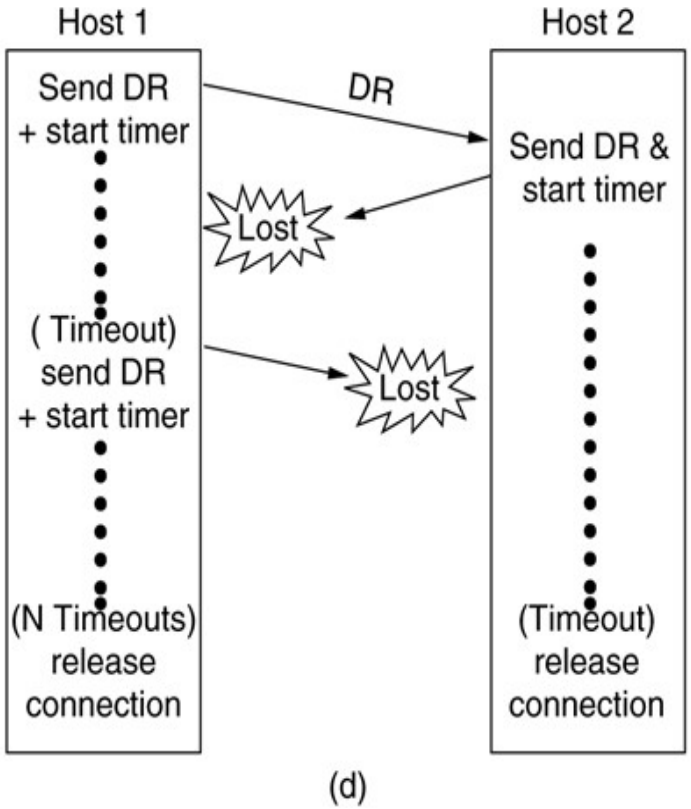
Final ACK lost

3 Way Handshake(2)



(c)

Response lost



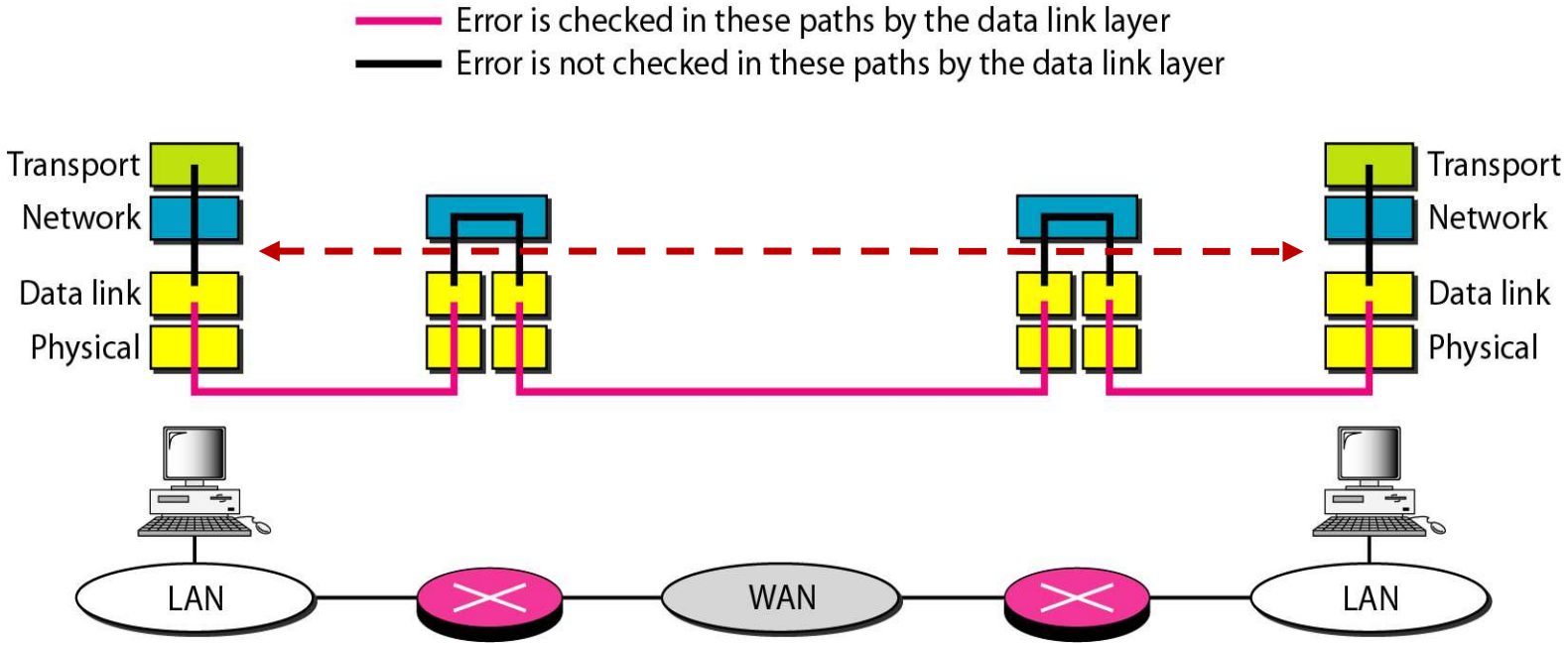
(d)

Response lost and subsequent DRs lost

Error Control

- ARQ(Automatic Repeat reQuest)
  - A segment carries an error-detecting code, a checksum
  - A segment carries a sequence number to identify itself and is retransmitted by the sender until it receives an acknowledgement of successful receipt from the receiver
  - Sliding window protocol combines the features and is also used to support bidirectional data transfer.
- End-to-end argument(端到端/进程-进程)

Error Control: Transport Layer vs. Data Link Layer

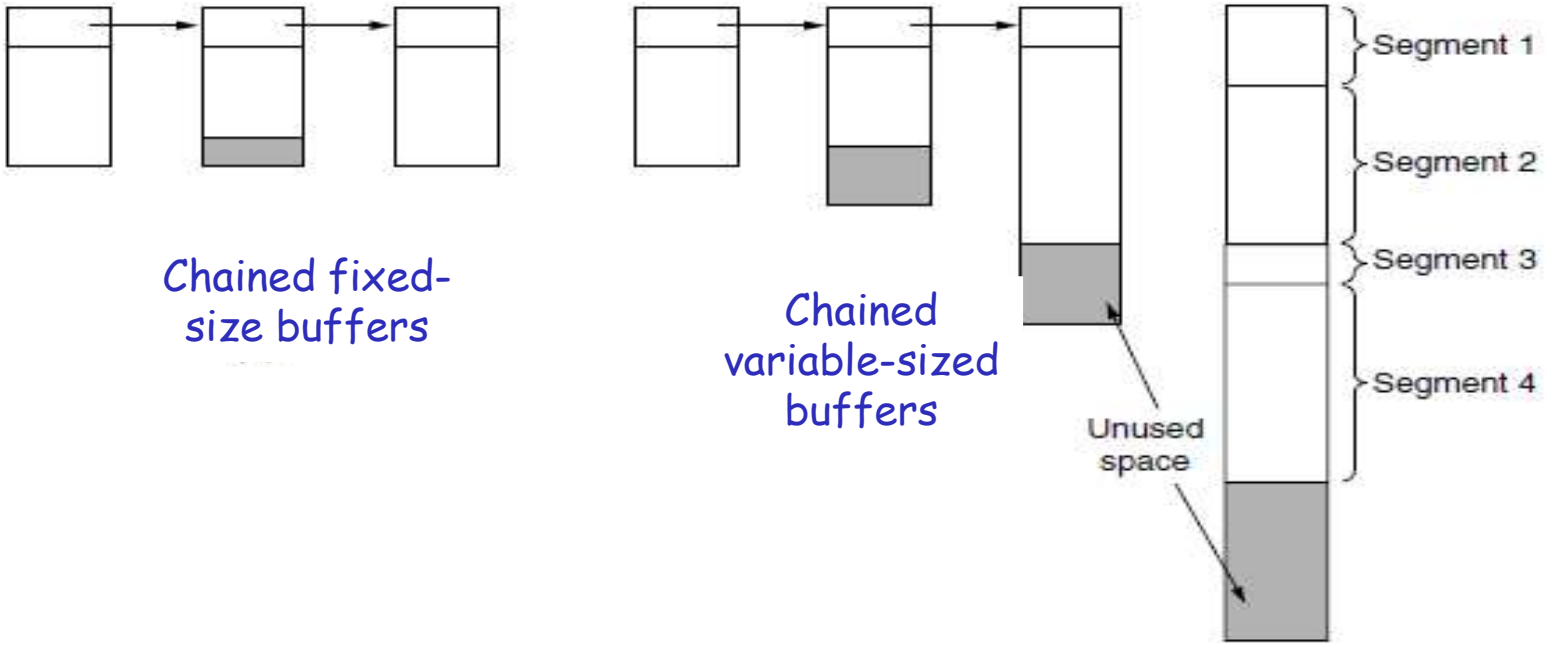


- Difference in function
  - Reliability crossing a single link vs. Reliability crossing an entire network path
- Difference in degree
  - Small window size vs. much larger window size(complex)

Buffering Trade-off

- Source buffering or destination buffering?
  - Depending on the type of traffic carried by the connection
- For low-bandwidth bursty traffic, such as BBS, it is reasonable not to dedicate any buffers, but rather to acquire them dynamically at both ends, relying on buffering at the sender.
- For file transfer and other high-bandwidth traffic, the receiver shall does dedicate a full window of buffers, to allow the data to flow at maximum speed.
  - TCP uses this strategy.

How to organize the buffer pool



Flow Control: Dynamic Buffer Allocation

	A	Message	B	Comments
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	• • •	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	• • •	<ack = 6, buf = 4>	←	Potential deadlock

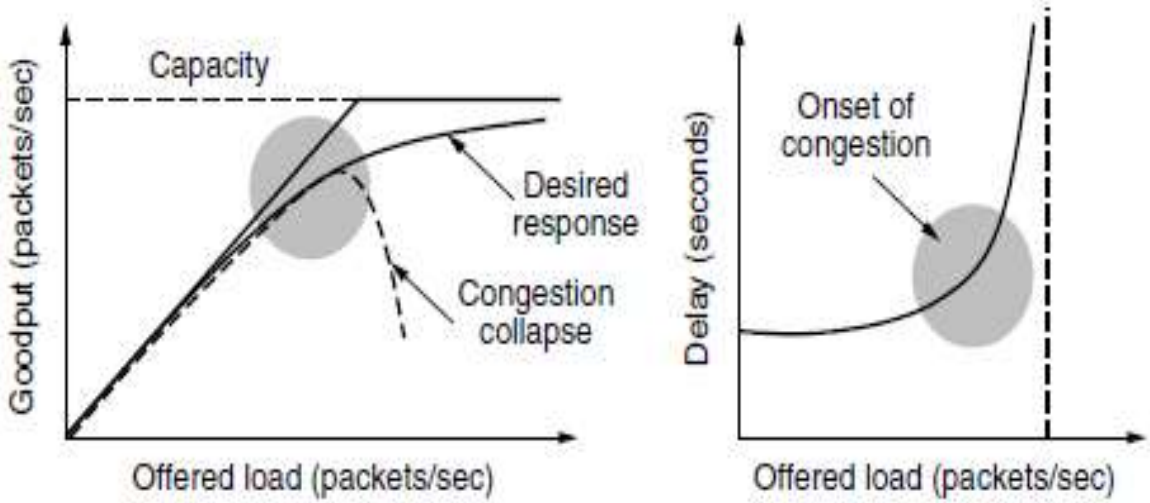
- The arrows show the direction of transmission
- An ellipsis (...) indicates a lost segment

What is Congestion?

- If the transport entities on many machines send too many packets into the network too quickly, the network will become congested
- Symptoms:
  - Lost packets
  - Long delays
- Efficiency and Power

$$power = \frac{load}{delay}$$

衡量拥塞程度的指标

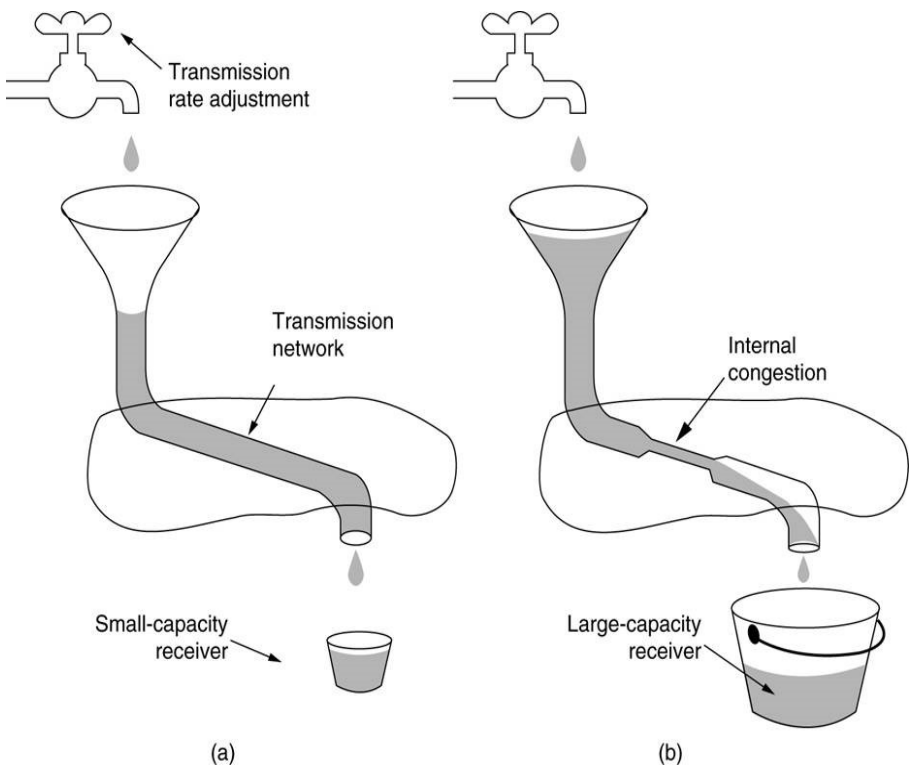


The load with the highest power represents an efficient load for the transport entity to place on the network. （最高效）

Congestion Control

- Mechanisms and techniques to control the congestion and keep the load below the capacity
  - Either prevent congestion before it happens, or remove congestion after it has happened
- Different from flow control

Common solution:
Regulating the sending rate



Flow control

Congestion control

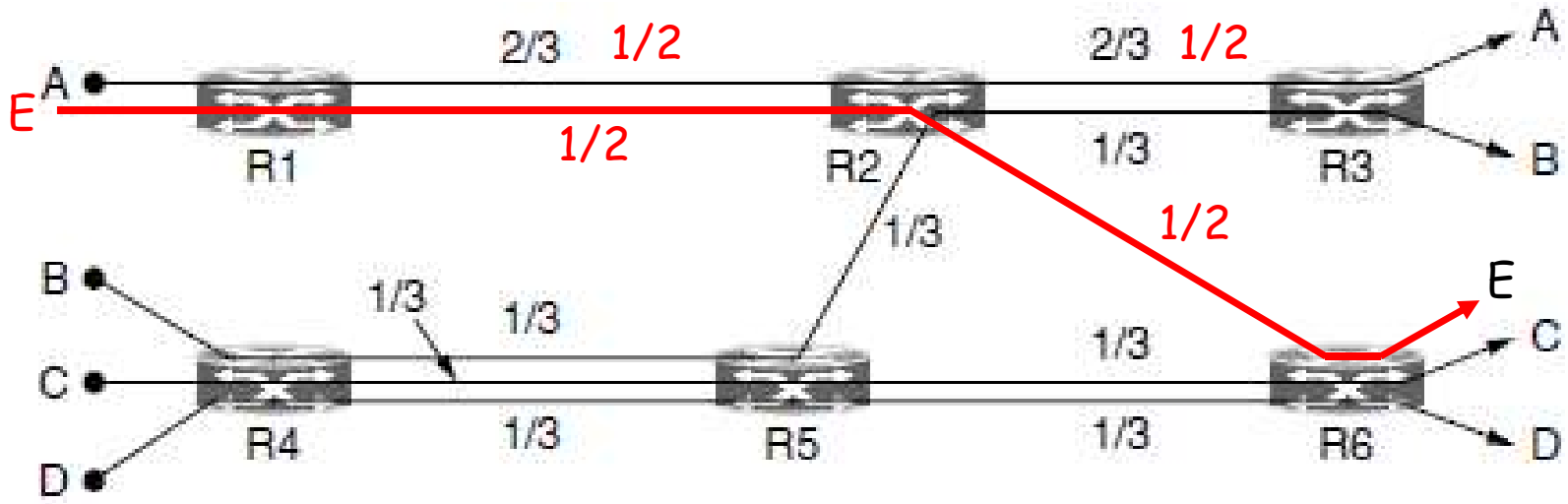
Congestion Control (cont.)

- Responsibility of the network and transport layers
  - Congestion occurs at routers, so it is detected at the network layer
  - The only effective way is for the transport protocols to send packets into the network more slowly
- Internet relies heavily on the transport layer for congestion control (TCP)
- Goals of Congestion Control Algorithm
  - Avoid congestion
  - Find a good allocation of bandwidth to use all the available bandwidth (Efficiency)
  - Be fair across competing transport entities (Fairness)
  - Quickly track changes in traffic (Convergence)

How to divide bandwidth between transport senders fairly?

- Max-Min Fairness(最大最小公平性)
  - If and only if the allocation is feasible and an attempt to increase the allocation of any participant necessarily results in the decrease in the allocation of some other participant with an equal or smaller allocation.
  - 基本含义：使得资源分配向量的最小分量的值最大，防止任何网络流被‘饿死’，同时在一定程度上尽可能增加每个流的速率。
  - 原则：一个信道上，在满足最小需求的前提下，各流尽量均分带宽；如果增加任何一个参与者（分量）的带宽，将导致其它的具有相同或者更少带宽的参与者（分量）的带宽下降，此时即满足了最大最小公平性。

Example of Max-Min Fairness



A-D: 4 flows

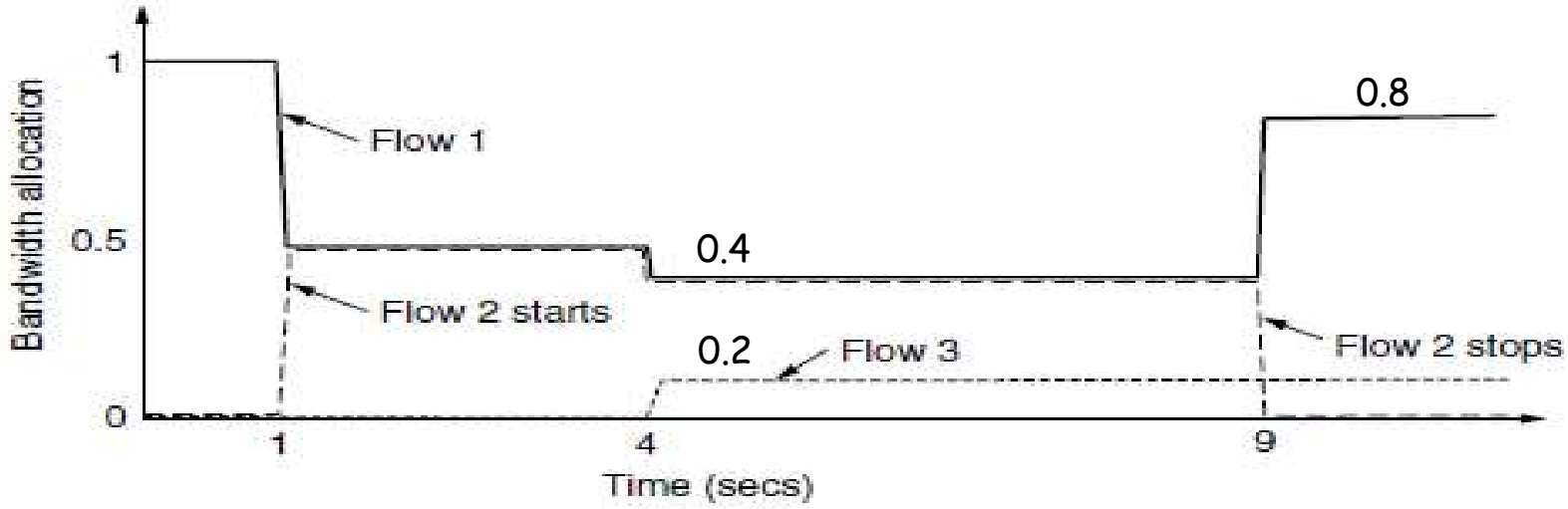
Each link has the same capacity, 1 unit(单位1)

如果增加B的带宽，将导致与B具有相同带宽的C和D的带宽减少，因此当前分配已满足Max-Min Fairness

Q: 假如加入一个新的流E，路径E->R1->R2->R6，5个流的带宽？

Convergence(收敛性)

- A good congestion control algorithm should rapidly converge to the ideal operating point(a fair and efficient allocation of bandwidth), and it should track that point as it changes over time.（尽快达到理想分配）



At all times, the total allocated bandwidth is approximately 100%, so that the network is fully used, and competing flows get equal treatment

Congestion Control in Internet

- 三步走

1. 发现拥塞：路由器根据队列长度或者线路利用率判断
2. 将拥塞情况通知源主机

   显式拥塞通知：TCP with ECN

   或 隐式拥塞通知：源主机(TCP)自己根据丢包情况判断
3. 源主机采取措施（TCP）

   在无拥塞时，增加发送速率；

   在出现拥塞时，降低发送速率

Feedback from Network

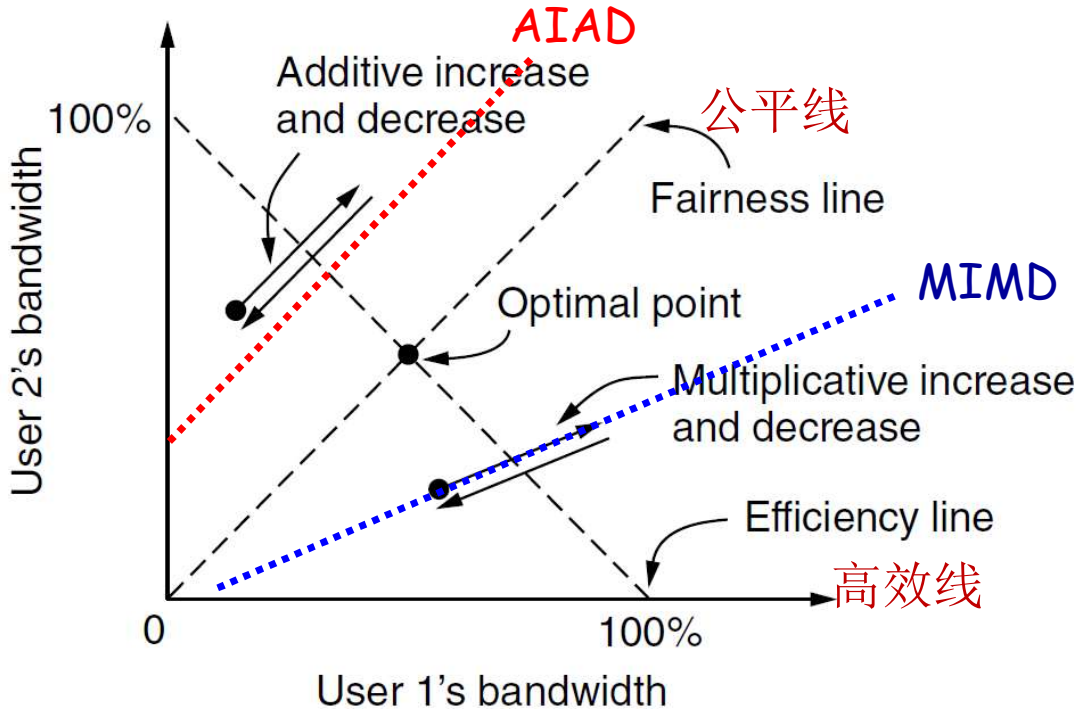
Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
Compound TCP	Packet loss & end-to-end delay	No	Yes
CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

- How to regulate the Sending Rate
  - In the absence of a congestion signal, the senders should increase their rates.
  - When a congestion signal is given, the senders should decrease their rates.
  - Details to be discussed in TCP

Control Law: AIMD(加法增乘法减)

- Motive: to arrive at the efficient and fair operating point

Assumed two connections competing for the bandwidth of a single link.



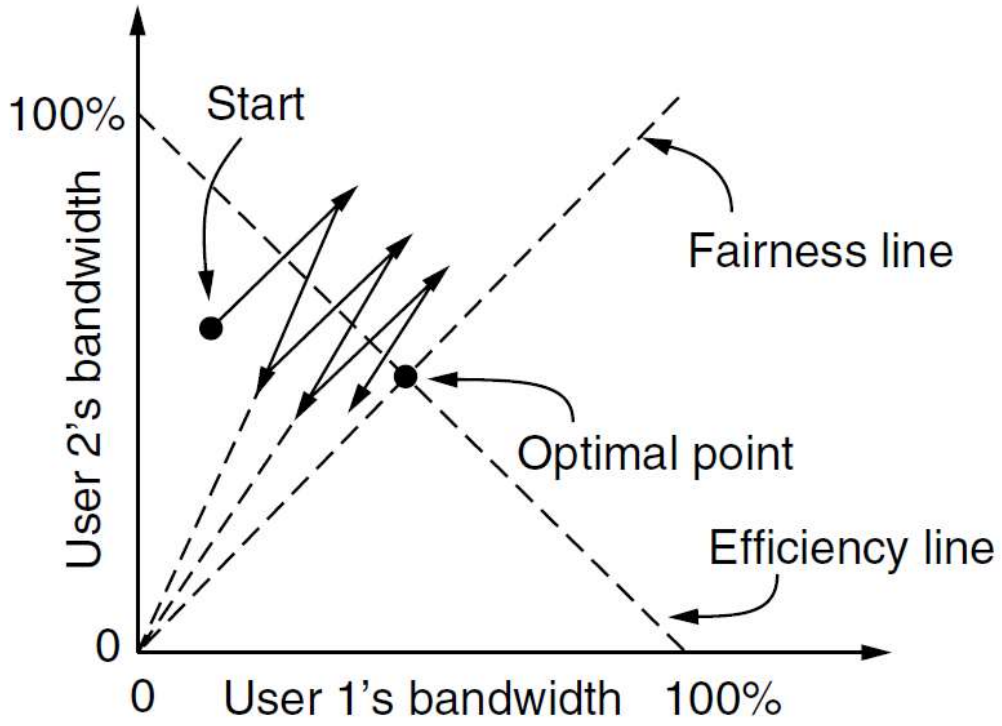
AIAD or MIMD are hard to converge to the optimal sending rates(which is both fair and efficient) !

Control Law: AIMD

- Additive Increase Multiplicative Decrease (加法增乘法减：缓增急减)
- The users additively increase bandwidth allocations and multiplicatively decrease them in congestion
- Used by TCP

AIMD does converge to the optimal point that is both fair and efficient.

AIMD可以收敛于最优点



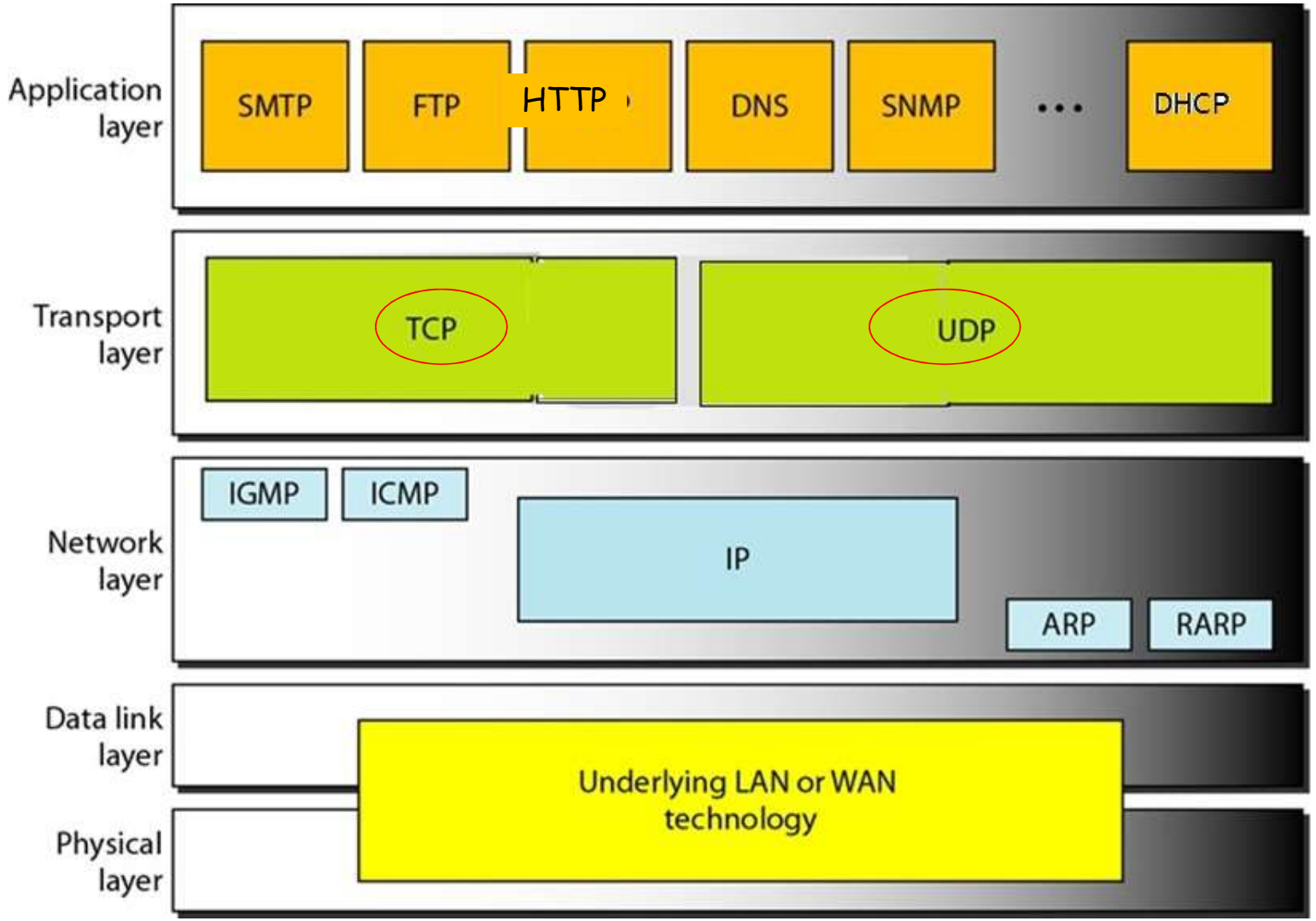
Control Law of TCP

- AIMD
- Stability argument: the increase policy should be gentle and the decrease policy aggressive
- Strategy in practice
  - Adjusting the size of a sliding window
  - Instead of adjusting the rate directly
- TCP-friendly congestion Control
  - Different protocols compete with different control laws to avoid congestion will lead to unequal bandwidth allocations.
  - TCP and non-TCP transport protocols shall be freely mixed with no ill effects

Outline

- Function and Services to Transport Layer
- Elements of Transport Protocols
- *The Transport Layer in Internet*
  - User Datagram Protocol (UDP)
  - Transmission Control Protocol (TCP)

Transport Layer in TCP/IP Model



UDP: User Datagram Protocol

- Connectionless
- Unreliable transfer
  - No acknowledgements to indicate delivery of data
  - No mechanism to detect missing or mis-sequenced messages
  - No mechanism for automatic retransmission
  - No mechanism for flow control
  - Same best-effort service model as IP
- Provides multiplexing/demultiplexing over IP
- Requires minimal Overhead, Computation and Communication

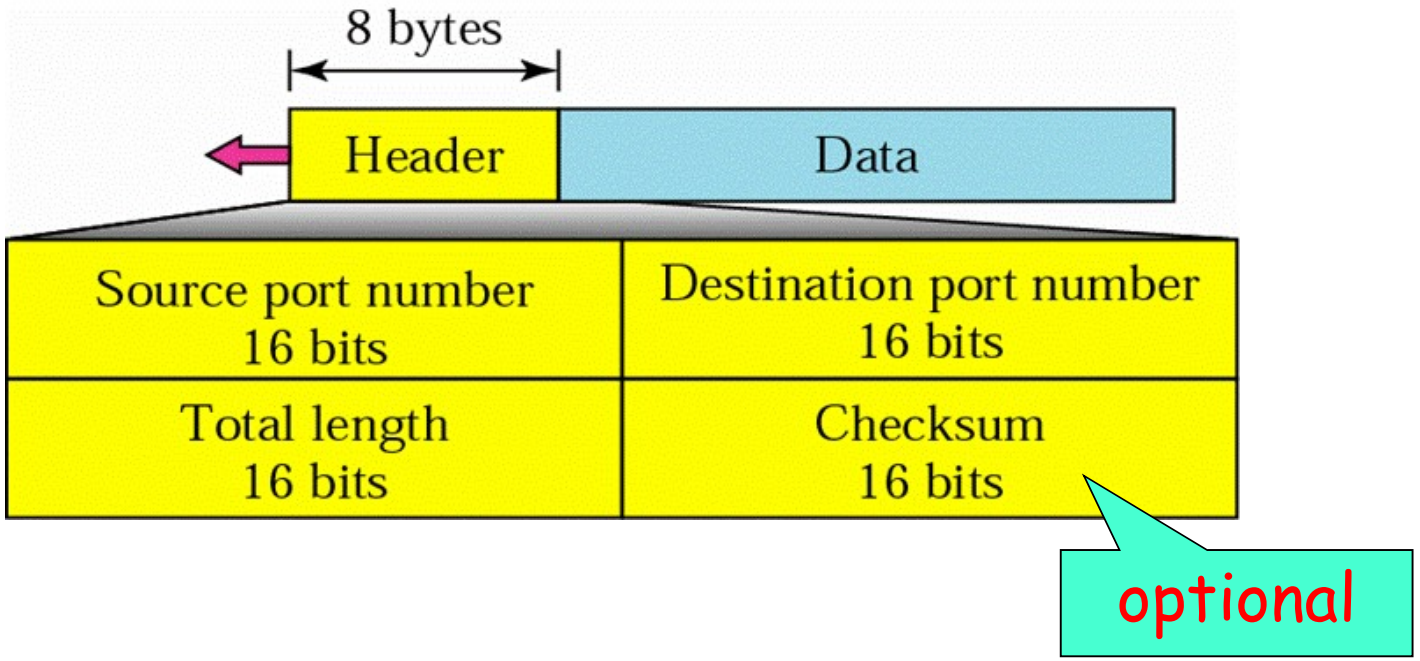
Applications over UDP

- DNS, DHCP
- Network management (SNMP)
- Routing Table Update (RIP)
- Real-time multimedia communications(RTP)
- P2P (Peer-to-Peer, DHT, 'Kademlia')
- ……

Well-known Ports of UDP

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	Bootps	Server port to download bootstrap information
68	Bootpc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

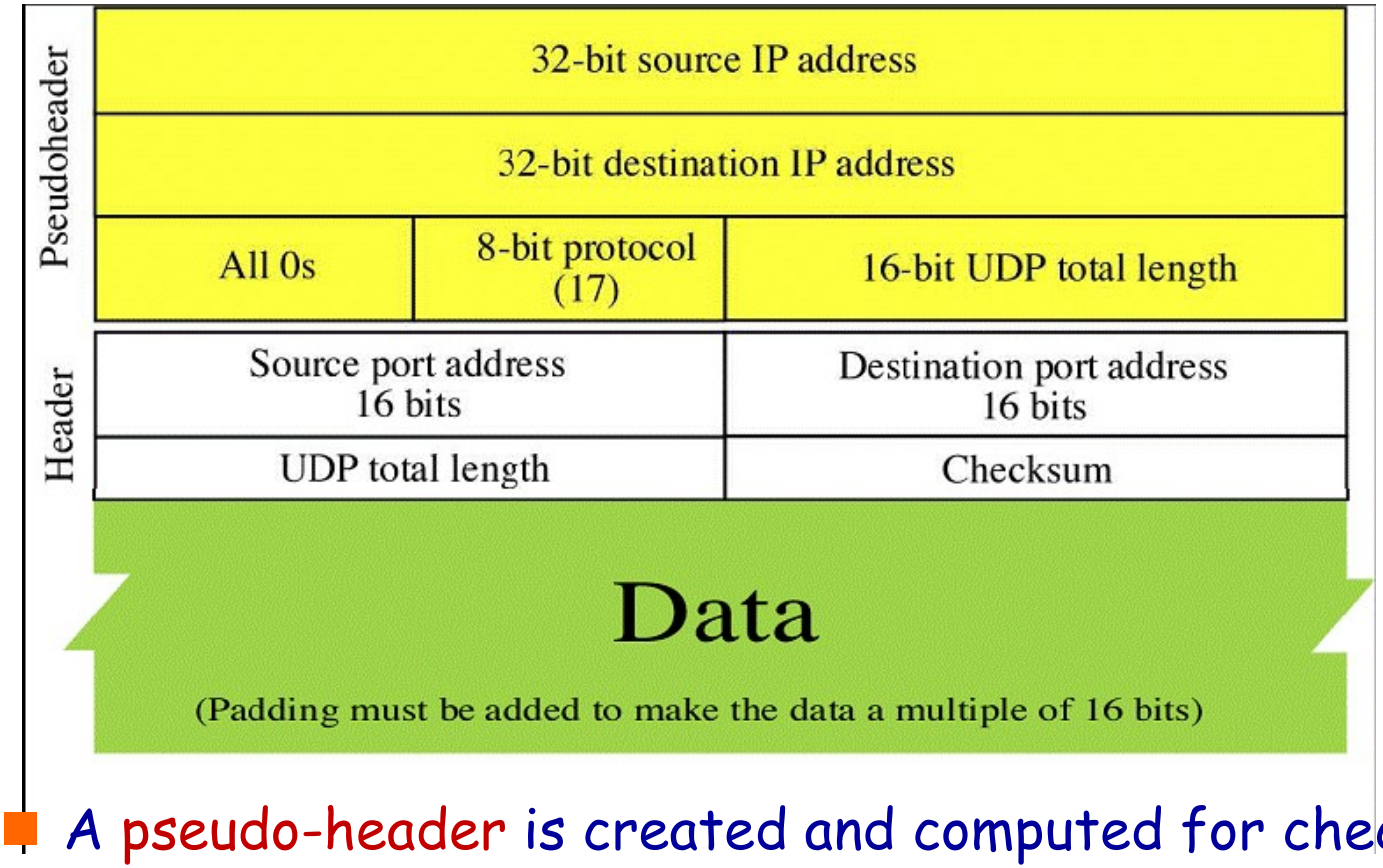
UDP Datagram Format



UDP preserves message boundaries（保留消息边界）between the sender and the receiver.

[Forouzan]

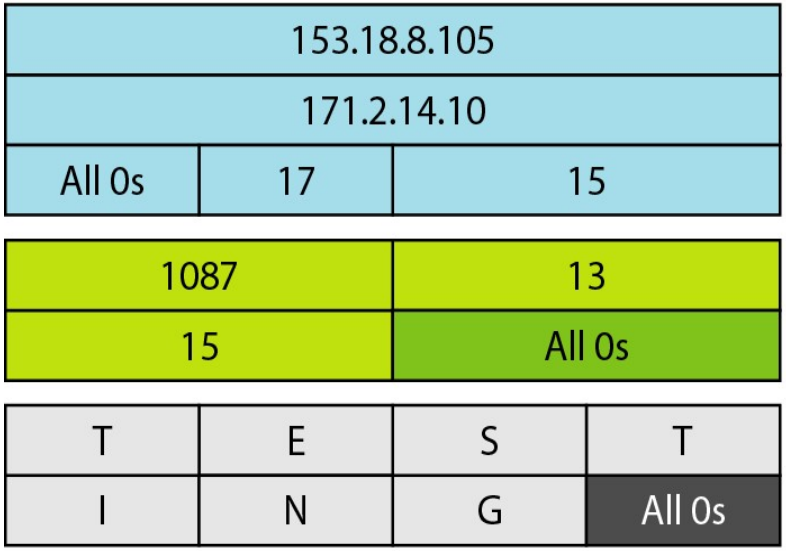
Pseudoheader(伪报头)



- A pseudo-header is created and computed for checksum, but not transferred.

Calculation of Checksum

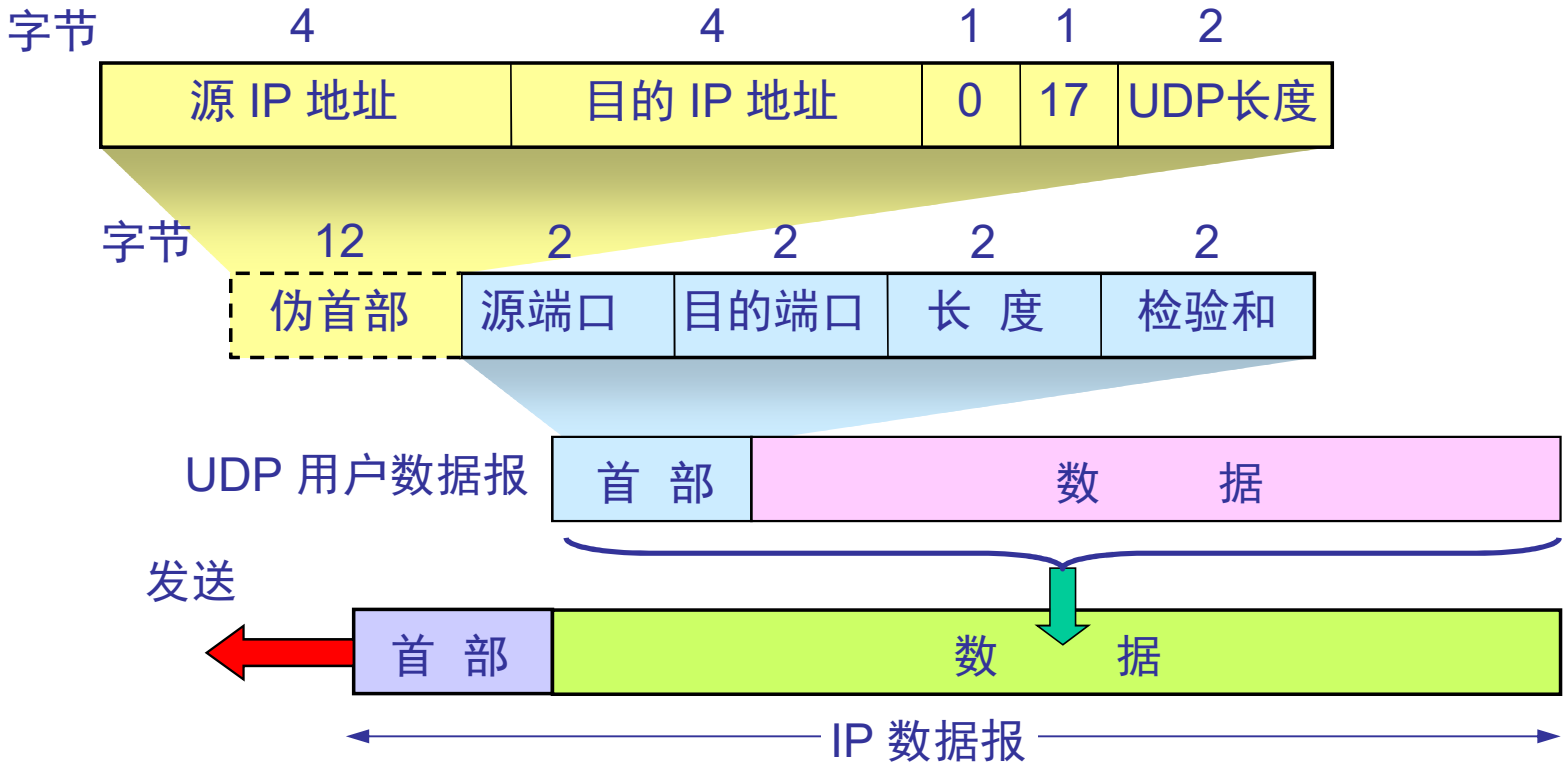
Same algorithm as used in IP



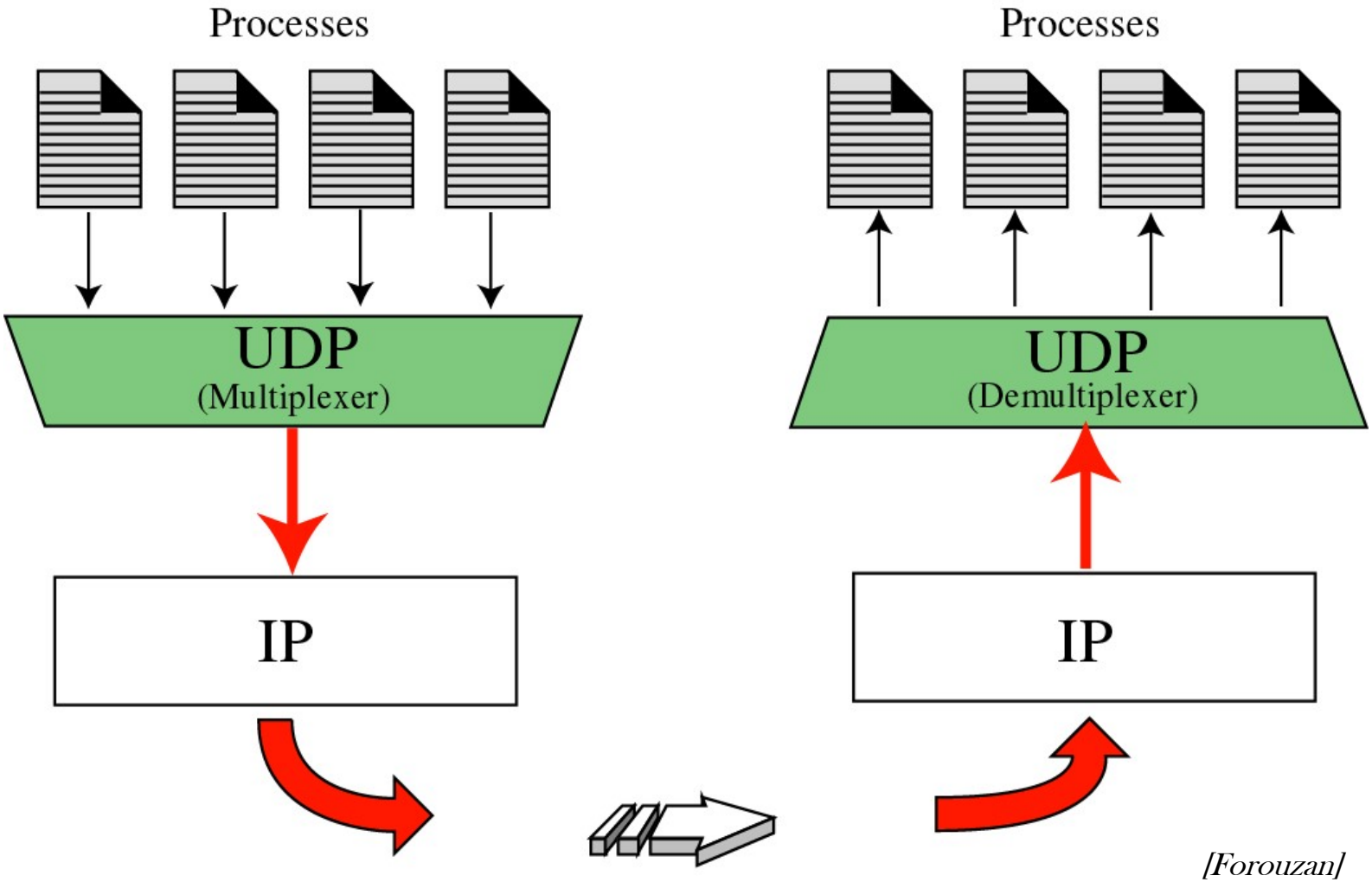
10011001 00010010	153.18
00001000 01101001	8.105
10101011 00000010	171.2
00001110 00001010	14.10
00000000 00010001	0 and 17
00000000 00001111	15
00000100 00111111	1087
00000000 00001101	13
00000000 00001111	15
00000000 00000000	0 (checksum)
01010100 01000101	T and E
01010011 01010100	S and T
01001001 01001110	I and N
01000111 00000000	G and 0 (padding)
10010110 11101011	Sum
01101001 00010100	Checksum

The pseudoheader as well as the padding will be dropped when the user datagram is delivered to IP.

UDP Checksum



Multiplexing and demultiplexing



[Forouzan]

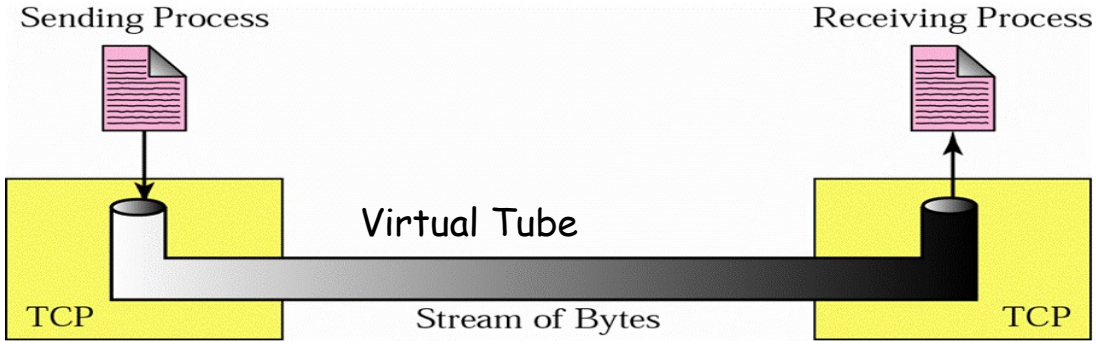
TCP: Transmission Control Protocol

- *TCP Features*
- TCP Segment Format
- Connection Management
- Flow Control
- Error Control
- Congestion Control
- TCP Timer Management

Features of TCP

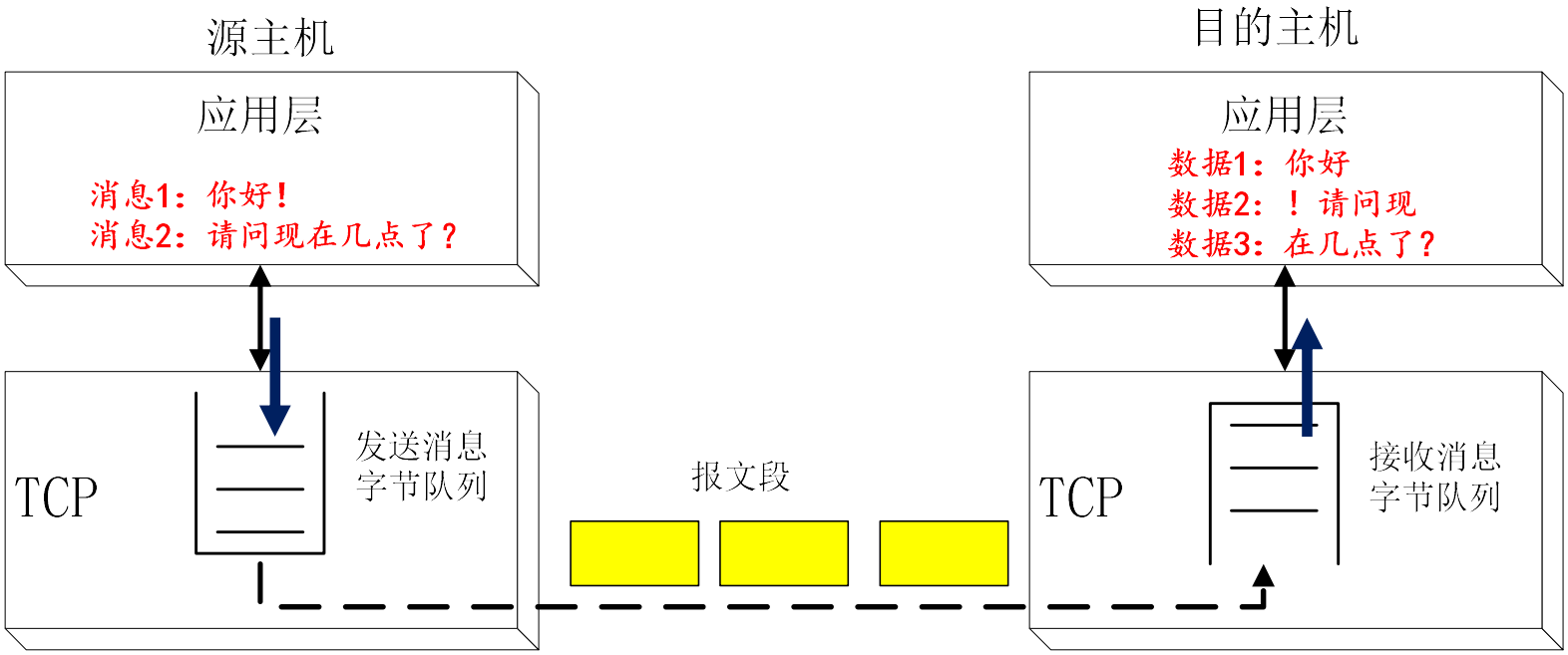
- Connection-oriented
- Reliable delivery (ARQ)
  - Acknowledgements indicate delivery of data
  - Checksums are used to detect corrupted data
  - Corrupted data is retransmitted after a timeout
  - Sequence numbers detect missing, or mis-sequenced data
  - (Window-based) Flow control prevents flooding of receiver
- Using congestion control to share network capacity among users
- Byte stream oriented: messages can be of arbitrary length

No guarantee of application message boundary.(不能保留消息边界)

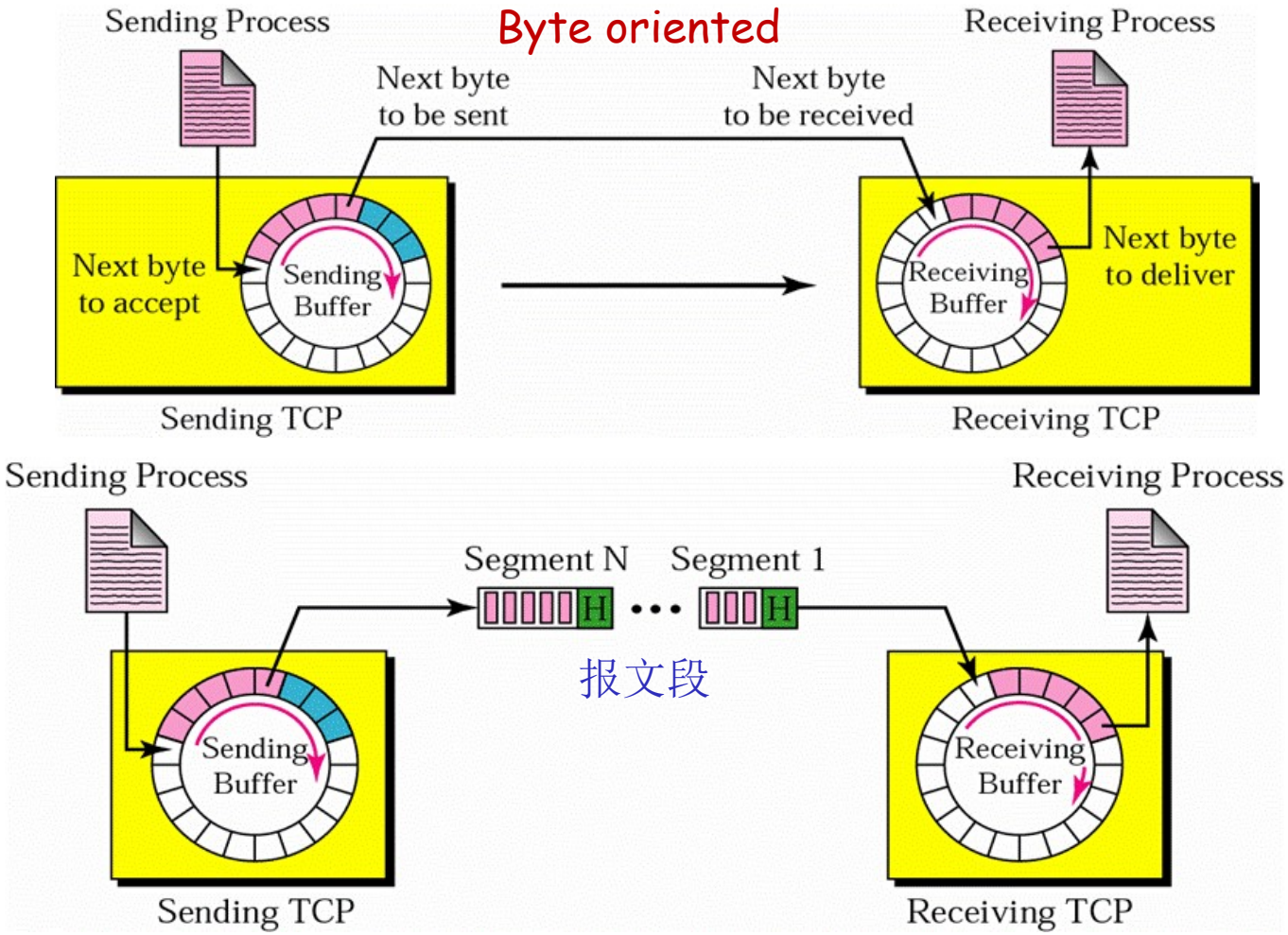


[Forouzan]

TCP不能保证上层应用消息的边界



TCP: Buffers and Segments



[Forouzan]

More on TCP Segment(报文段)

- 20 or more bytes head, n bytes data
- Segments without any data are legal and are commonly used for ACK and control messages
- Byte-oriented sequence number and ACK number
- MSS (Maximum Segment Size)
  - Maximum data length in a segment
  - Limited by IP datagram length
  - Limited by MTU of each data link, so router may fragment a segment
- Variable sized sliding window

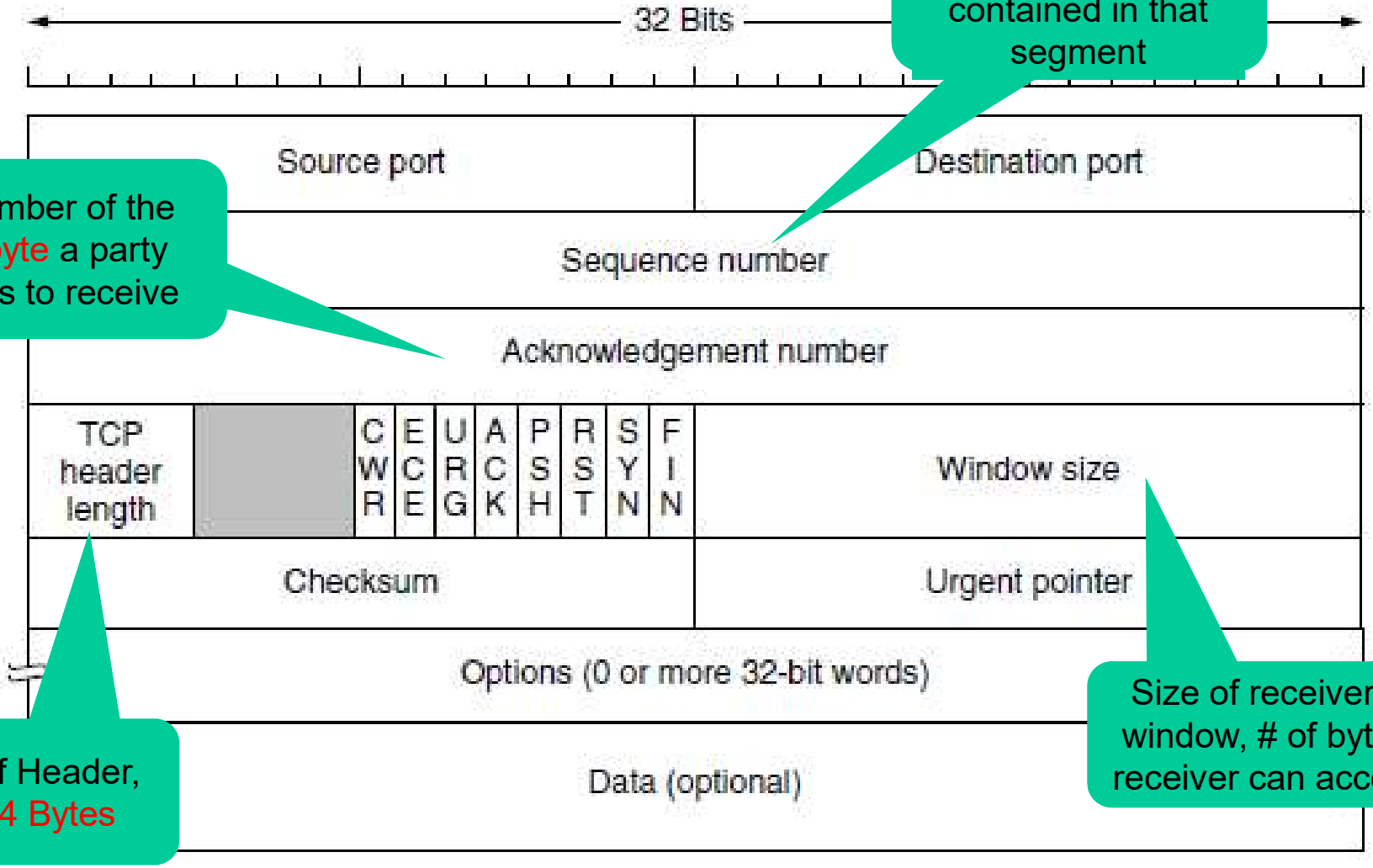
Well-known Ports of TCP

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing
23	Telnet	Remote login

TCP: Transmission Control Protocol

- TCP Features
- *TCP Segment Format*
- Connection Management
- Flow Control
- Error Control
- Congestion Control
- TCP Timer Management

TCP Segment Format



The number of the first data byte contained in that segment

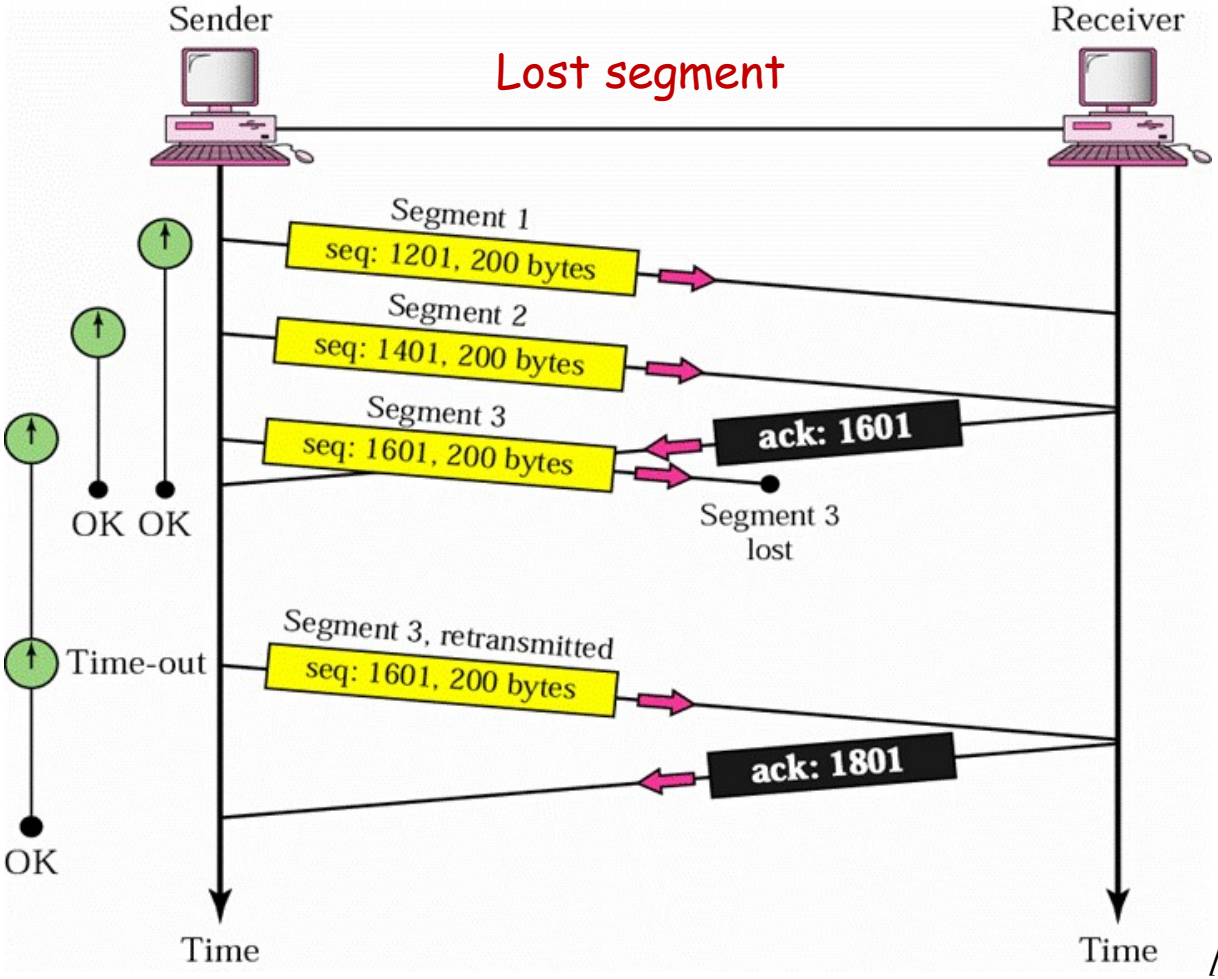
the number of the next byte a party expects to receive

Length of Header, unit by 4 Bytes

Size of receiver's window, # of bytes receiver can accept

[Forouzan]

Example of Seq. No. and ACK No.



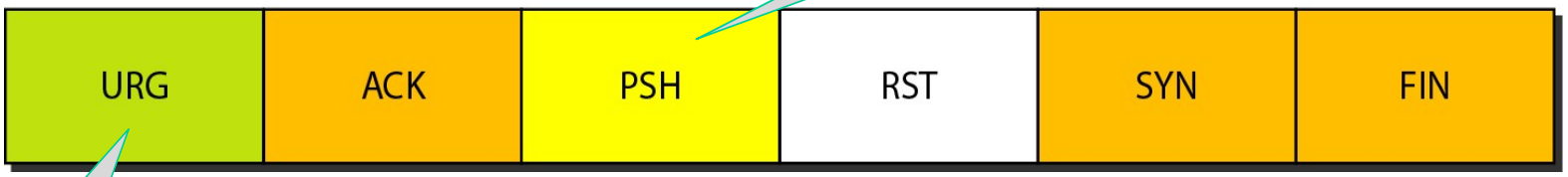
[Forouzan]

Control Field

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push

RST: Reset the
SYN: Synchron quence numbers
FIN: Ter te the connection

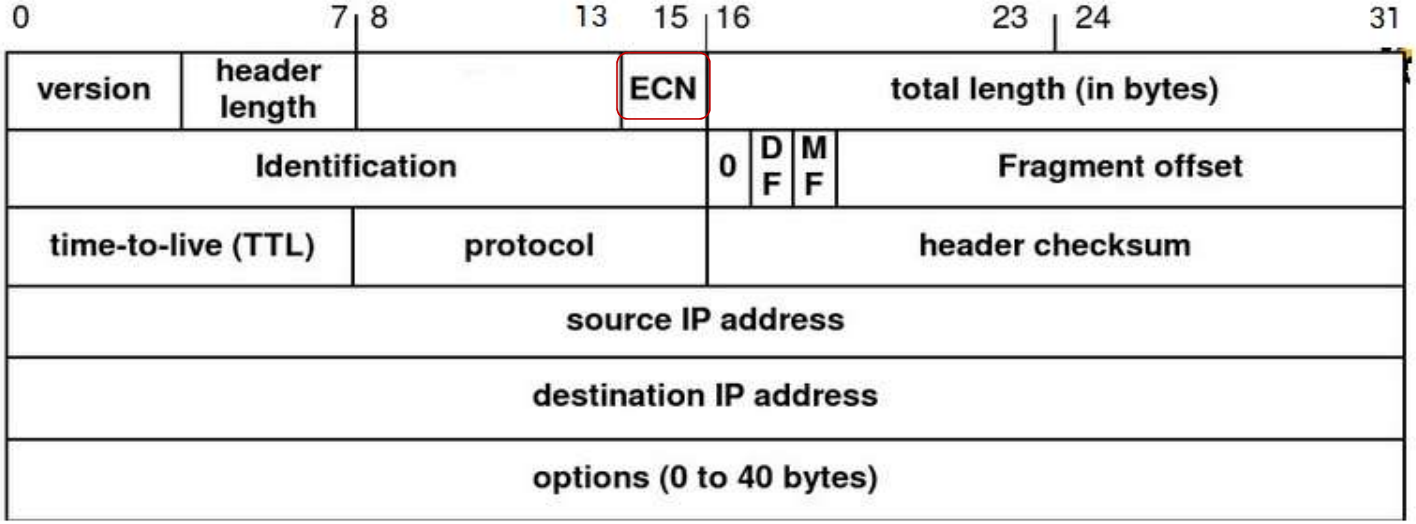
Not to delay the transmission or delivery, eg. Interactive game

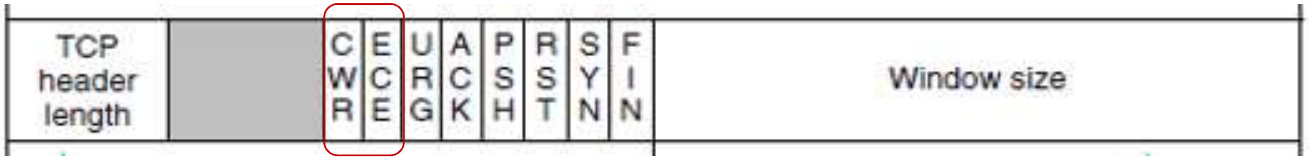


To be treated first and delivered to application layer ASAP , eg. Interrupt signal

Flag	*Description*
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

ECN and ECE/CWR



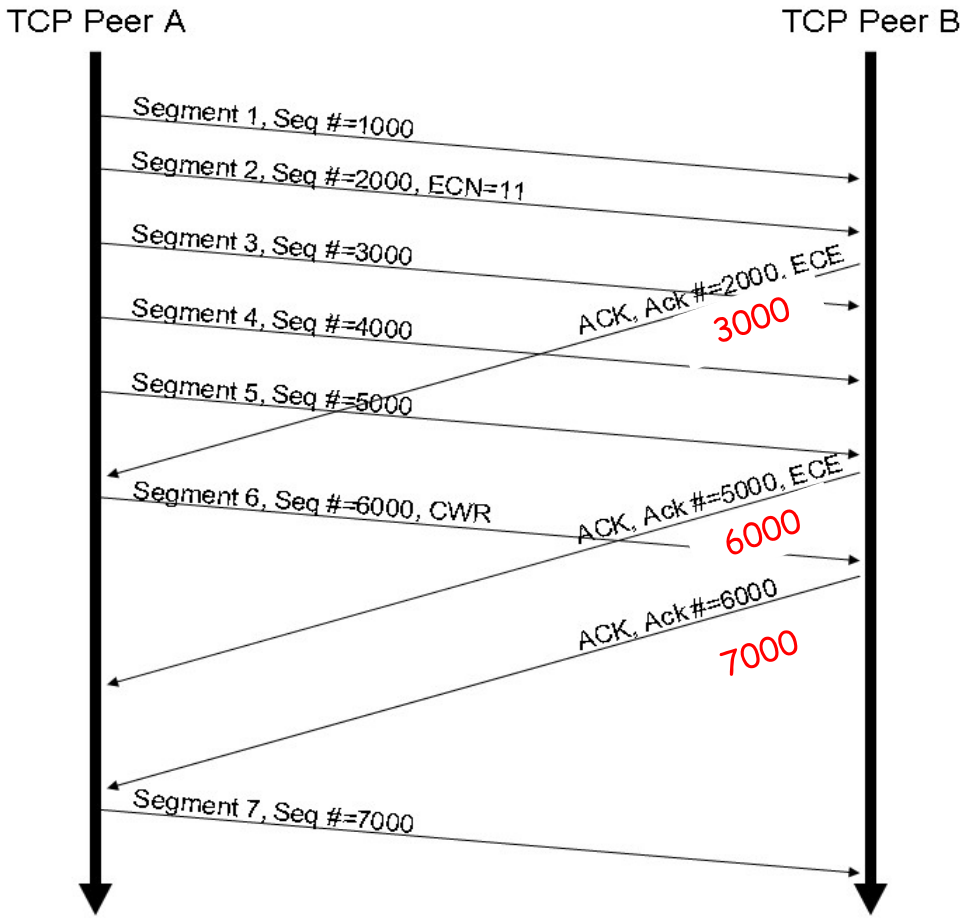


Example of using ECE and CWR

- RFC 3168

ECN is set by router;
ECE is set by TCP receiver
CWR is set by TCP sender

显式拥塞通知



Options in TCP Header

- MSS(Maximum Segment Size)
  - Allow hosts to specify the maximum TCP payload
  - Negotiated during connection setup
  - Default MSS is 536B without this option
  - MSS need not be the same in the two directions
- Window scale
  - Negotiate a window scale factor
- Timestamp
  - Used to compute round-trip time samples
- SACK (Selective ACK)
  - Allow a receiver to tell a sender the ranges of sequence numbers that it has received.

TCP option: SACK

- SACK (Selective ACK)
  - When TCP connection is set up, both the sender and receiver send the SACK option to signal that they understand SACK
  - SACK lists up to 3 ranges of bytes that have been received
  - Now widely deployed

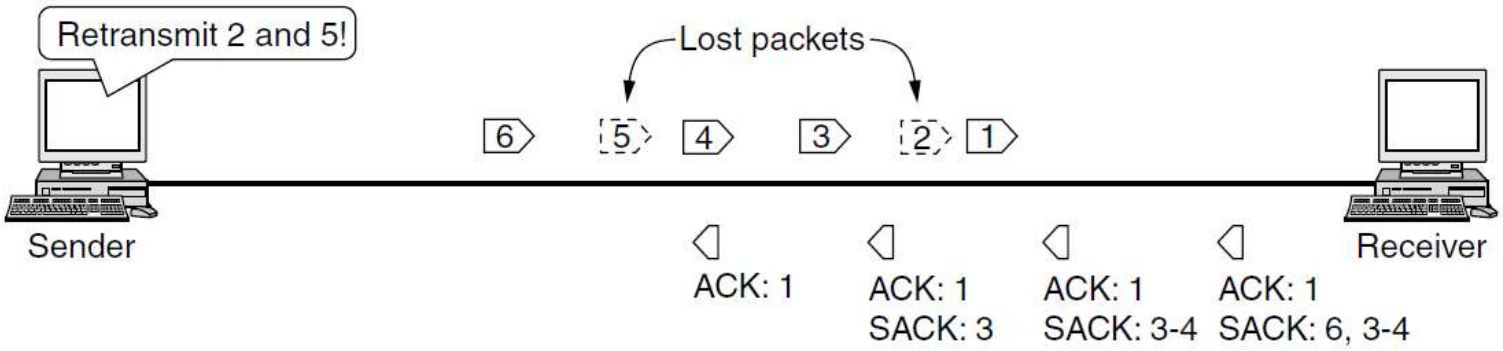


Figure 6-48. Selective acknowledgements

Review

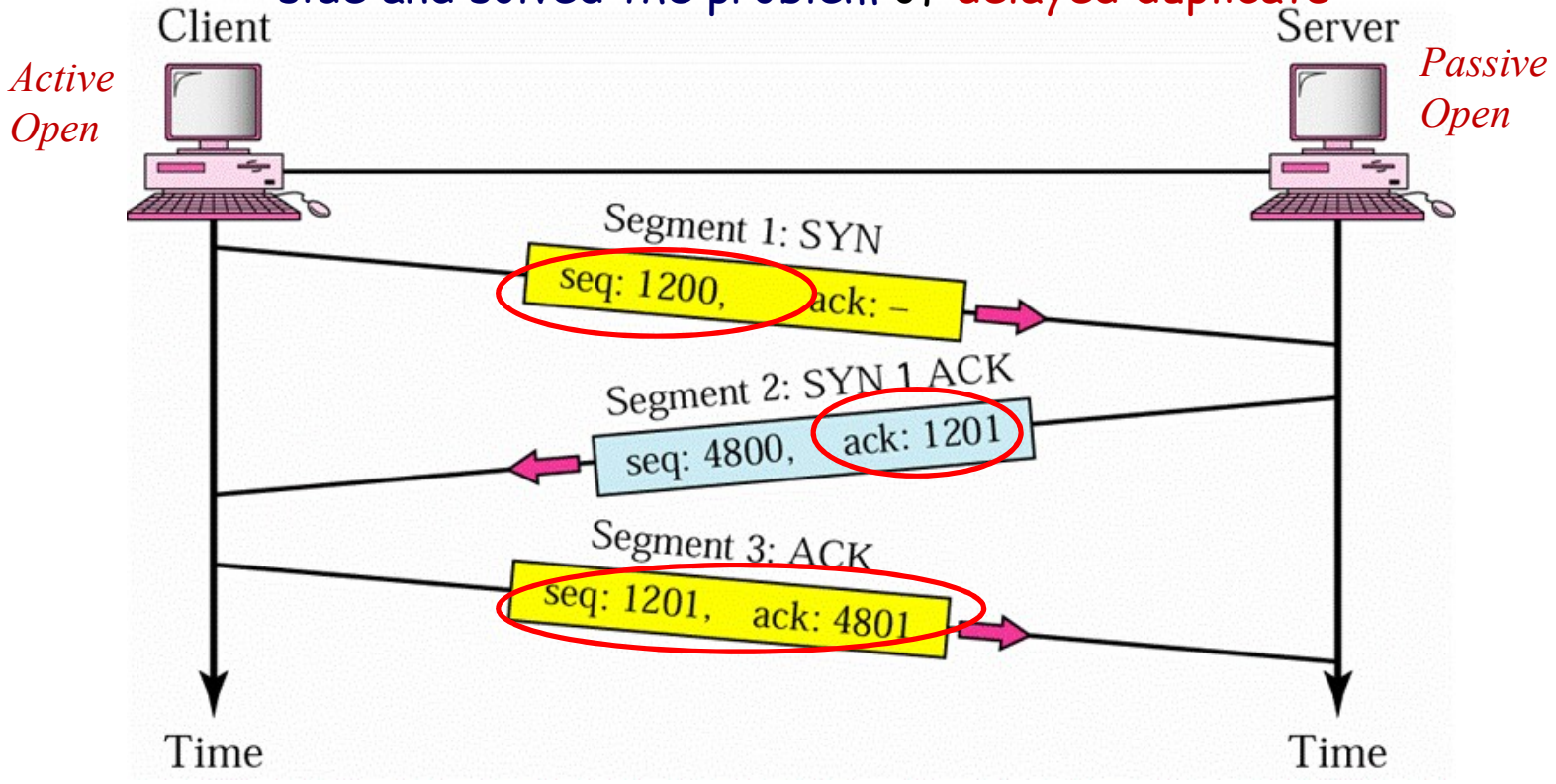
- UDP
  - 无连接、快捷、不可靠
  - 通过端口号寻址到进程和复用
  - 可以保留上层消息的边界
- TCP
  - 面向连接、可靠的字节流传输
  - 序号采用字节编号
  - 不能保留上层消息的边界
  - 显式拥塞通知：和路由器配合
  - MSS：最大数据长度；SACK选项：支持选择重传
- 校验和
  - IP、UDP和TCP使用相同的校验算法
  - IP: 只对包头校验
  - UDP: 可选，对伪报头+UDP数据报（报头+数据） 进行校验
  - TCP：必选，对伪报头+整个报文段（TCP头+数据）进行校验

TCP

- TCP Features
- TCP Segment Format
- *Connection Management*
- Flow Control
- Error Control
- Congestion Control
- TCP Timer Management

3-step Connection Establishment

Agreed on **starting sequence number** for each side and solved the problem of delayed duplicate

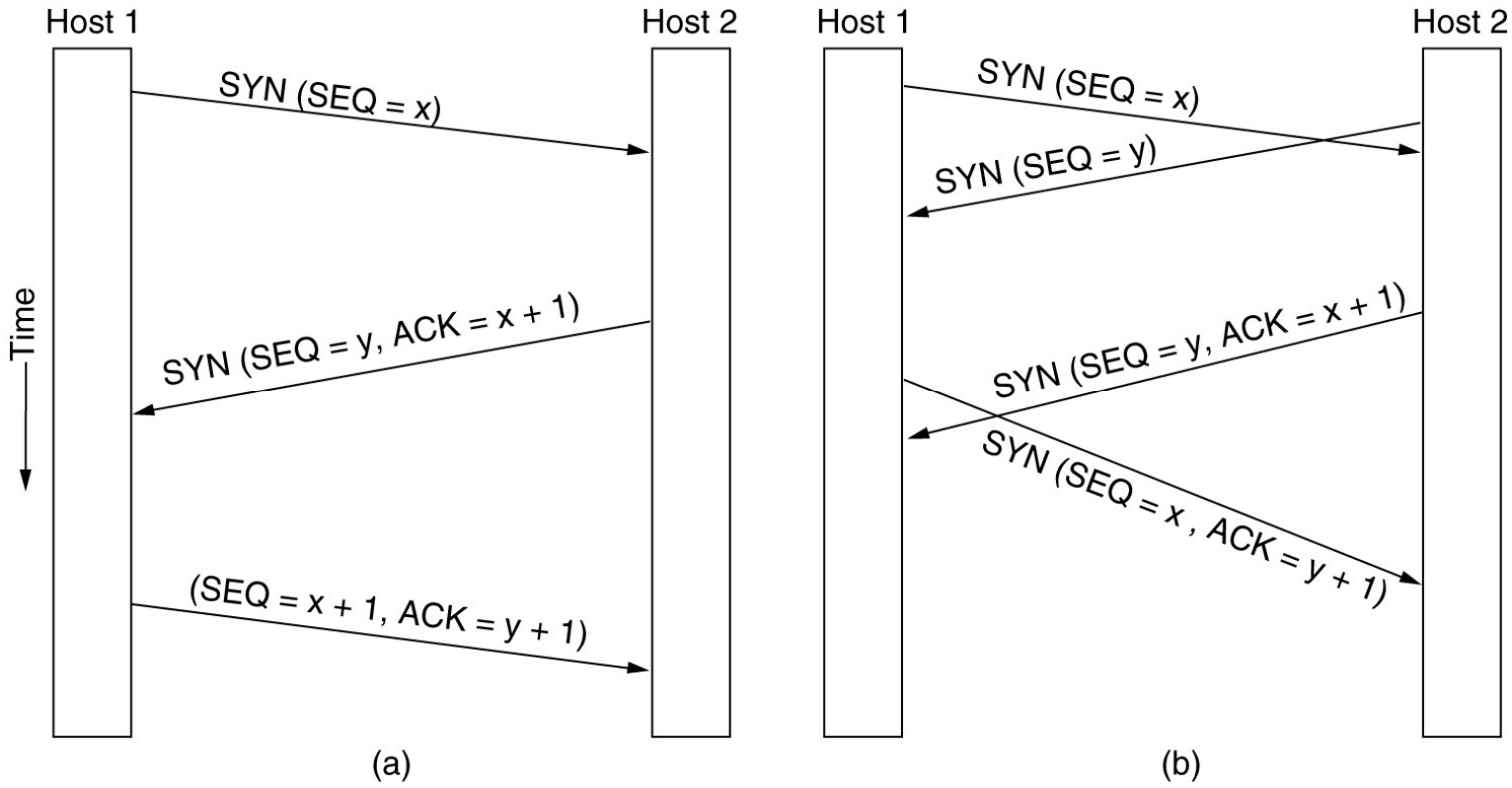


A SYN segment cannot carry data, but consuming **1** sequence number.

A SYN+ACK segment cannot carry data, but consuming **1** sequence number.

[Forouzan]

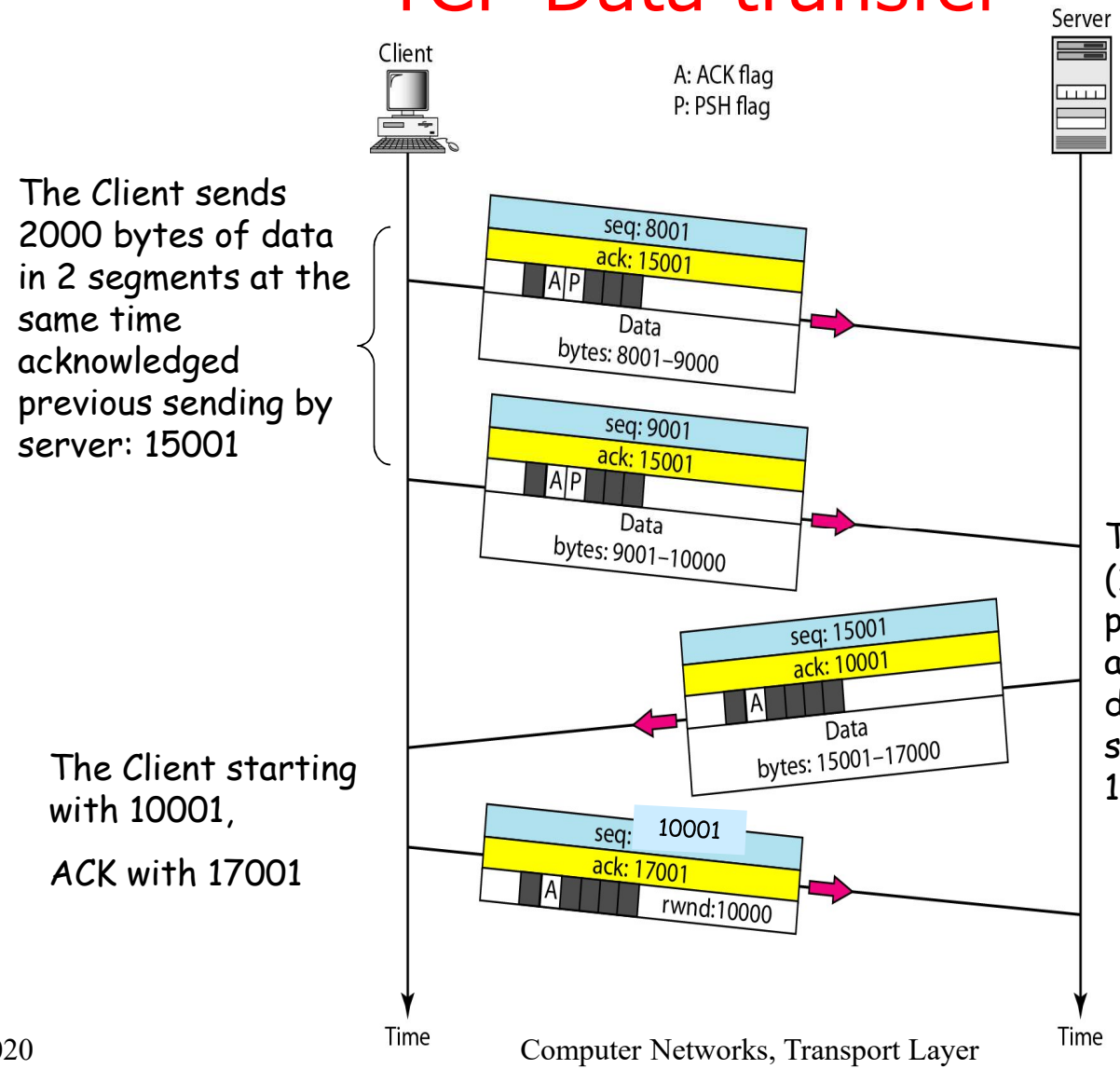
Normal case vs. Simultaneous connection establishment



Initial sequence number

- Initial sequence number
  - A clock-based scheme is used, with a clock tick every 4 μsec
- When a host crashes
  - It may not reboot for the maximum packet lifetime to make sure that no packets from previous connections are still roaming around the Internet somewhere
  - Maximum Segment Lifetime(MSL)

TCP Data transfer



The Client sends 2000 bytes of data in 2 segments at the same time acknowledged previous sending by server: 15001

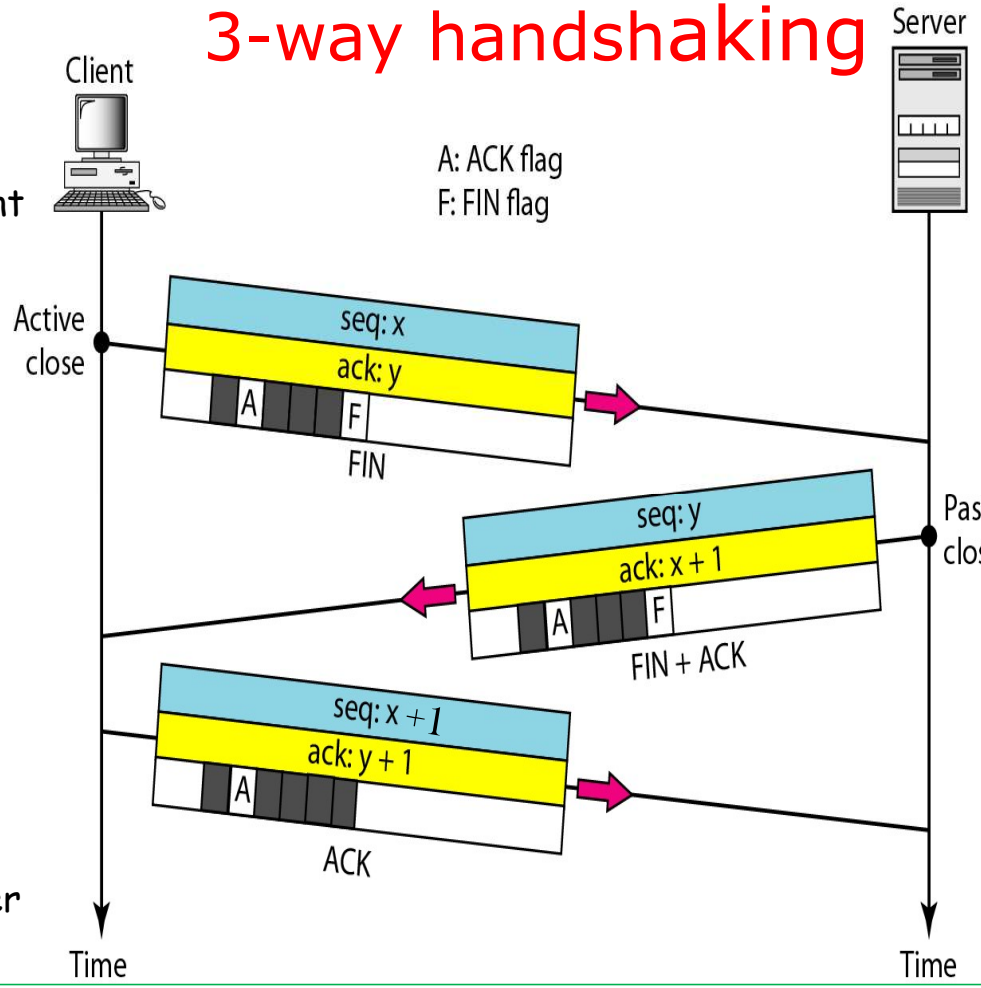
The Server sends ACK (10001) and piggybacked with another 2000 bytes of data starting with sequence number 15001

The Client starting with 10001,

ACK with 17001

Connection Termination: 3-way handshaking

Receiving close command from Application, Client TCP, sends a FIN segment to initiate active close.



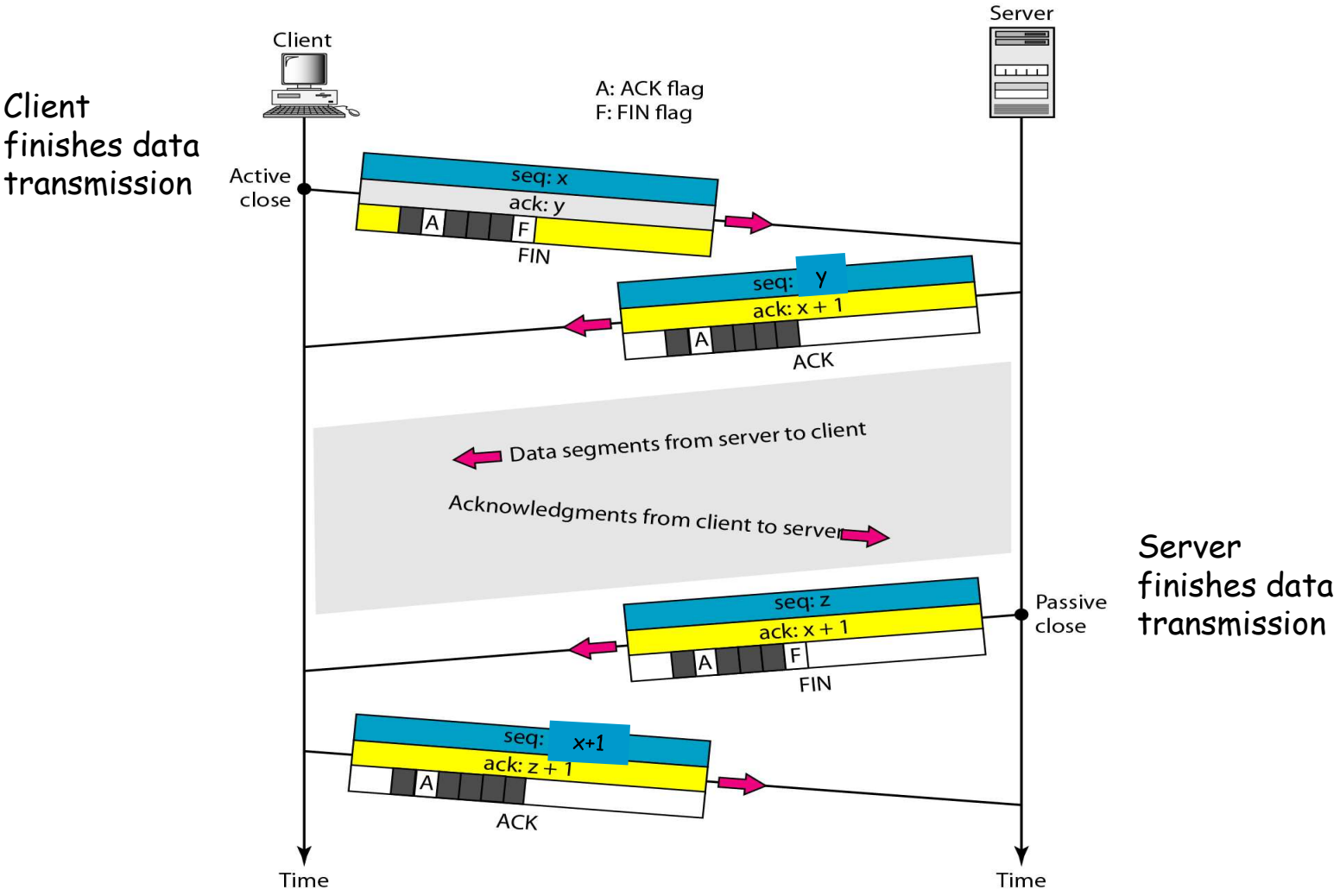
The Server TCP informs its process and sends back a FIN + ACK, to confirm previous FIN and announce closing of its return path

The Client TCP sends last ACK segment, to confirm the receipt of the FIN from server

A FIN segment cannot carry data, but consuming 1 sequence number.

A FIN+ACK segment cannot carry data, but consuming 1 sequence number.

4-step Connection Release: Half-close(半关闭连接)



TCP连接建立和连接释放示例

- 连接建立：三次握手

Source	Destination	Protocol	Length	Info
192.168.0.104	203.119.206.248	TCP	66	50531 → 80 [SYN] Seq=0 Win=17520 Len=0 MSS=1460 WS=256 SACK_PERM=1
203.119.206.248	192.168.0.104	TCP	66	80 → 50531 [SYN, ACK] Seq=0 Ack=1 Win=7300 Len=0 MSS=1412 SACK_PERM=1 WS=512
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [ACK] Seq=1 Ack=1 Win=17408 Len=0
192.168.0.104	203.119.206.248	TCP	181	50531 → 80 [PSH, ACK] Seq=1 Ack=1 Win=17408 Len=127 [TCP segment of a reasser

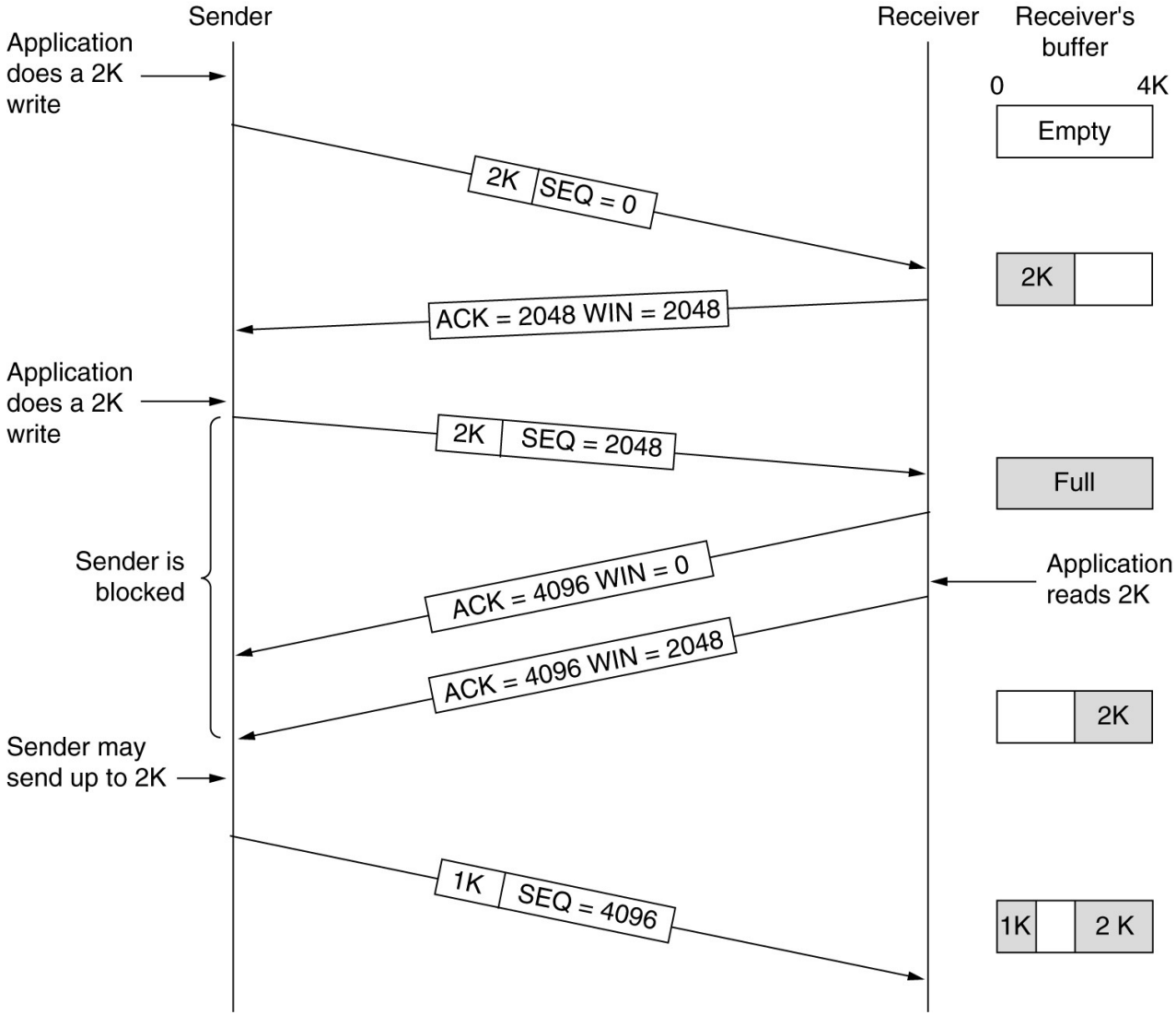
- 连接释放：四步释放

203.119.206.248	192.168.0.104	TCP	54	80 → 50531 [FIN, ACK] Seq=643 Ack=4211 Win=15360 Len=0
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [ACK] Seq=4211 Ack=644 Win=16640 Len=0
192.168.0.104	203.119.206.248	TCP	54	50531 → 80 [FIN, ACK] Seq=4211 Ack=644 Win=16640 Len=0
203.119.206.248	192.168.0.104	TCP	54	80 → 50531 [ACK] Seq=644 Ack=4212 Win=15360 Len=0

TCP

- TCP Features
- TCP Segment Format
- Connection Management
- *Flow Control and Sliding Window*
- Error Control
- Congestion Control
- TCP Timer Management

Flow Control: Sliding Window(Figure 6-40)



Differences with Data Link Layer:

- SW in TCP is byte oriented, but SW in DL is frame oriented
- TCP SW is variable size, but DL SW is fixed size

Window management in TCP

- When the window announced by receiver is 0
  - Sender **may not** normally send segments
  - Urgent data **may** be sent
  - Sender may re-send a 1-byte segment to probe ACK & window update to prevent deadlock

Features of TCP's Sliding Windows

- The source does not have to send a full window's worth of data.
- Dynamic Window Size
  - The size of the window can be increased or decreased by the destination.
  - The sender window size is totally controlled by the receiver window value. However, the actual window size can be smaller if there is congestion in the network.
- The destination can send an acknowledgment at any time.

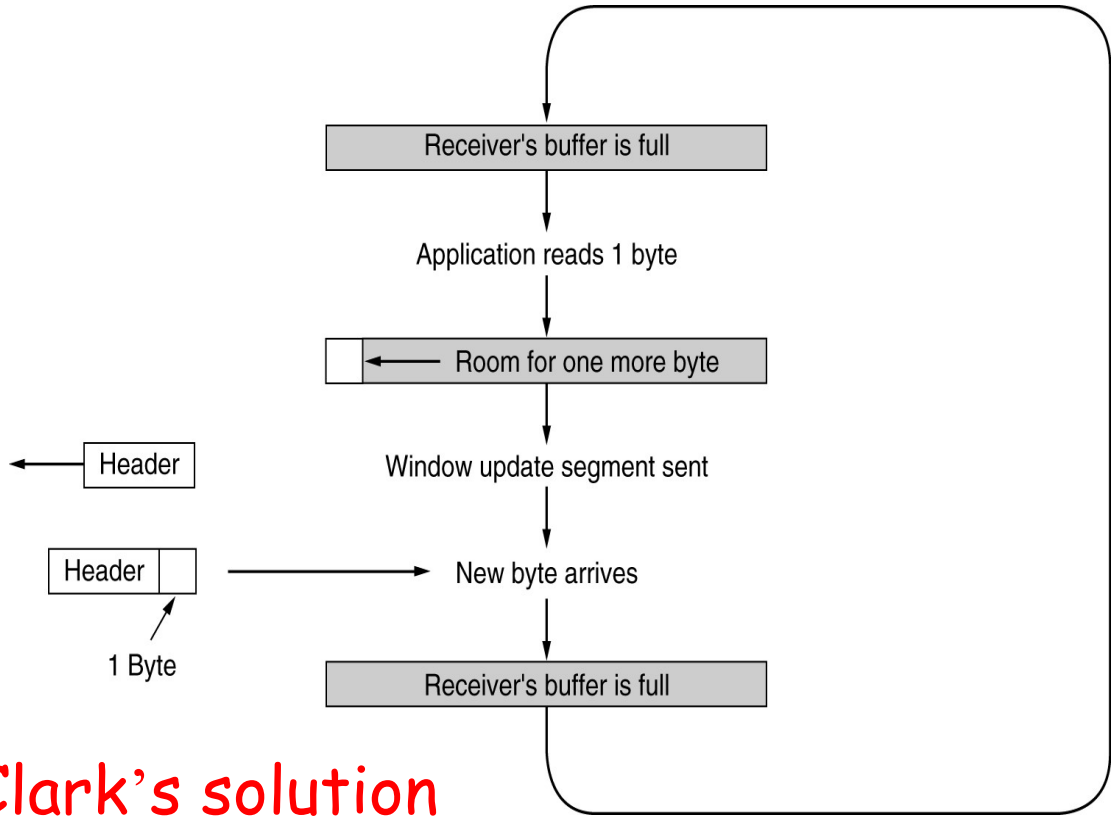
Nagle's algorithm

- Problem
  - Overhead: TCP head + IP head=20+20 bytes
  - When used in TELNET connection, 1 byte data carried with 40 bytes overhead

    (41+40+40+41) (DATA +TCP-ACK+ TCP-WIN-update + DATA)
- Improvement
  - Sending larger segments
  - Sender: Nagle's algorithm

    Do not send until either of the following conditions exist:
    - Acknowledgement to last segment has been received
    - Buffered data fills half the window or a maximum segment(MSS)

Problem 2: Silly Window Syndrome



- Clark's solution

  The **receiver** should not send a window update until it can handle the MSS or its buffer is half empty, whichever is smaller

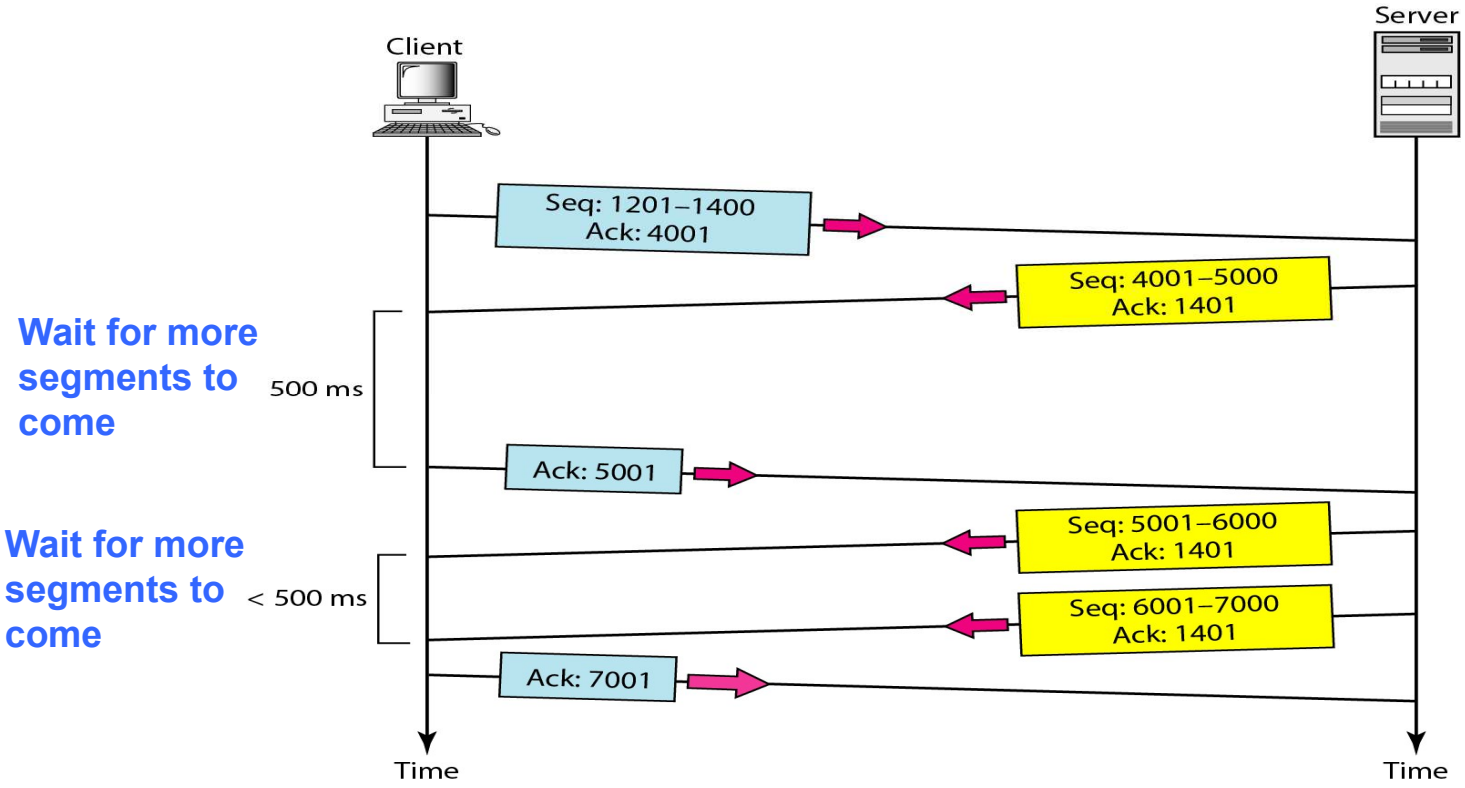
TCP

- TCP Features
- TCP Segment Format
- Connection Management
- Flow Control
- *Error Control*
- Congestion Control
- TCP Timer Management

TCP Error Control

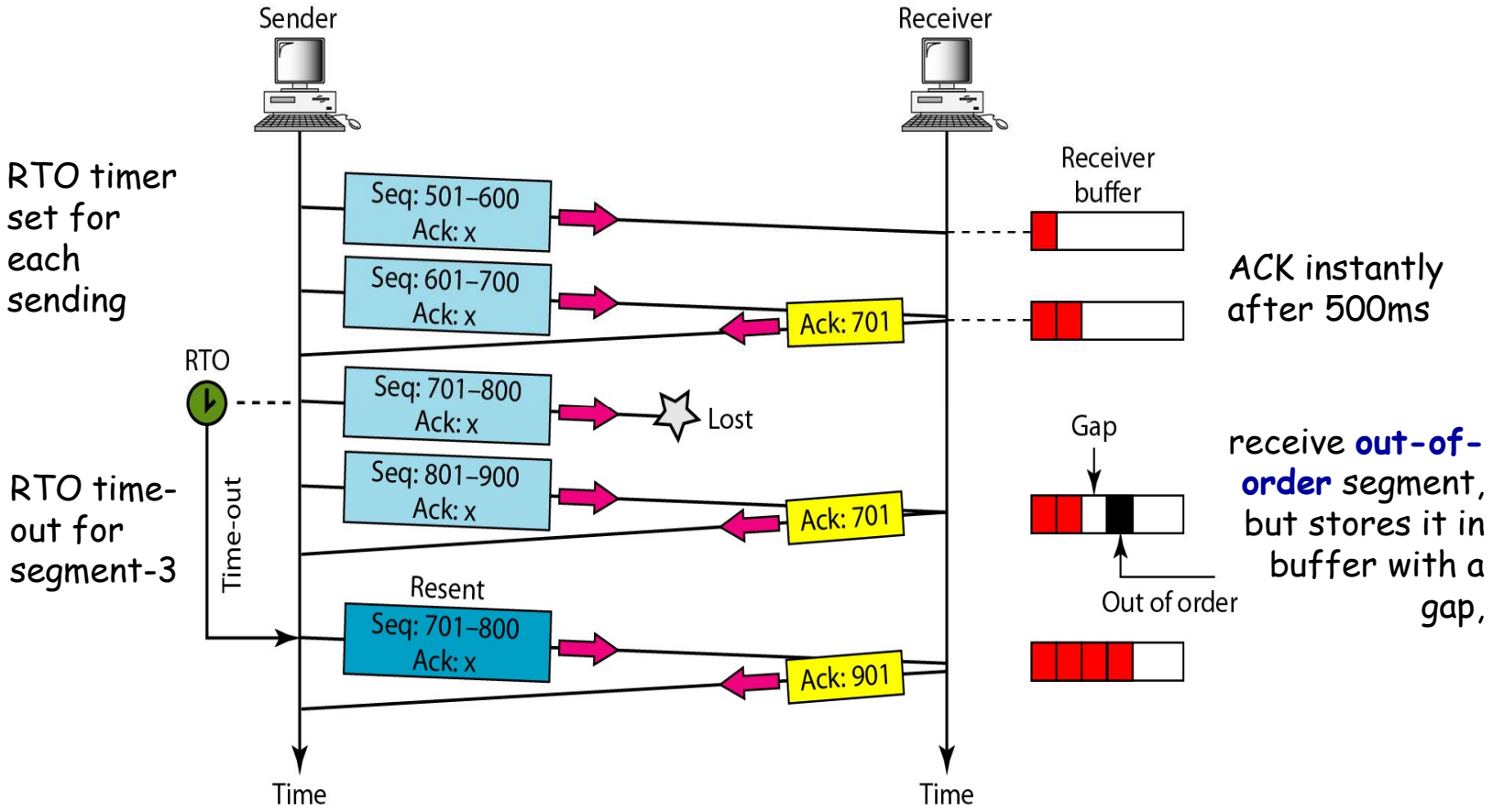
- TCP provides reliable transmission
- Error control includes mechanism for detecting corrupted segments, lost segments, out-of-order segments and duplicated segments.
- Error checking and correction using ARQ
  - Checksum: to check for corrupted segment and discard it.
  - Acknowledgement (ACK): to confirm the receipt of segments.
  - Time-out : Timer set for retransmission of segments, referred as RTO(Retransmission Time-Out) timer.
- The heart of error control mechanism is Retransmission.
- No retransmission and timer set for an ACK segment

Normal Operation

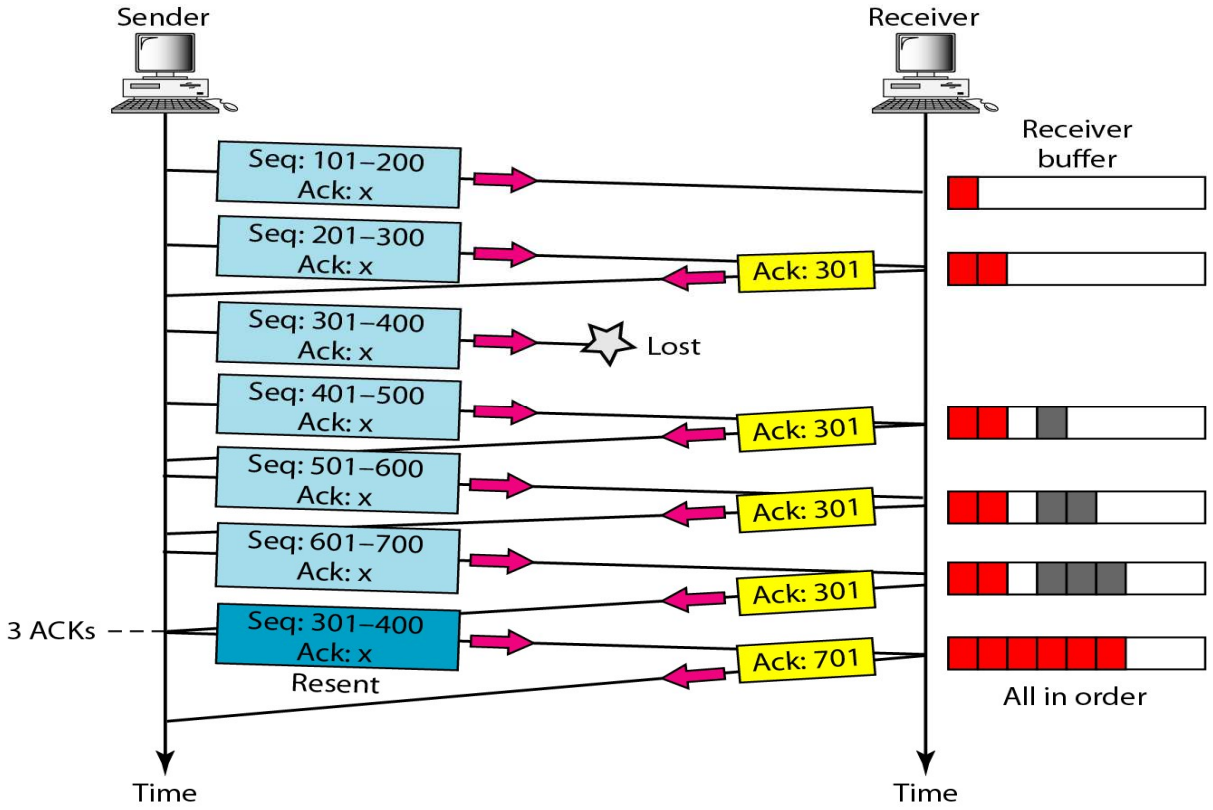


A slight delay waiting of 500 ms before sending off the ACK, to see any more segments arrive so that ACK can cover for this range of time(Cumulative ACK). (e.g. ACK: 6001 is skipped).

Lost Segment

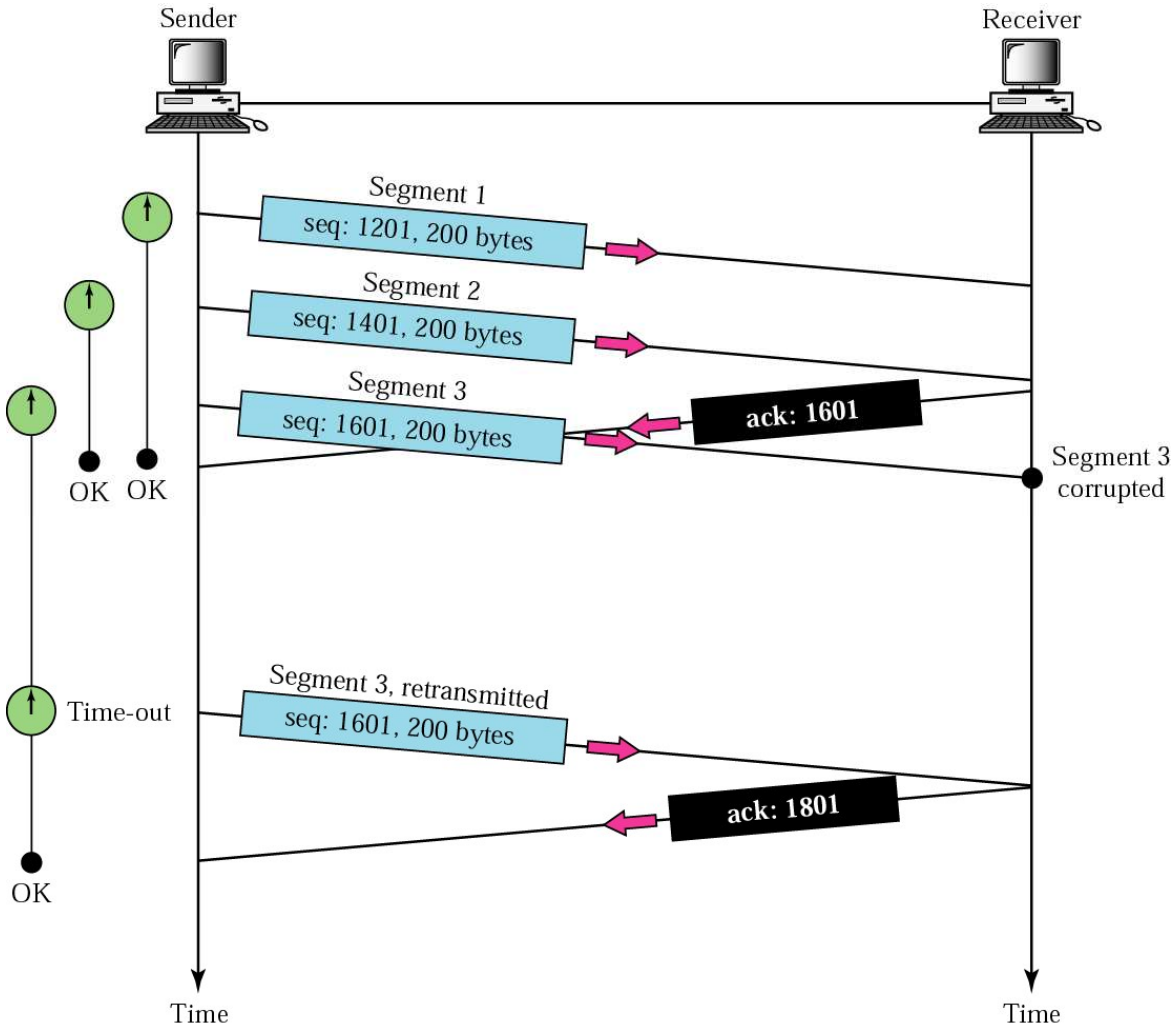


Fast Retransmission(快速重传)

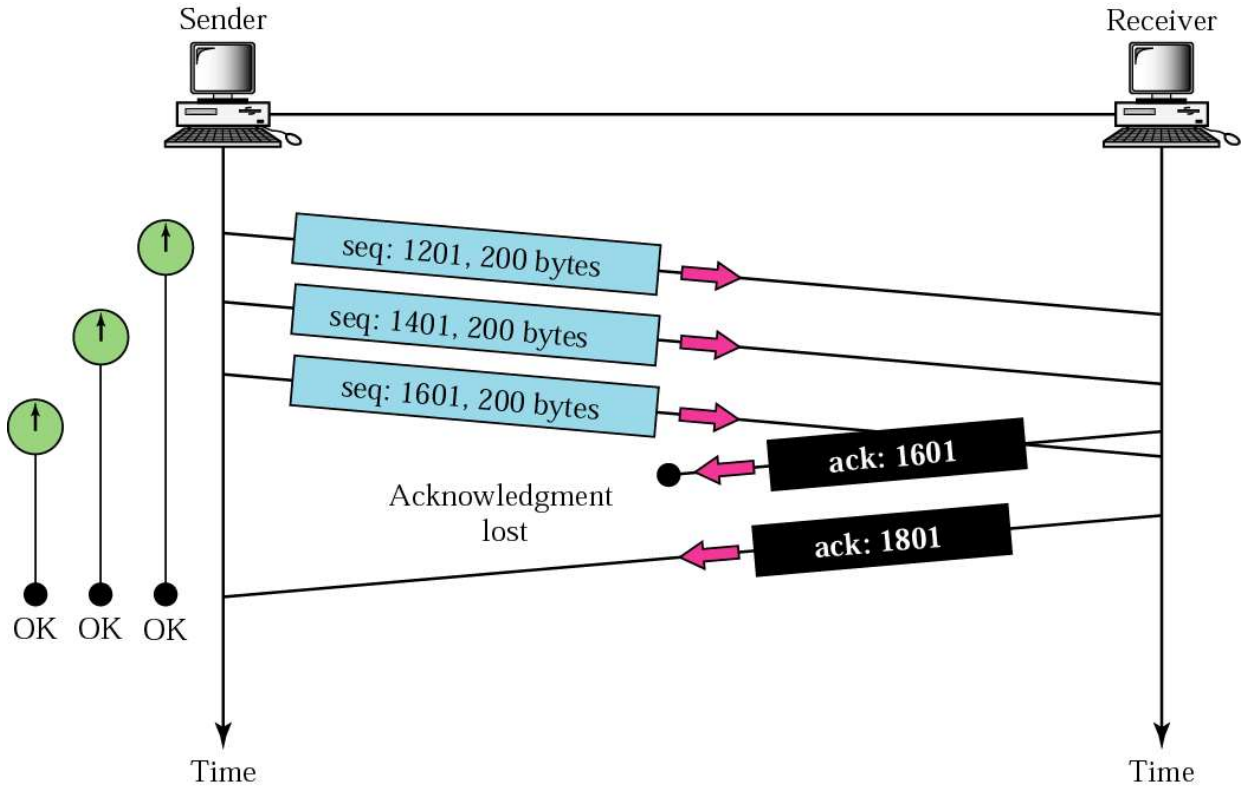


FAST re-transmission: When 3 duplicate ACKs are returned, the sender resend the lost segments immediately without waiting for RTO.

Corrupted segment



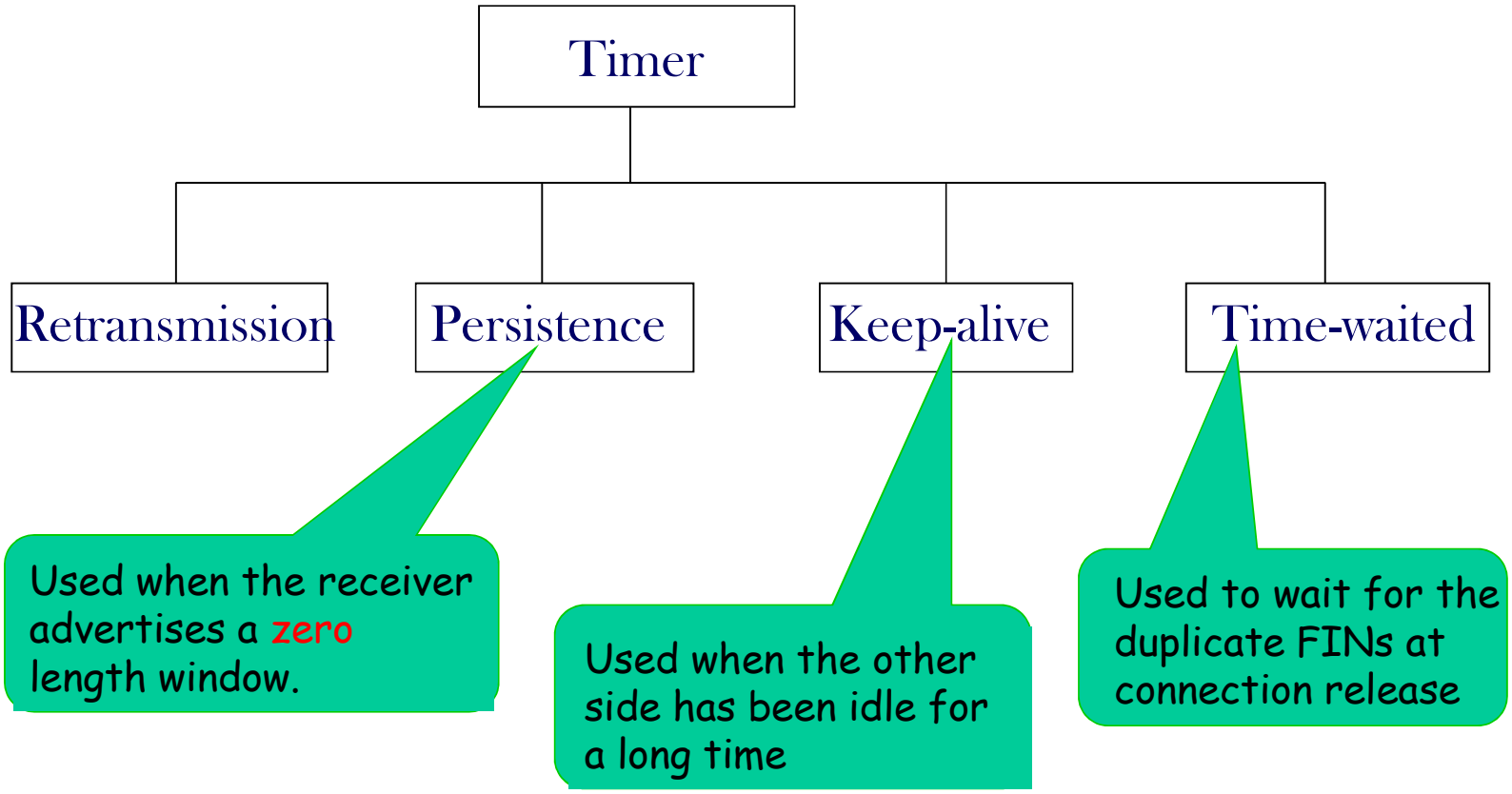
Lost acknowledgment



TCP

- TCP Features
- TCP Segment Format
- Connection Management
- Flow Control
- Error Control
- *TCP Timer Management*
- Congestion Control

TCP Timer Management

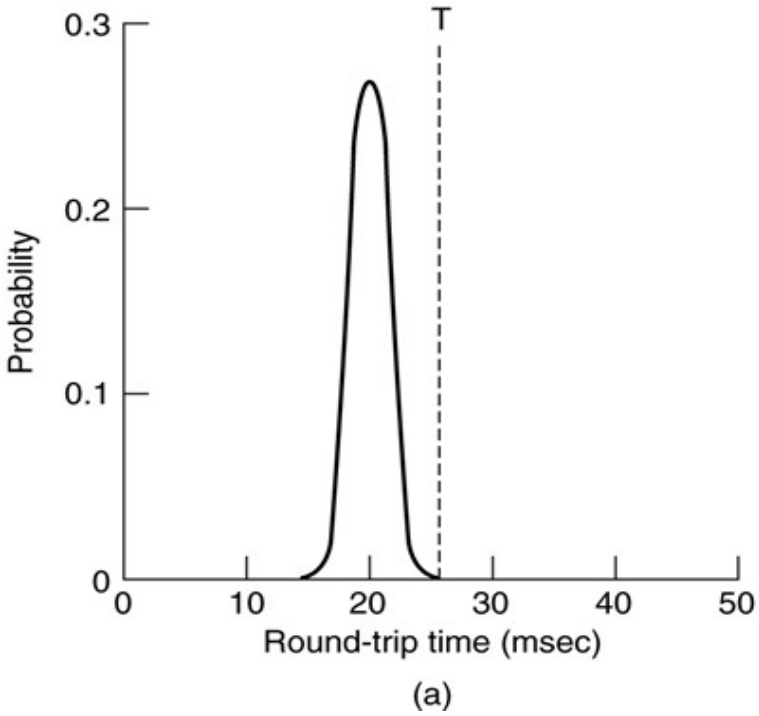


TCP Timers

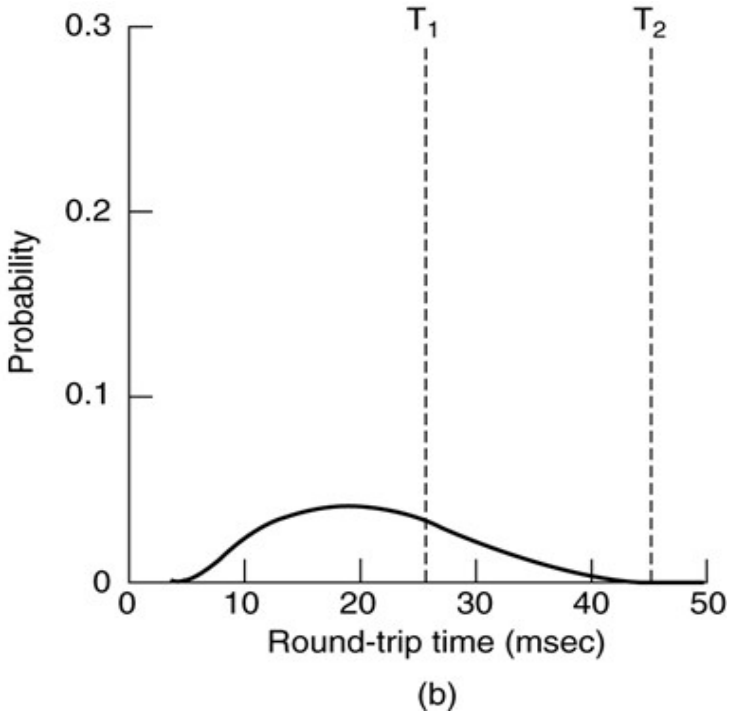
- Persistence timer
  - It is designed to prevent the 0-window deadlock
  - When timer goes off, the sender transmits a probe to the receiver. The response to the probe gives the window size
- Keepalive timer
  - It is designed to prevent the half-open connection
  - When a connection has been idle for a long time, the keepalive timer may go off to cause one side to check whether the other side is still there
- The timer used in the TIMED WAIT state
  - Avoid the two-army problem
  - It runs for twice the maximum packet lifetime to make sure that when a connection is closed, all packets created by it have died off

RTT Estimation: Data Link Layer vs. TCP

- Probability density of Round Trip Time (**RTT**)



Data Link Layer



TCP

Retransmission Timer (Textbook 6.5.10)

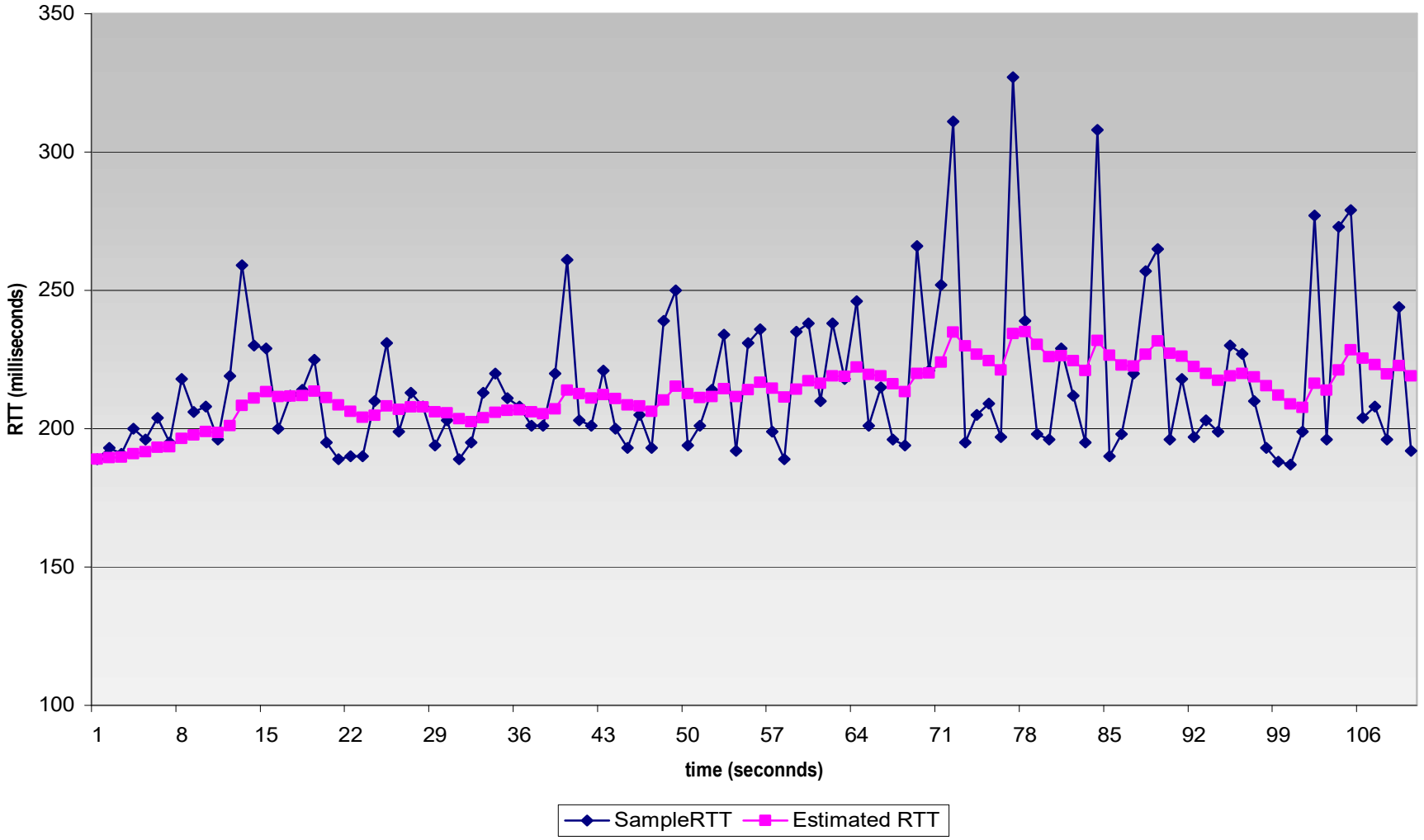
- How to set timeout?
  - Too long: low throughput, inefficiency
  - Too short: duplicate packets, which may makes congestion worse
- Solution: making timeout proportional to RTT
  - New RTT = α x (old RTT) + (1 – α) x (new sample)

    Sample: AckRcvdTime – SendPacketTime

    $0 \leq \alpha < 1$, usually 7/8

  - SRTT: Smoothed RTT

Example RTT estimation:



Retransmission Timeout: RTO

- Jacobson: Using mean deviation
- RTO is sensitive to in RTT-variance (RTTVAR) as well as SRTT
  - $RTTVAR = \beta \times RTTVAR + (1-\beta) \times |SRTT\text{-}Sample|$
  - $RTO = SRTT + 4 \times RTTVAR$
- *Question: For retransmission, how to calculate Sample and RTO?*
  - Karn's algorithm: do not update estimates on any segments that have been retransmitted.
  - RTO is doubled on each successive retransmission

TCP

- TCP Features
- TCP Segment Format
- Connection Management
- Flow Control
- Error Control
- TCP Timer Management
- *Congestion Control*

Congestion Control in Internet

- Router (network layer) detects congestion when queues grow large and tries to manage it, if only by dropping packets.
- Source host (transport layer, TCP) receives congestion feedback from the network layer and slow down the rate of traffic that it is sending into the network.

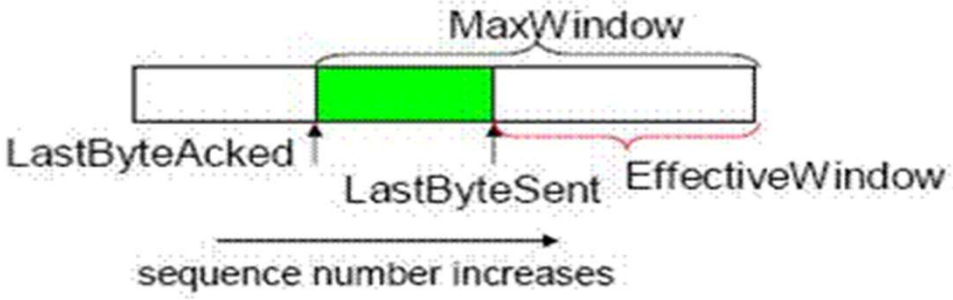
TCP Congestion Control

- Congestion is inevitable without reserved end-to-end bandwidth
- TCP Senders detect congestion (Implicit notification) and reduce sending rate by cutting their congestion window size
- TCP modifies its rate according to "Additive Increase, Multiplicative Decrease (AIMD)".
- To quickly guess at a congestion window size, TCP uses a fast restart mechanism (called "slow start")
- TCP tries really hard to avoid timeouts using "fast retransmission"

Congestion Window

- CongestionWindow (*cwnd*) is a variable held by the TCP source for each connection.
  - Denotes how many bytes network is able to absorb
- Sender's maximum window
  - Min (receiver's advertised window, *cwnd*)
- Sender's effective window:
  - Max window - unacknowledged segments=

    Max window- (LastByteSent -LastByteAcked)

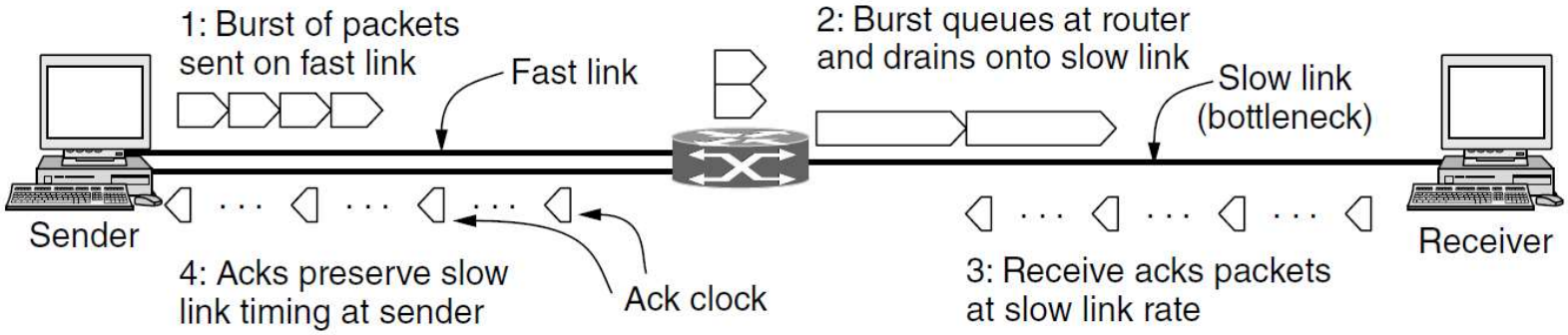


Sub-problems of Congestion Control

- How to detect congestions
  - Packet dropping is the best sign of congestion
  - How to detect packet drops? ACKs
    - TCP uses ACKs to inform receipt of data
    - ACK indicates last contiguous byte received
  - No ACK after time-out implies packet drop
- How to adjust sending rate
  - Upon receipt of ACK, increasing rate
  - Upon detection of packet loss, decreasing rate

ACK Clock

- The times for the ACKs reflect the times at which the packets arrived at the receiver after crossing the slow link
- As the ACKs travel over the net and back to the sender, routers preserve this timing
- If TCP injects packets at the rate of ACKs returned, packets will be sent as fast as the slow link permits, they will not queue up and congest any router along the path
- By using an ACK clock, TCP smoothes out traffic and avoids unnecessary queues at routers

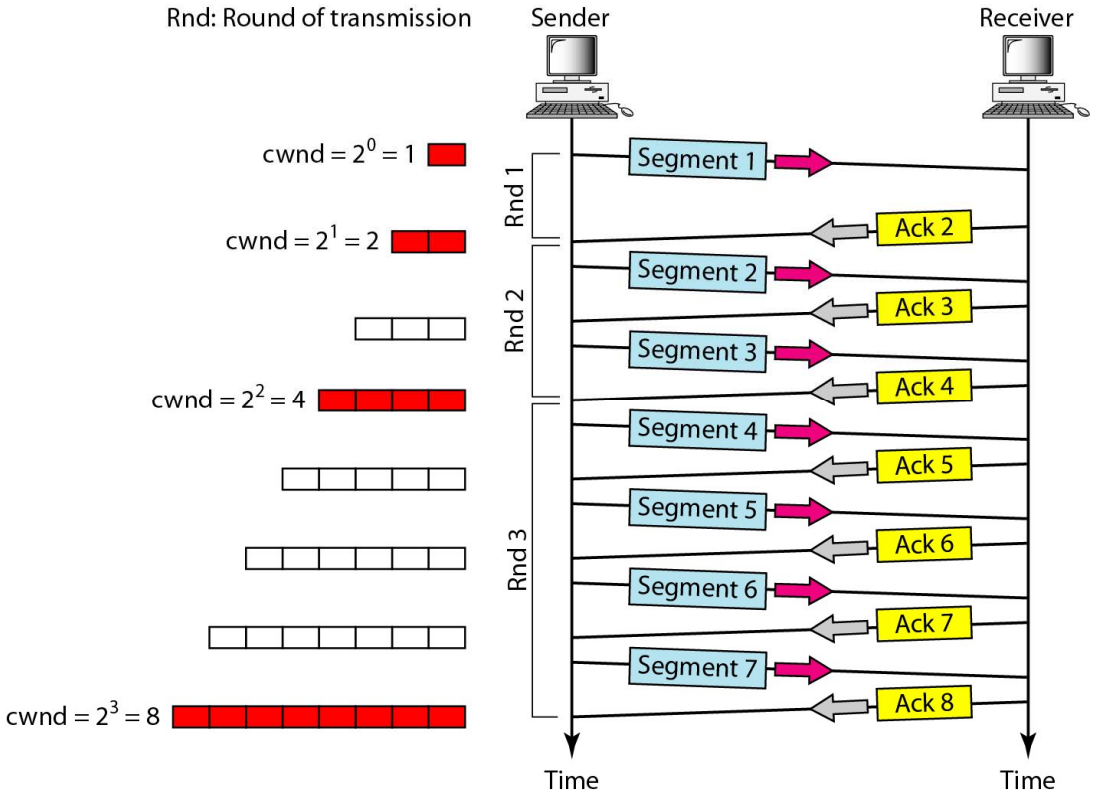


使用ACK时钟，源主机可按照最慢的链路速率来发送，避免在路由器中排队

TCP Congestion Control Algorithms

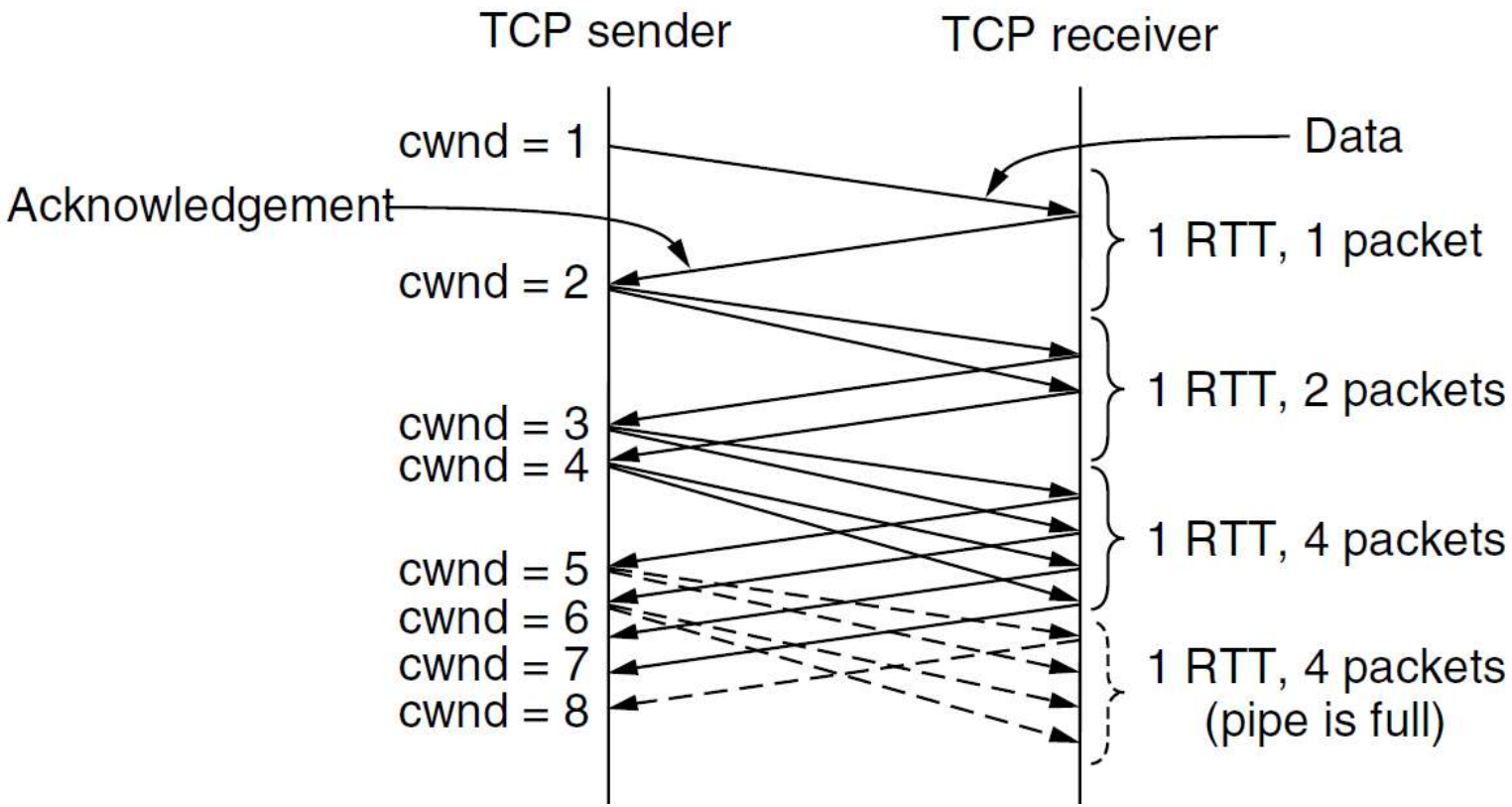
- For simplicity, *number of segments* is used instead of number of bytes
- **Slow-Start**: initial rate is slow (cwnd=1)
  - cwnd+1 upon receipt of ACK
  - Exponential growth fills the network "pipe" in no time
- A more gentle adjustment algorithm: **AIMD**
  - Additive Increase, Multiplicative Decrease
  - Gentle increase, Drastic decrease
    - On receipt of ACK, cwnd += 1/cwnd
    - On detection of packet loss (time out), cwnd = 1
- When to leave Slow Start and enter AIMD?
  - A Threshold is necessary: *ssthresh*

Slow Start: Exponential Increase(Figure 6-44)

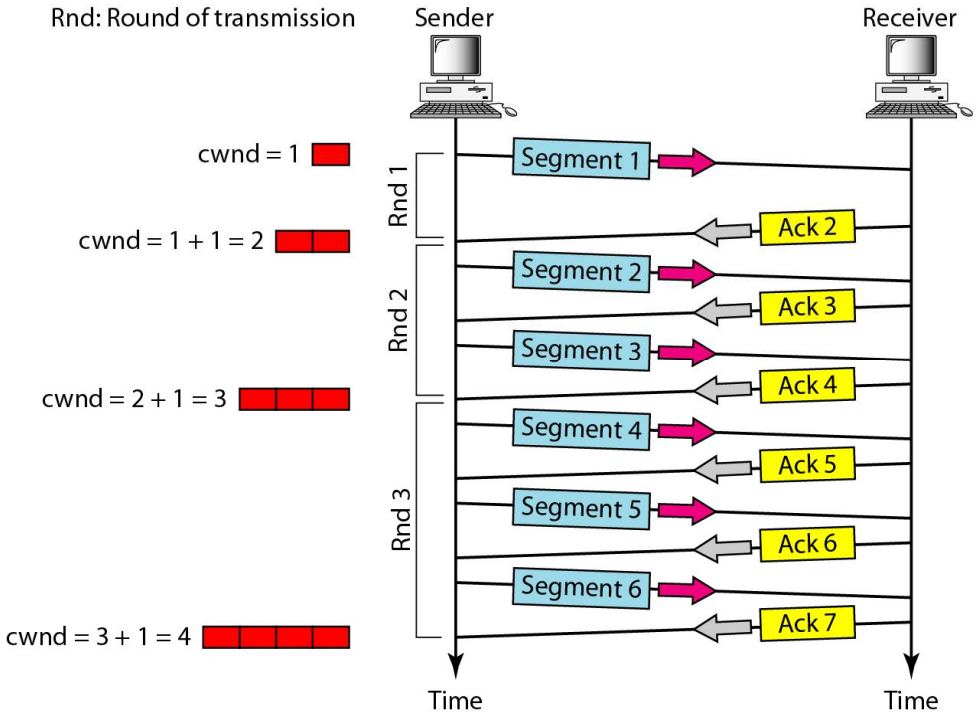


In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Slow start: initial window is 1 MSS

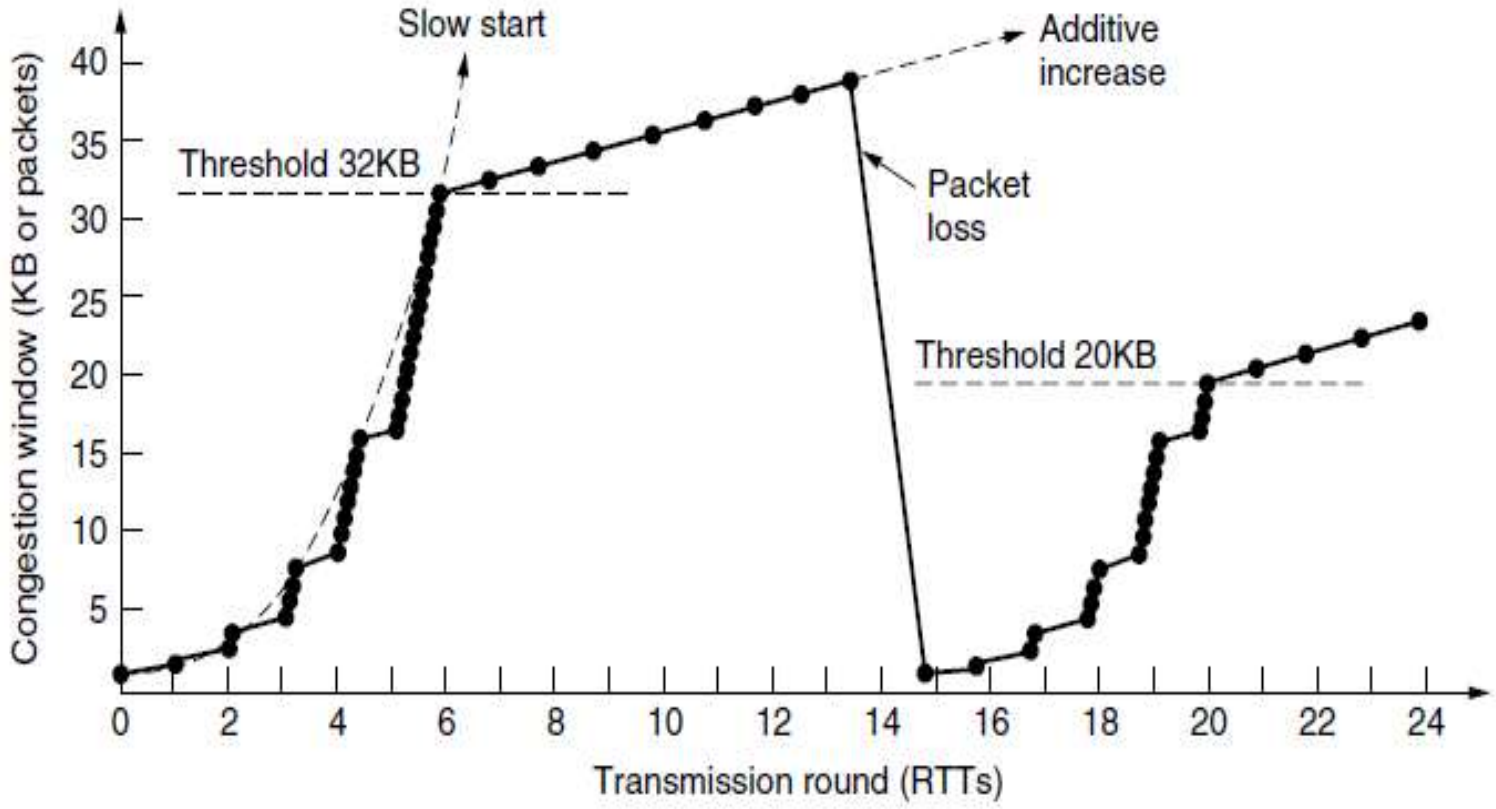


Congestion Avoidance: Additive Increase



In the congestion avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

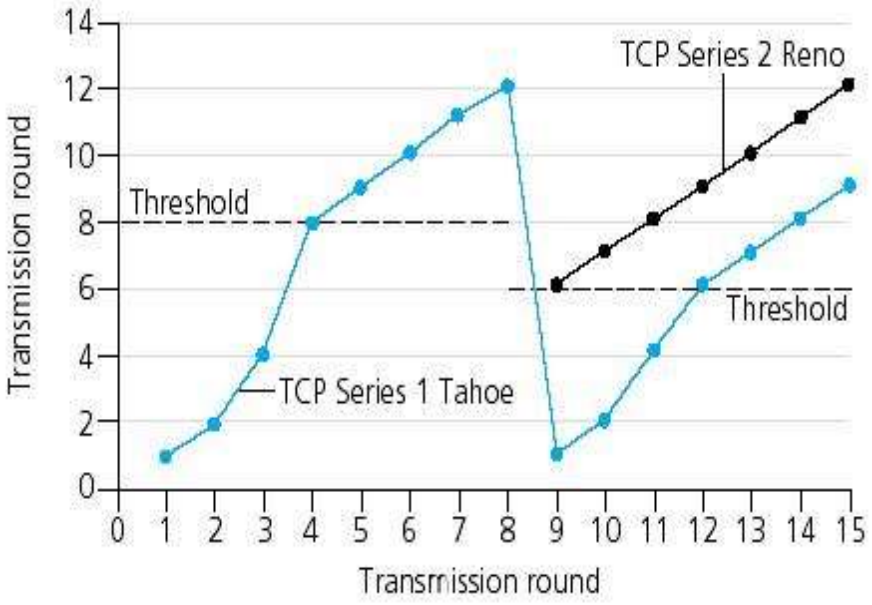
Slow Start and AIMD Example(Figure 6-46)



Fast Retransmission and Fast Recovery

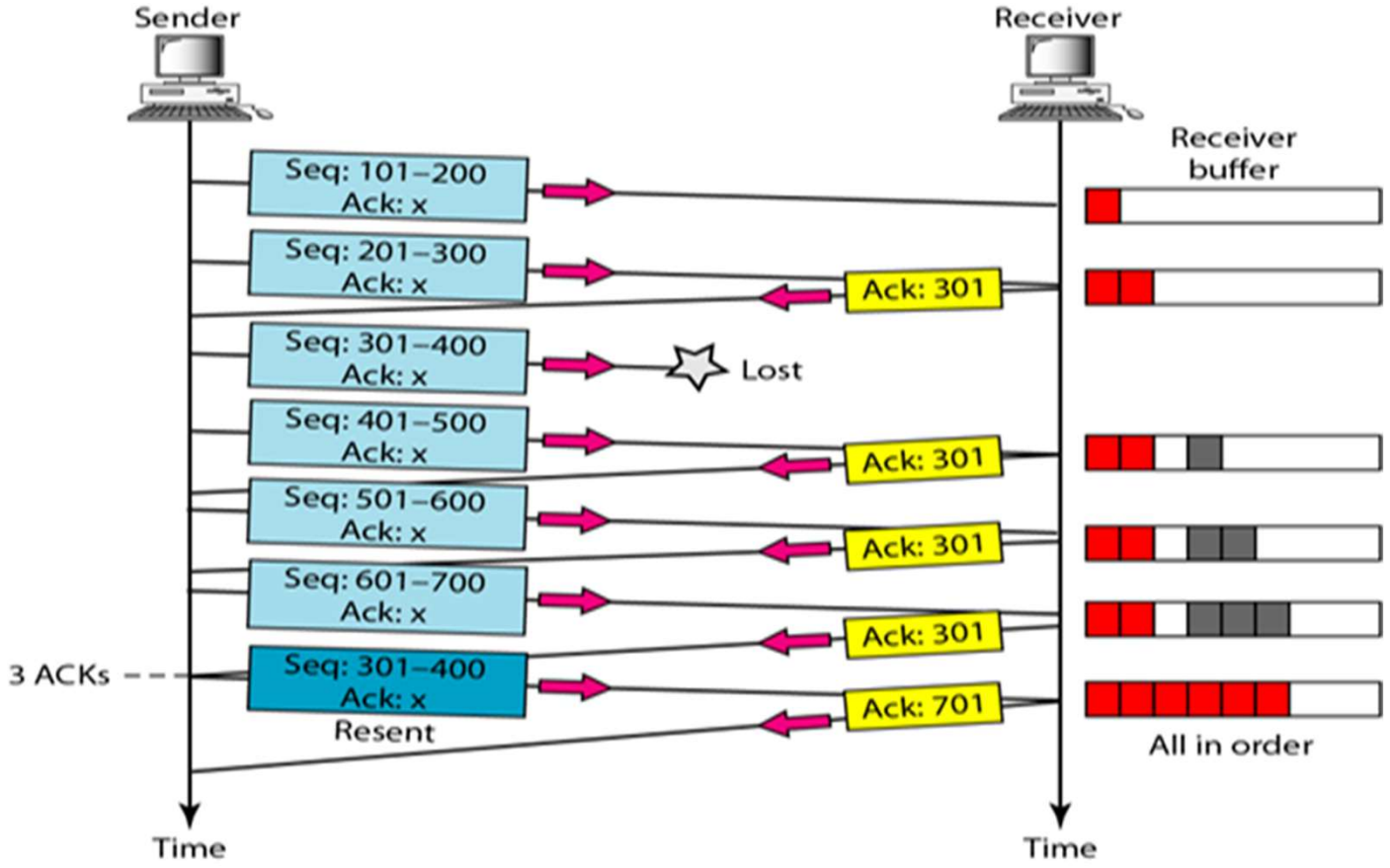
- TCP Tahoe treats 3 duplicate ACKS the same as a timeout, then a new slow start phase starts.
- For 3 duplicate ACKs, TCP Reno will halve the congestion window, perform a "fast retransmit", and enter a new congestion avoidance phase:

  Fast Recovery
  - threshold = $\frac{1}{2}$ window
  - cwnd = threshold



[Kurose]

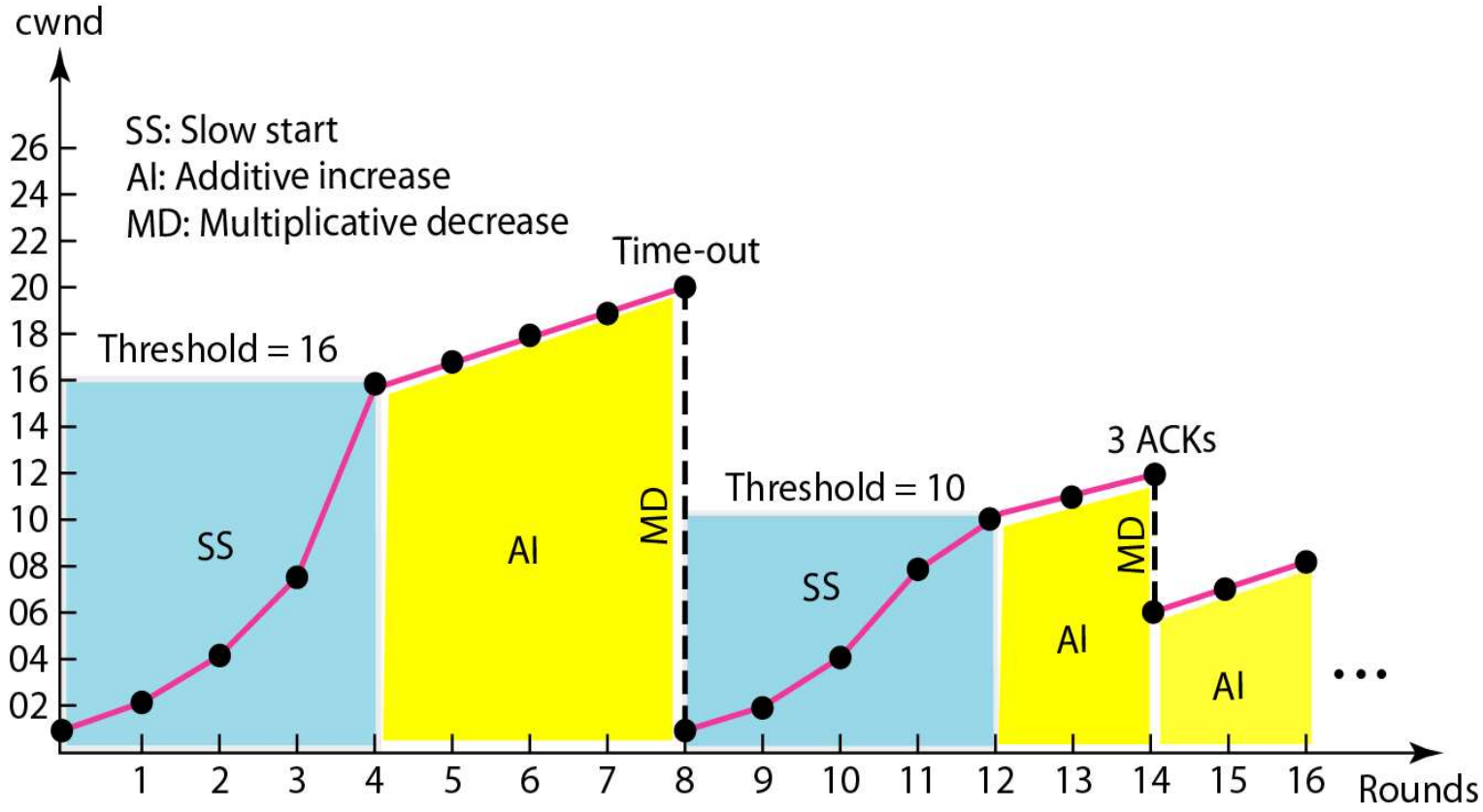
Example of Fast Retransmission(快速重传)



TCP的拥塞控制策略

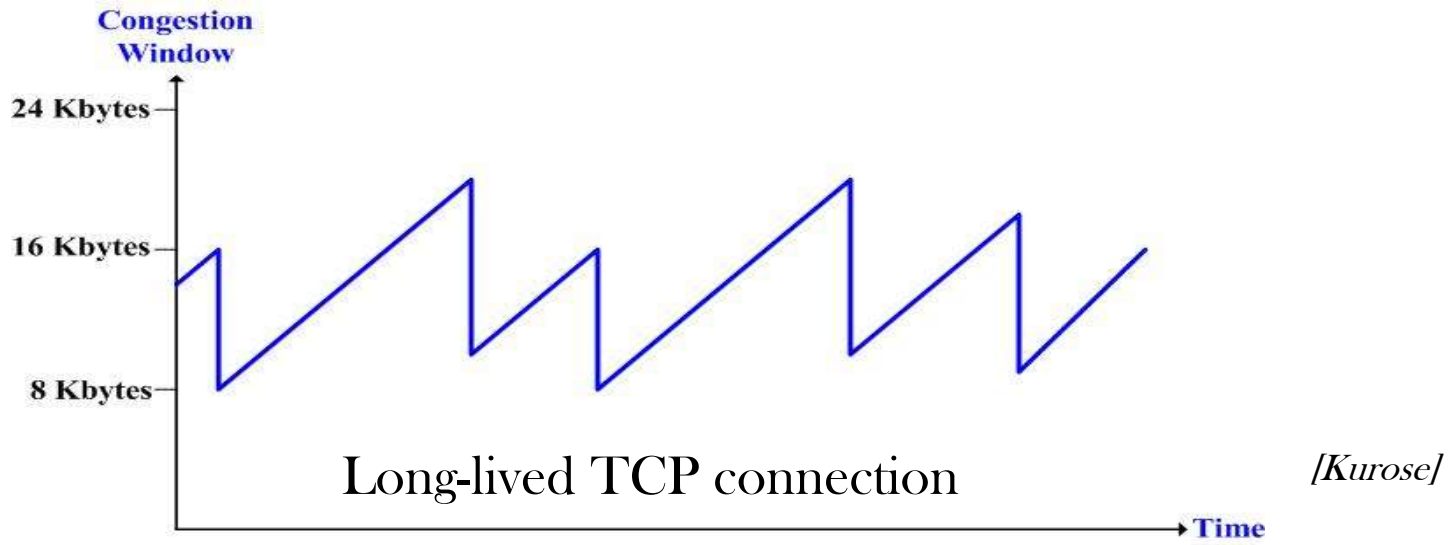
- 拥塞检测：隐式拥塞通知，TCP发送端根据丢包自行判断
- 慢启动：
  - 启动速率很低，TCP连接建立时， 拥塞窗口 = 1 MSS
  - 按指数级增加发送速率（每一轮发送结束，拥塞窗口加倍）
  - 直至拥塞窗口达到阈值或发现数据丢失
- AIMD（拥塞避免）
  - 拥塞窗口达到阈值时，降低发送速度的增长，按线性增长（每一轮发送结束，拥塞窗口增加1个MSS）
  - 出现定时器超时，拥塞窗口降为1个MSS，开始新的慢启动（阈值降为超时时的一半）
  - 收到3个重复的ACK，拥塞窗口降为当前值的一半，开始新的拥塞避免（AI）

TCP Congestion Control Example



AIMD Effect

- Addictive Increase
  - Increasing Congestion Window by 1 when a full window of packets has been sent without loss of packets
  - *Linear increasing* to avoid congestion
- Multiplicative decrease
  - Cutting Congestion Window in *half* after packet loss



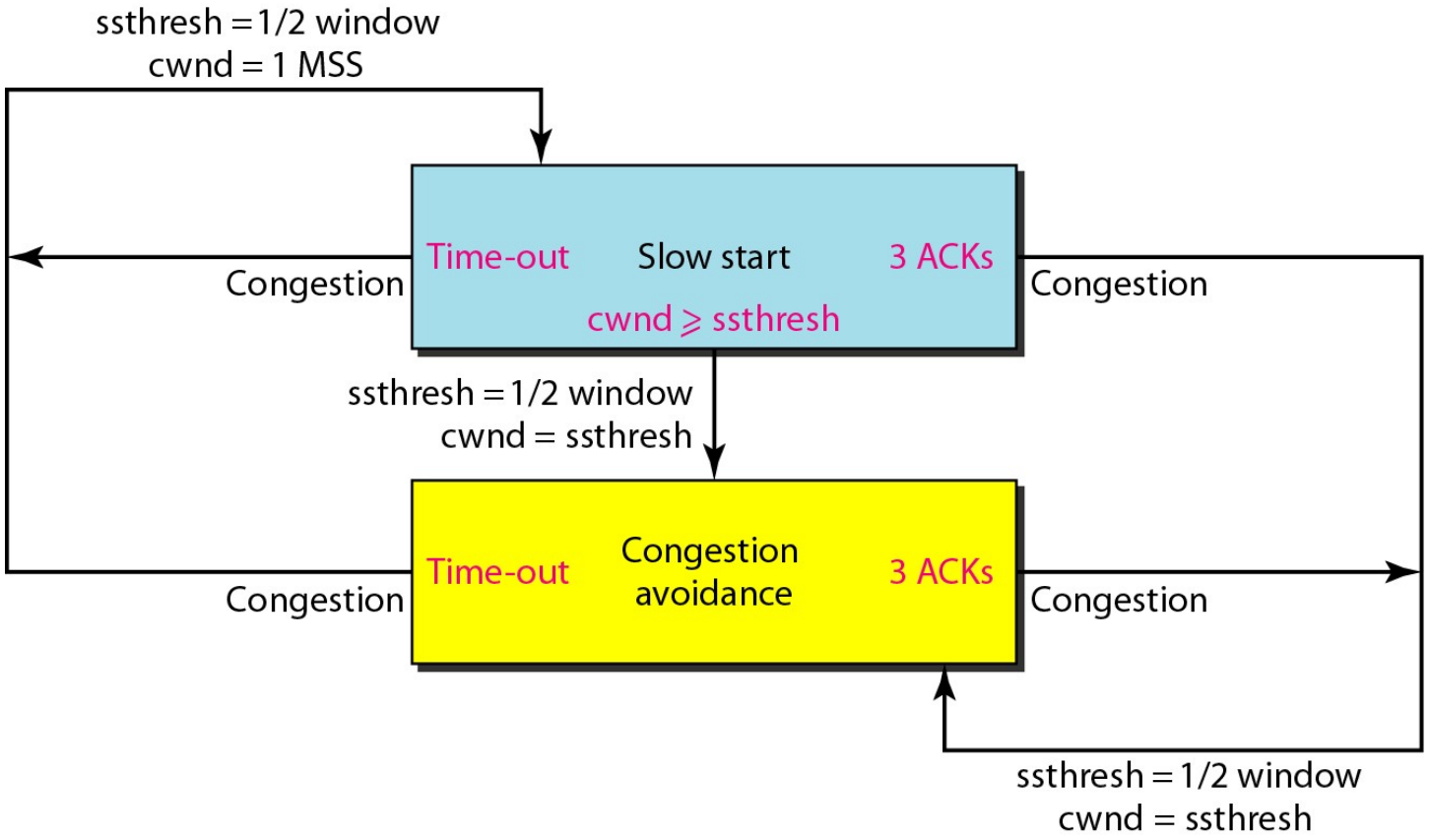
Long-lived TCP connection

[Kurose]

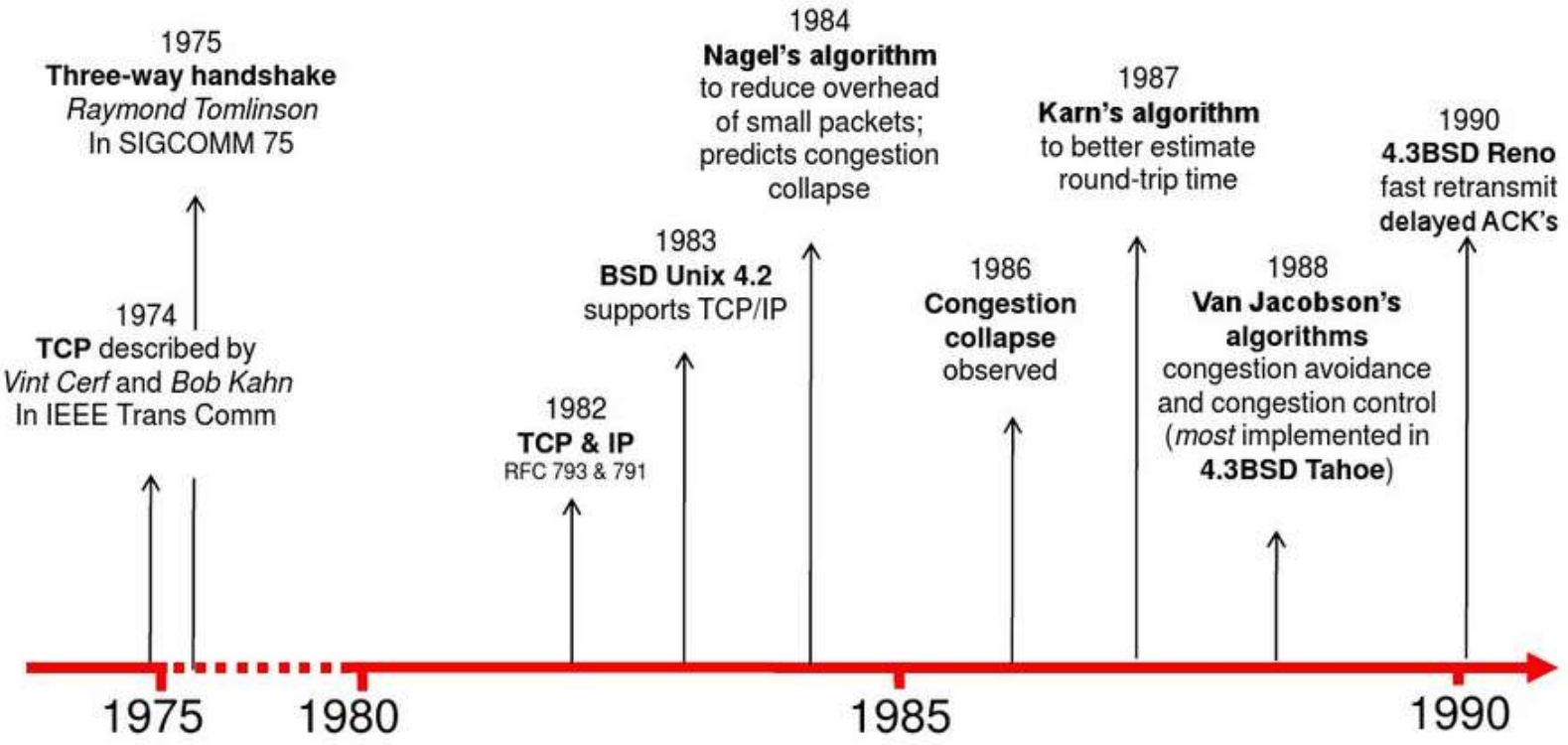
Summary of TCP Congestion Control (1)

- When **Congestion Window(Congwin)** is below **Threshold**, sender in slow start phase, window grows exponentially.
- When **CongWin** is above **Threshold**, sender is in congestion avoidance phase, window grows linearly.
- When a 3-duplicate-ACK occurs, **Threshold** set to **CongWin/2** and **CongWin** set to **Threshold**.
- When timeout occurs, **Threshold** set to **CongWin/2** and **CongWin** is set to 1 MSS.

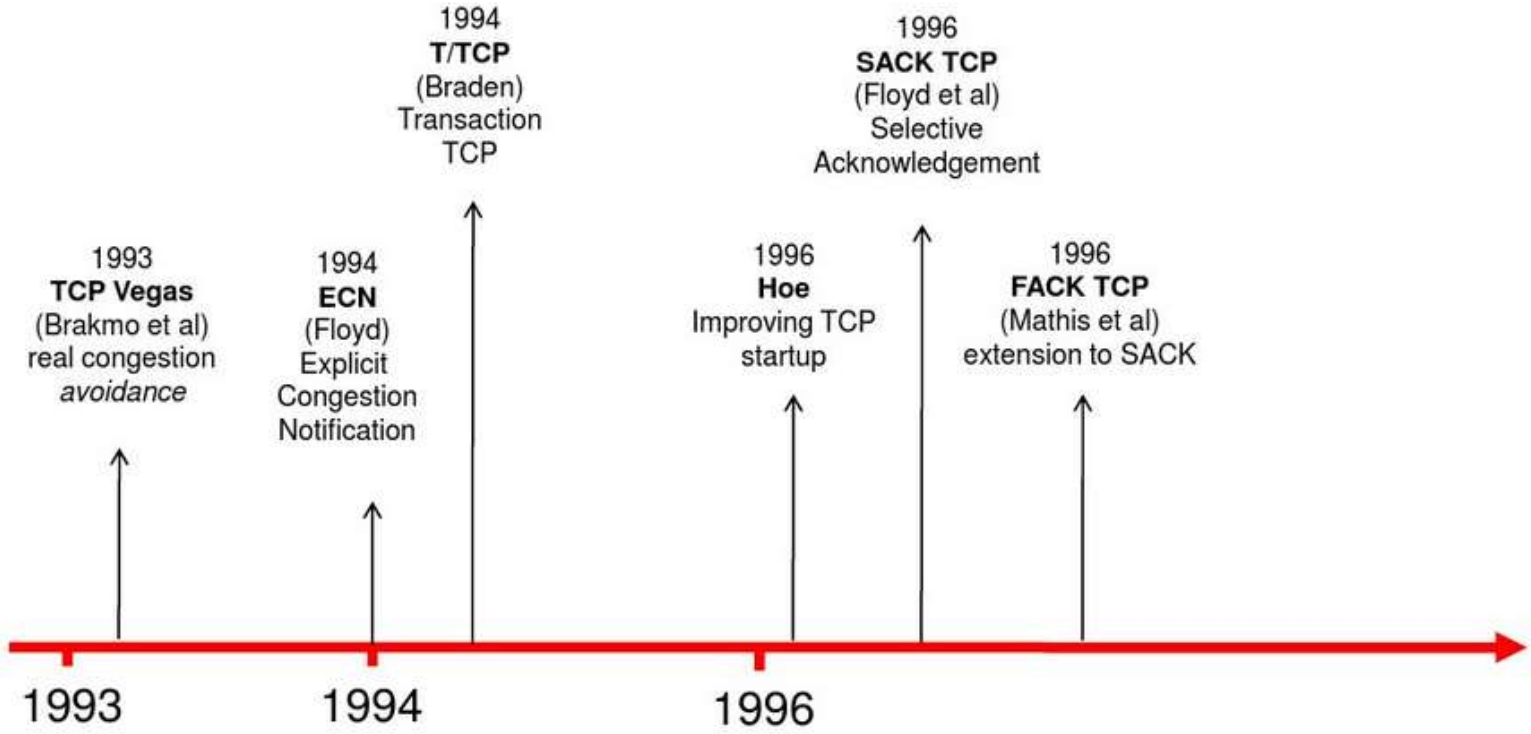
Summary of TCP Congestion Control(2)



TCP Evolution(1)



TCP Evolution(2)



Review of TCP(1)

- 提供面向连接的字节流服务
- 采用三次握手建立连接
- 采用四步释放或三次握手关闭连接
- 采用ARQ机制实现差错控制
- 采用动态的滑动窗口实现流量控制
  - 减少小包，提高传输效率
  - Nagle's算法：发送方使用，一次尽可能发送较多的数据
  - Clark's 算法：接收方使用，在有较大空闲缓存时才发送窗口更新通知，解决傻瓜窗口问题
- 超时重传时间RTO
  - Jacobson算法，基于RTT

$$RTT\ 估值 = \alpha \times RTT旧估值 + (1-\alpha) \times RTT采样值$$

Review of TCP(2)

- 拥塞控制
  - 开始时采用“慢启动”，拥塞窗口初值为1MSS，每轮发送成功之后加倍
  - 达到阈值之后，拥塞窗口按照AI递增，每轮发送成功之后增加一个MSS
  - 一旦出现定时器超时，则采用MD，拥塞窗口降为1，开始新的慢启动，新阈值为超时发生时的拥塞窗口值的一半
  - 当收到3个重复的ACK时，采用快速重传，从当前拥塞窗口值的一半开始新的AI

Key Points(1)

- Functions and Services of Transport Layer
  - Process-to-process delivery
  - Reliable delivery, congestion control
- Design issues
  - Data link layer vs. transport layer
  - Addressing: port number
  - Connection management: 3-way handshaking
  - Congestion control: min-max fairness
- UDP
  - Fast and efficient, best-effort service
  - Multiplexing over IP
- TCP
  - Connection setup: 3-way handshaking
  - Connection release: 3-way handshaking or 4-step
  - Error control: checksum, ARQ, Fast retransmission

Key Points(2): TCP

- Flow control: byte-oriented sliding window
  - sending window=min(recv-window, congestion window)
  - Nagles algorithm: solution at sender
  - Silly Window Syndrome: Clark's solution, solution at receiver
- Estimation of RTT and RTO
- Congestion control
  - Slow Start: congestion window exponentially increases until threshold reached
  - Congestion Avoidance(AI): congestion window linearly increases
  - MD: starting a new Slow Start when ACK times out
  - Fast retransmit: starting a new Congestion Avoidance when 3 duplicated ACKs received

Terms

缩写	英文全称	中文
ACK	Acknowledgement	确认位
	addressing	编址/寻址
AIMD	Additive Increase Multiplicative Decrease	加法增乘法减
API	Application Program Interface	应用编程接口
ARQ	Automatic Retransmission reQuest	自动请求重发
	checksum	校验和/检查和
	Client-Server model	客户/服务器模型
	Congestion Avoidance	拥塞避免
	congestion control	拥塞控制
	congestion window	拥塞窗口
	connectionless service	无连接服务
	connection-oriented service	面向连接的服务

缩写	英文全称	中文
CWR	Congestion Window Reduced	拥塞窗口缩减
	Cumulative ACK	累积ACK
	datagram	数据报
ECE	Explicit Congestion Notification-ECHO	显示拥塞通知-回声
ECN	Explicit Congestion Notification	显式拥塞通知
	Fast Recovery	快速恢复
	Fast Retransmission	快速重传
	flow control	流量控制
FIN	Finish	终止(连接)位
	Internet	因特网

缩写	英文全称	中文
	Max-Min Fairness	最大最小公平性
	Maximum Segment Size	最大报文段长度
MTU	Maximum Transmission Unit	最大传输单元
	Multiplexing/Demultiplexing	复用/解复用(分用)
	packet	分组/包
	process	进程
	Pseudoheader	伪报头
PSH	push	推送(数据)位
QoS	Quality of Service	服务质量
RST	Reset	(连接)重启位

缩写	英文全称	中文
RTT	Round-Trip Time	环回时间
RTO	Retransmission Time-Out	重传超时时间
	TCP segment	TCP报文段
SACK	Selective ACK	选择确认
SAP	Service Access Point	服务访问点
SCTP	Stream Control Transmission Protocol	流控制传输协议
	Service Primitives	服务原语
	Silly Window Syndrome	傻瓜窗口症状
	sliding window	滑动窗口
	Slow Start	慢启动
	Smoothed Round-Trip Time	平滑的环回时间
SYN	Synchronize	(序号)同步位
TCP	Transmission Control Protocol	传输控制协议
UDP	User Datagram Protocol	用户数据报协议
URG	Urgent	紧急位
WAN	Wide Area Network	广域网

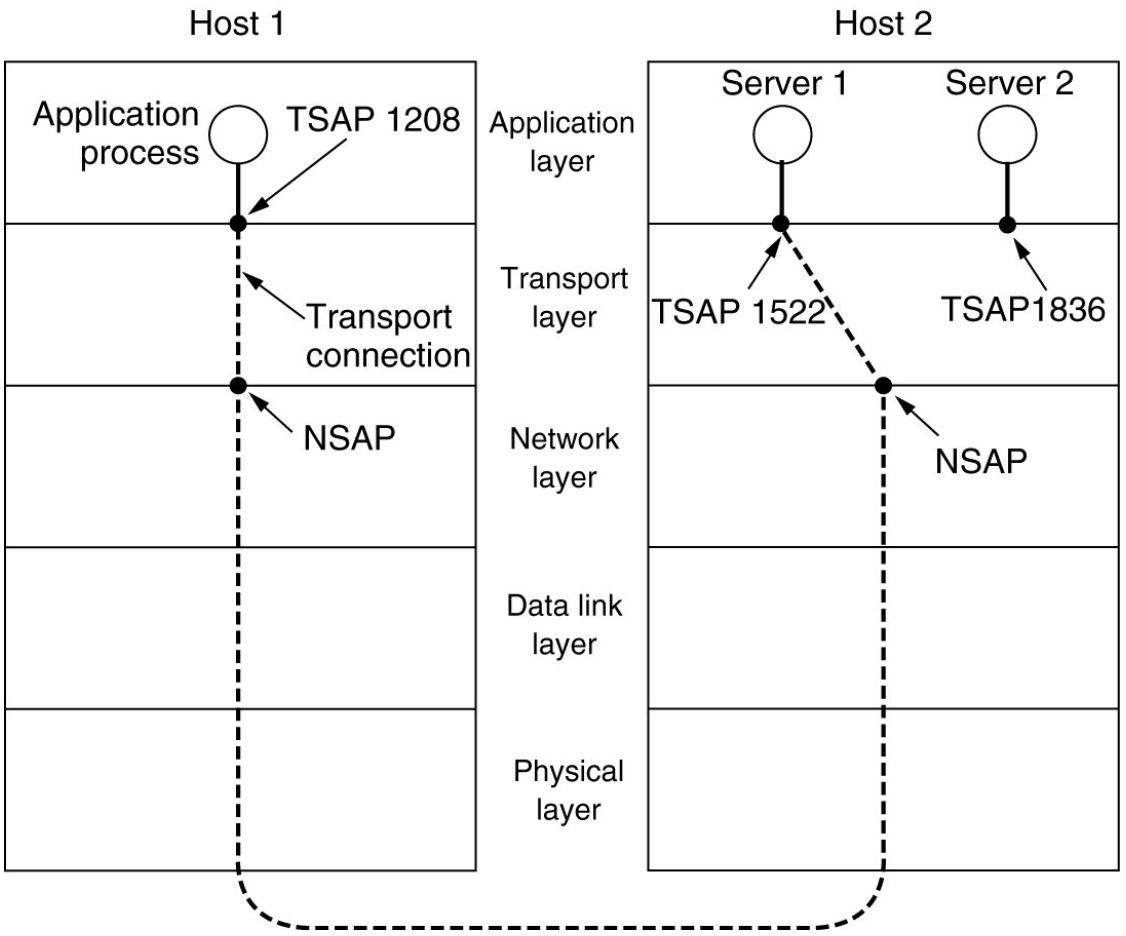
Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
  - Lecture notes of book "Computer Networks, a Top-down Approach", J.Kurose and W.Ross, 3rd Edition, which are marked with *[Kurose]*
  - "Data communications and networking", B.A.Forouzan, which are marked with *[Forouzan]*
  - "计算机网络", 谢希仁，第五版，电子工业出版社，2008年，which are marked with *[谢]*

Self-Study

Addressing

TSAPs, NSAPs and transport connections.



Addressing

- Gets the Server's TSAP
  - Well-known TSAP
  - Look up a name server or directory server
- Process server（节省资源）
  - If there are many server processes, most of which are rarely used, it is wasteful to have each of them active (inetd)
  - 进程服务器作为不常用的服务器的代理
  - 仅适用于服务器进程可按需创建的场合

Process Server

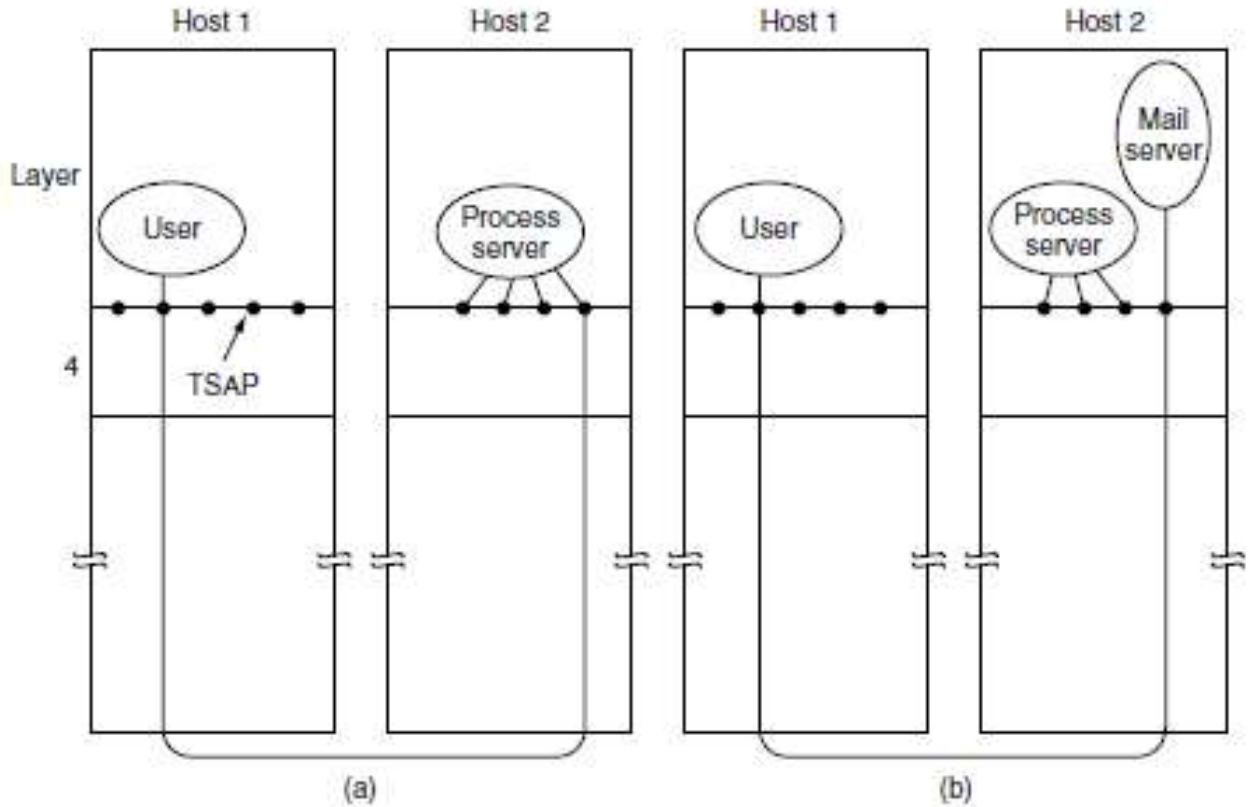


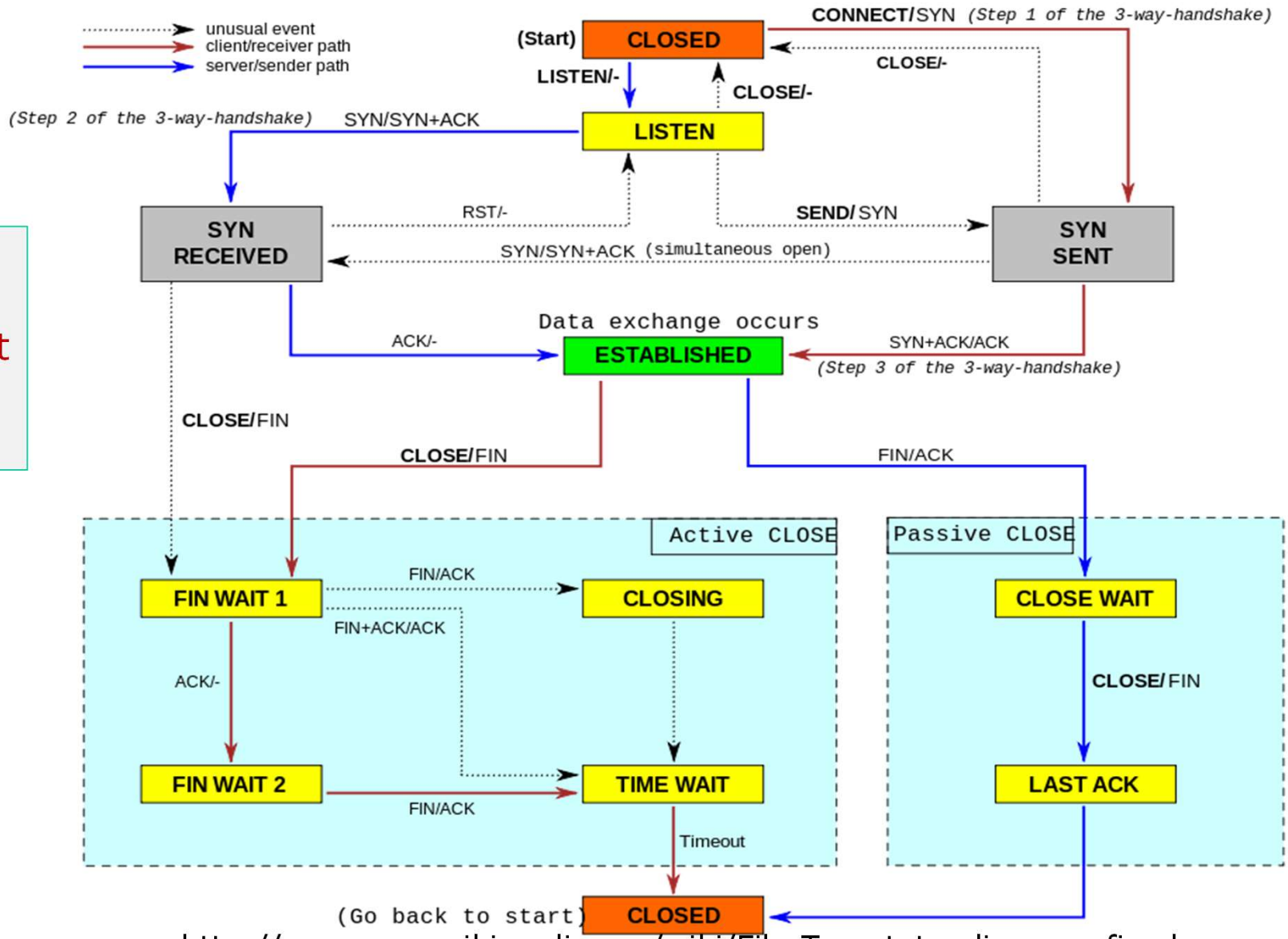
Figure 6-9. How a user process in host 1 establishes a connection with a mail server in host 2 via a process server.

TCP Connection Management Modeling

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

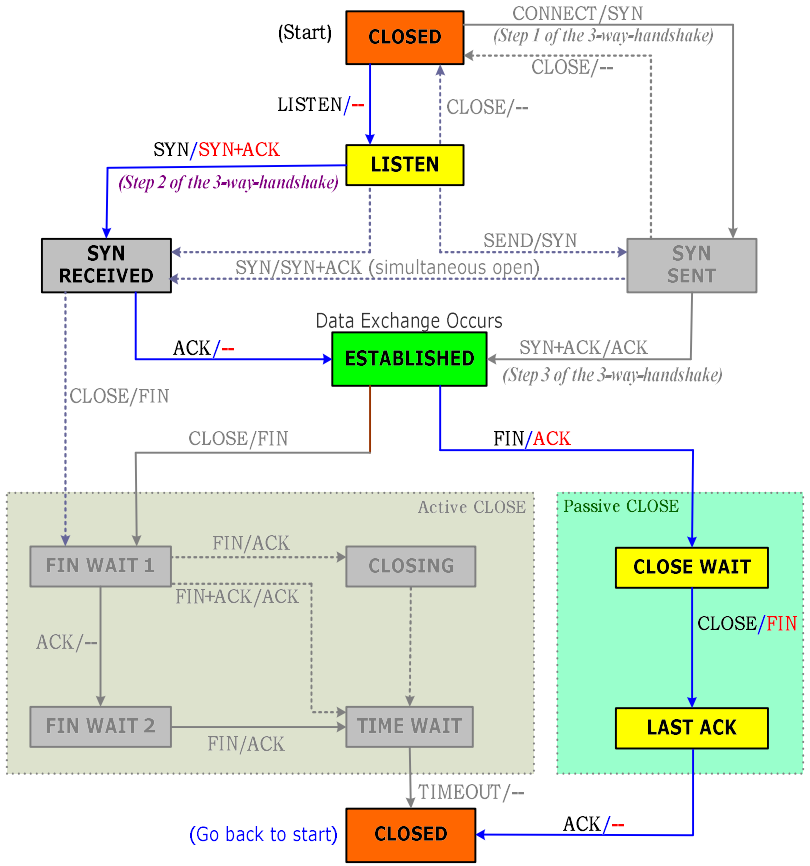
States used in the TCP connection management finite state machine

TCP connection management finite state machine.

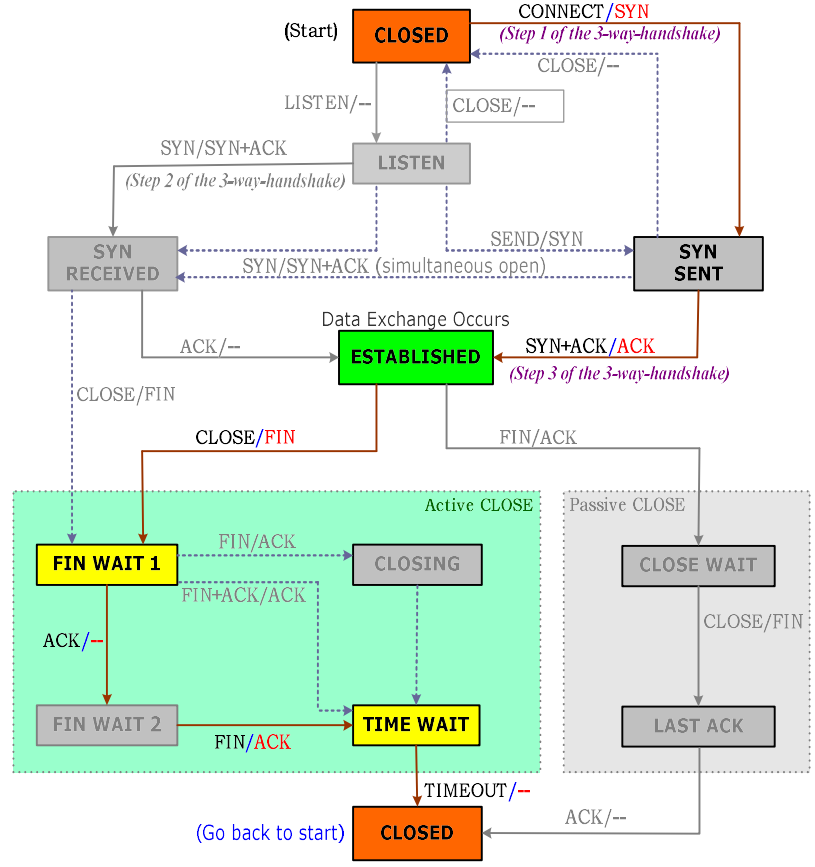


http://commons.wikimedia.org/wiki/File:Tcp_state_diagram_fixed.svg

TCP有限状态机

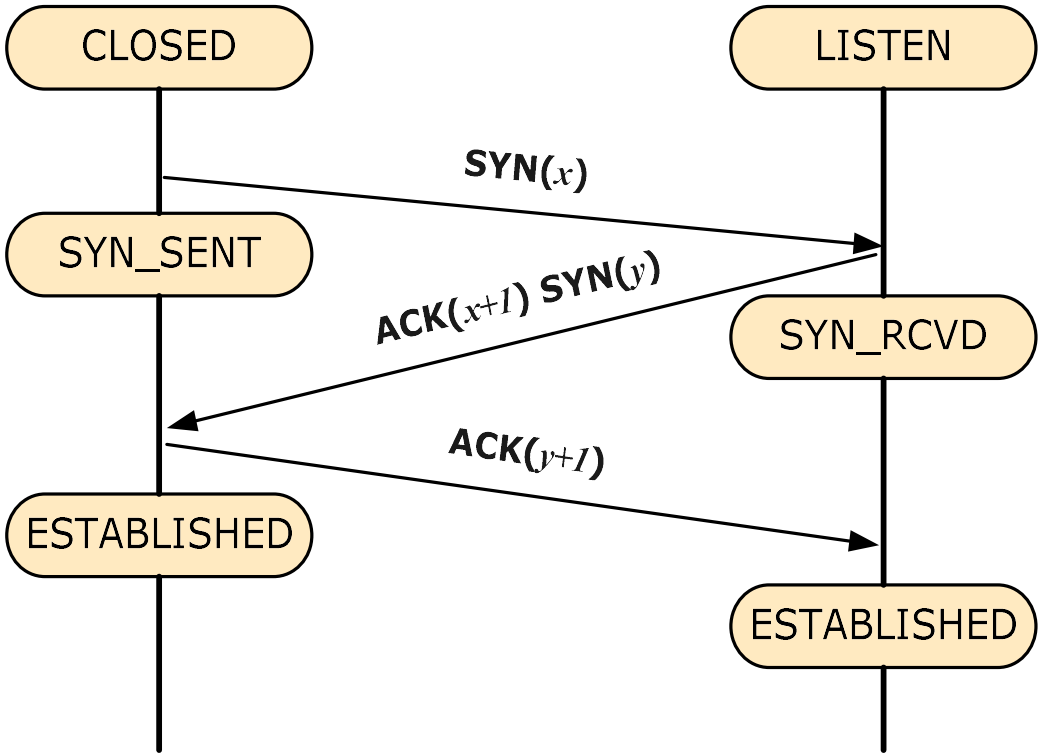


(Server Host)

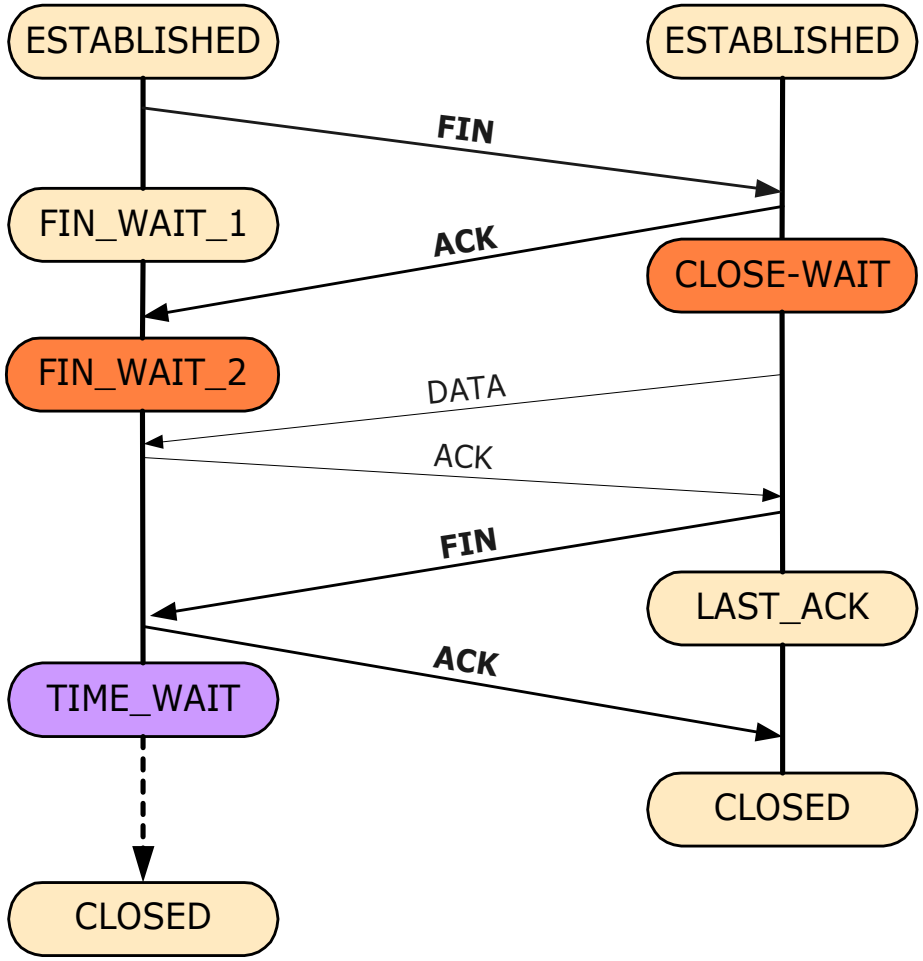


(Client Host)

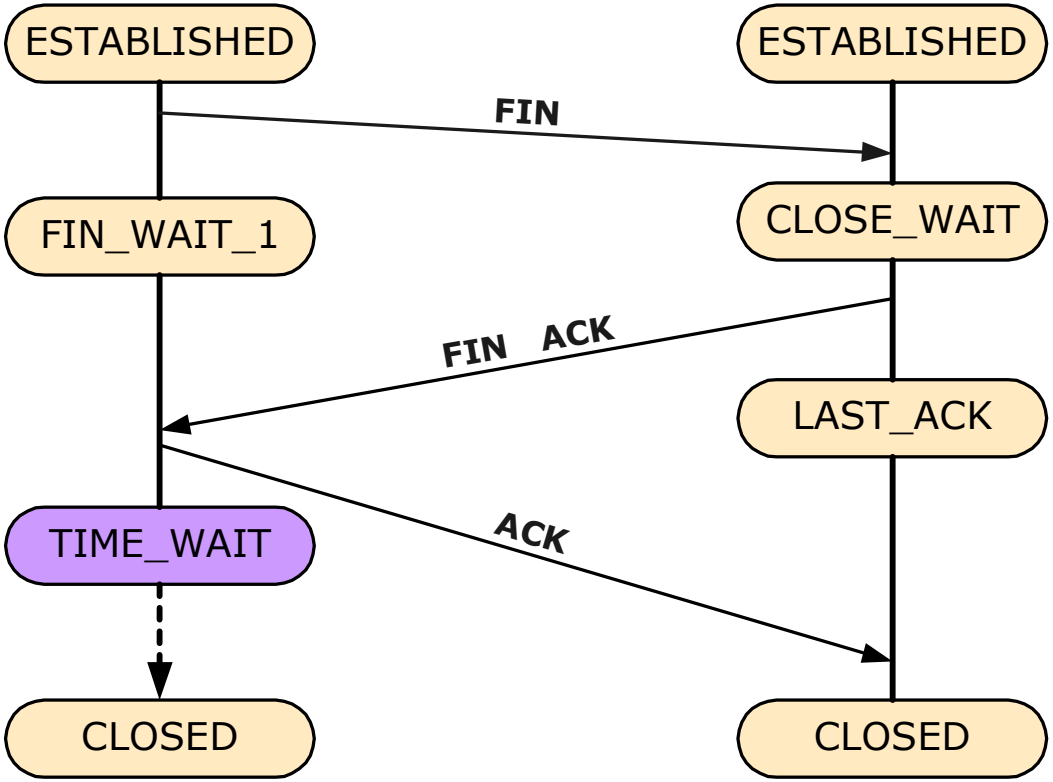
TCP连接建立


CLOSED
LISTEN
SYN(x)
SYN_SENT
ACK(x+1) SYN(y)
SYN_RCVD
ACK(y+1)
ESTABLISHED
ESTABLISHED

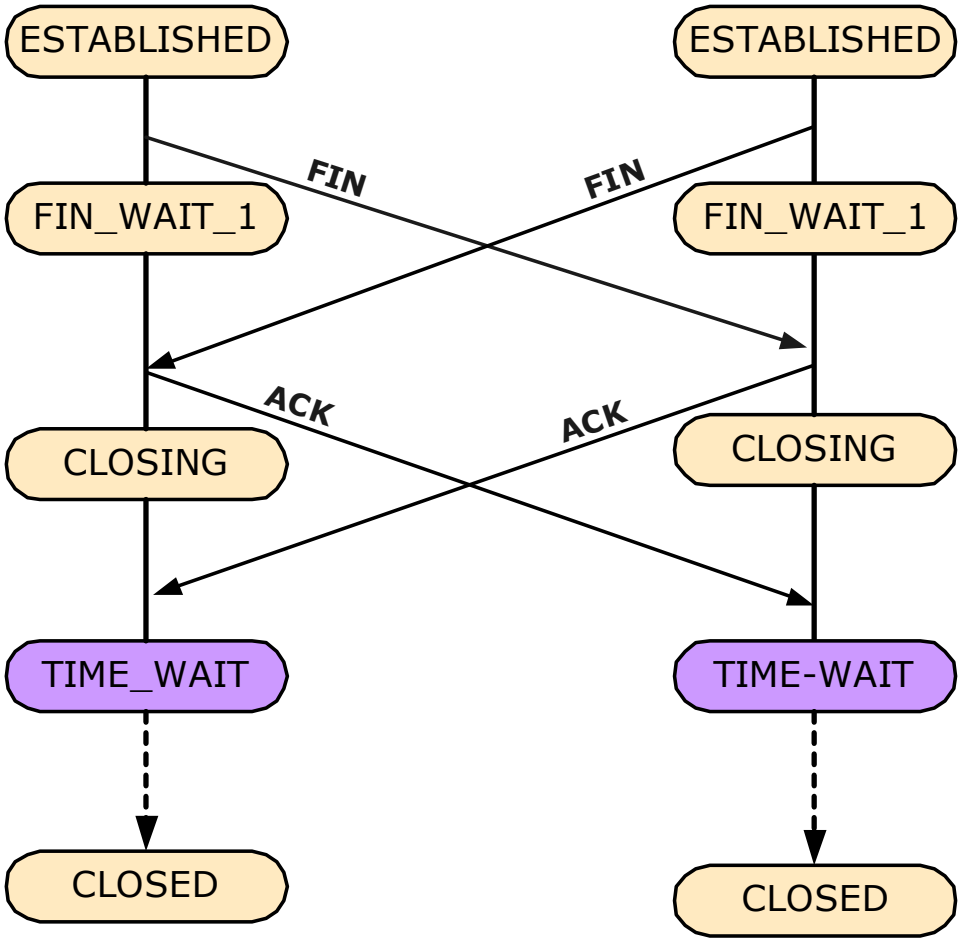
TCP 连接释放 (1)


ESTABLISHED
ESTABLISHED
FIN
FIN_WAIT_1
ACK
CLOSE-WAIT
FIN_WAIT_2
DATA
ACK
FIN
LAST_ACK
ACK
TIME_WAIT
CLOSED
CLOSED

TCP 连接释放(2)


ESTABLISHED
ESTABLISHED
FIN
FIN_WAIT_1
CLOSE_WAIT
FIN ACK
LAST_ACK
TIME_WAIT
ACK
CLOSED
CLOSED

TCP连接释放 (3)


ESTABLISHED
ESTABLISHED
FIN
FIN
FIN_WAIT_1
FIN_WAIT_1
ACK
ACK
CLOSING
CLOSING
TIME_WAIT
TIME-WAIT
CLOSED
CLOSED